

DOSSIER PROJET

Concepteur Développeur d'Applications

VINDHELLFEST

Application web de gestion d'un festival de musique

Aurélien Arnaud

Sommaire

Sommaire.....	2
I. Liste des compétences mises en œuvre.....	6
II. Cahier des charges / Expression des besoins	7
II.1 Contexte et origine du projet.....	7
II.2 Objectifs du projet.....	7
II.3 Périmètre fonctionnel.....	8
II.4 Acteurs et rôles utilisateurs.....	9
II.5 Fonctionnalités attendues.....	9
II.5.1 Espace public.....	9
II.5.2 Espace administration.....	11
II.6 Environnement et choix techniques.....	11
III. Gestion de projet.....	13
III.1 Méthode et organisation	13
III.2 Découpage en tâches / backlog.....	13
III.3 Rétroplanning	14
III.4 Outils de gestion utilisés	15
III.5 Difficultés rencontrées et solutions apportées	15
IV. Spécifications fonctionnelles.....	17
IV.1 Cas d'utilisation	17
IV.1.1 Diagramme de cas d'utilisation global.....	17
IV.1.2 Description des cas d'utilisation principaux.....	17
IV.2 Maquettes et design	19
IV.2.1 Wireframes (basse fidélité)	19
IV.3 Parcours utilisateur.....	19
IV.3.1 Parcours visiteur public.....	19
IV.3.2 Parcours administrateur.....	20
V. Architecture logicielle	22
V.1 Vue d'ensemble de l'architecture.....	22
V.2 Frontend — Next.js 16 (App Router).....	23
V.2.1 Système de types — src/type.ts :	23
V.2.2 Système de styles :.....	23
V.2.3 Couche utilitaire — src/functions/ :	24
V.2.4 Hooks personnalisés — src/hooks/ :	24
V.2.5 Configuration applicative — src/config/ :	25
V.2.6 Composants partagés — src/components/ :	26

V.2.7 Structure des layouts et routage :	26
V.2.8 Gestion des erreurs (frontend) :	26
V.3 Backend — Express.js 5 (API REST).....	27
V.3.1 Système de types — src/type.ts :	28
V.3.2 Fichiers racine — src/	28
V.3.3 Middlewares — src/middlewares/	29
V.3.4 Validation des données — src/schemas/	30
V.3.5 Couche services — src/services/	30
V.3.6 Controllers — src/controllers/.....	30
V.3.7 Routes — src/routes/	31
V.3.8 Gestion des erreurs — src/errors/.....	32
V.4 Base de données — PostgreSQL 16	32
V.5 Infrastructure Docker.....	33
V.6 Pipeline CI/CD — GitHub Actions	34
V.7 Justifications des choix technologiques.....	35
VI. Spécifications techniques	37
VI.1 Structure du projet.....	37
VI.2 modèle Conceptuel de données (MCD)	38
VI.3 modèle Logique de Données (MLD).....	39
VI.4 modèle Physique de Données (MPD)	39
VI.5 API REST — description des endpoints.....	41
VI.5.1 Routes publiques	41
VI.5.2 Authentification	41
VI.5.3 Administration — Artistes.....	42
VI.5.4 Administration — Actualités	42
VI.5.5 Administration — Utilisateurs.....	42
VI.5.6 Contact	43
VI.6 Authentification et gestion des sessions.....	43
VI.7 Upload de fichiers.....	44
VI.8 Envoi d'emails transactionnels	45
VI.9 RGPD et protection des données personnelles.....	46
VI.10 Accessibilité et RGAA	46
VII. Réalisations	48
VII.1 Interface publique.....	48
VII.1.1 Page d'accueil.....	48
VII.1.2 Programmation — liste des artistes	48
VII.1.3 Détail d'un artiste	48

VII.1.4 Actualités — liste et détail	49
VII.1.5 Informations pratiques	49
VII.1.6 Formulaire de contact	49
VII.2 Interface d'administration	49
VII.2.1 Connexion / Déconnexion	49
VII.2.2 Dashboard	50
VII.2.3 Gestion des artistes (CRUD)	50
VII.2.4 Gestion des actualités (CRUD)	50
VII.2.5 Gestion des utilisateurs (CRUD)	51
VII.2.6 Changement de mot de passe	51
VII.3 Backend — composants métier	51
VIII. Sécurité de l'application	52
VIII.1 Authentification JWT et cookies httpOnly	52
VIII.2 Sessions en base de données	52
VIII.3 Contrôle d'accès par rôle (RBAC)	52
VIII.4 Validation des entrées	53
VIII.5 Rate limiting	53
VIII.6 Upload sécurisé	54
VIII.7 Variables d'environnement	54
VIII.8 CORS	54
VIII.9 Hachage des mots de passe	55
IX. Plan de tests et jeu d'essai	56
IX.1 Stratégie de test	56
IX.2 Tests unitaires — backend	56
IX.2.1 Middlewares	56
IX.2.2 Services	57
IX.3 Tests d'intégration — backend (Vitest + Supertest)	58
IX.3.1 Routes publiques	58
IX.3.2 Authentification	58
IX.3.3 Administration	58
IX.4 Tests de composants et fonctions — frontend (React Testing Library)	59
IX.4.1 Composants UI	59
IX.4.2 Hooks	59
IX.4.3 Fonctions utilitaires	59
IX.5 Pipeline CI/CD et exécution automatisée	60
IX.6 Jeu d'essai — Création d'un artiste avec image	60
X. Veille sécurité / Veille technologique	62

XI. Annexes	63
A. Schémas de base de données	63
B. Maquettes de l'application	67
C. Captures d'écran de l'application	71
D. Extraits de code significatifs	75
E. Rapport d'exécution des tests	82
F. Configuration DevOps	84
G. Backlog produit — vue Kanban.....	91
H. Audits Lighthouse et WAVE	93
I. Diagramme de séquence.....	96

I. Liste des compétences mises en œuvre

Le tableau ci-dessous reprend les onze compétences du référentiel Concepteur Développeur d'Applications regroupées par activité-type (AT1 Développer une application sécurisée, AT2 Concevoir et développer une application sécurisée organisée en couches, AT3 Préparer le déploiement d'une application sécurisée), avec la façon dont chacune a été mise en œuvre concrètement et un renvoi vers la section du rapport qui la détaille.

AT	Compétence	Mise en œuvre dans Vindhellfest	Renvoi
AT1	Installer et configurer son environnement de travail en fonction du projet	Monorepo conteneurisé (Docker Compose : frontend, backend, PostgreSQL), dépôt Git avec README, variables d'environnement validées au démarrage (env.ts / Zod).	VI.1, V.5
AT1	Développer des interfaces utilisateur	Interfaces publique et d'administration en Next.js 16 (App Router), intégration des maquettes, composants réutilisables.	VII.1, VII.2, V.2
AT1	Développer des composants métier	Logique métier backend sécurisée (services, contrôleurs Express.js) et composants métier frontend.	VII.3, V.3
AT1	Contribuer à la gestion d'un projet informatique	Backlog Notion, rétroplanning, suivi des difficultés rencontrées et des solutions apportées.	III
AT2	Analyser les besoins et maquetter une application	Cahier des charges, wireframes et maquettes desktop / tablette / mobile.	II, IV.2
AT2	Définir l'architecture logicielle d'une application	Architecture multicouche frontend / backend / API REST / base de données, justification des choix techniques.	V
AT2	Concevoir et mettre en place une base de données relationnelle	Modèle Conceptuel, Logique et Physique de Données, scripts de création PostgreSQL.	VI.2, VI.3, VI.4
AT2	Développer des composants d'accès aux données SQL et NoSQL	Accès aux données via requêtes SQL paramétrées (pg) — le projet n'utilise qu'une base PostgreSQL, aucun composant NoSQL n'entre dans son périmètre.	V.3.5, VI.5
AT3	Préparer et exécuter les plans de tests d'une application	Tests unitaires et d'intégration backend (Vitest + Supertest), tests frontend (React Testing Library), jeu d'essai.	IX
AT3	Préparer et documenter le déploiement d'une application	Conteneurisation Docker (environnements de développement et de production), documentation de l'infrastructure.	V.5, Annexe F
AT3	Contribuer à la mise en production dans une démarche DevOps	Pipeline CI/CD GitHub Actions (lint, tests, build).	V.6, IX.5

II. Cahier des charges / Expression des besoins

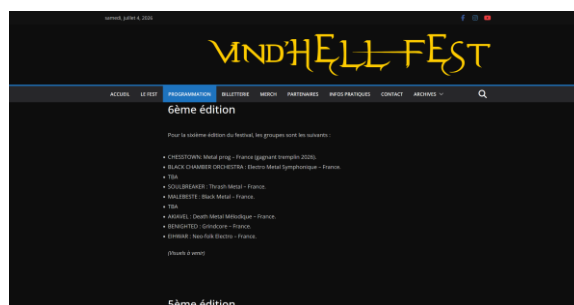
II.1 Contexte et origine du projet

Vindhellfest est un festival de musique organisé en Charente. Fondé sur une programmation rock et métal, il réunit chaque année plusieurs centaines de festivaliers autour de concerts sur deux scènes, la Main Stage et la Tremplin. Fort de cinq éditions réussies, il prépare sa sixième édition pour mai 2027.

Le site officiel (vindhellfest.fr) reposait sur WordPress/PHP, avec un design vieillissant et peu d'interactions, les fonctionnalités se limitaient à une présentation statique du festival. Toute mise à jour de contenu, ajout d'un artiste ou publication d'une actualité nécessitait l'intervention d'un développeur externe, ce qui ralentissait la communication de l'association. C'est ce qui a motivé la décision d'une refonte complète, avec un design moderne et des outils permettant aux membres de l'équipe de gérer eux-mêmes le contenu en ligne.



Ancien site — page d'accueil



Ancien site — page programmation

Cette refonte répond à plusieurs problèmes pour l'association, moderniser son image auprès des festivaliers et des partenaires, gagner en autonomie éditoriale pour publier rapidement les artistes et les actualités, et réduire la dépendance à un prestataire externe pour la maintenance courante du site.

Le développement s'est déroulé de novembre 2025 à juillet 2026 et il constitue le projet principal de fin de formation.

II.2 Objectifs du projet

Le projet Vindhellfest poursuit cinq objectifs principaux.

Modernisation de l'image. Offrir un site web moderne, responsive et attractif, reflétant l'identité visuelle du festival.

Vitrine publique. Permettre aux visiteurs de découvrir la programmation complète, les actualités, les informations pratiques et de contacter l'organisation directement depuis le site.

Administration autonome. Fournir aux membres de l'association une interface d'administration sécurisée pour gérer de façon autonome les artistes, les actualités et les comptes utilisateurs, sans dépendre d'un développeur externe pour les mises à jour courantes.

Concrètement, l'ancien site ne disposait d'aucune page d'actualités, l'association ne communiquait qu'une à deux fois par an, pour annoncer les dates du festival et la programmation. La nouvelle interface d'administration permet de publier une actualité, sans intervention technique, ouvrant la voie à une communication plus régulière.

Gestion différenciée des droits. Mettre en place un système de rôles (admin, artists, news) permettant à chaque membre d'accéder uniquement aux fonctionnalités relevant de ses responsabilités.

Qualité et maintenabilité. Livrer une application structurée, documentée, couverte par des tests automatisés et déployable via Docker, pour faciliter la reprise et l'évolution du projet..

Les cinq objectifs n'ont pas tous le même poids, la priorisation ci-dessous suit une logique MoSCoW.

Must-have :

- Vitrine publique, c'est ce que voient tous les visiteurs et partenaires.
- Administration autonome, gestion des artistes, des news et des administrateurs.
- Qualité et maintenabilité pour faciliter la reprise et l'évolution du projet
- Un test d'accessibilité à l'aide de l'outil WAVE, visant 0 erreur et 0 erreur de contraste. Cette validation ne constitue pas une certification RGAA formelle, mais atteste d'un socle d'accessibilité solide.

Should-have (valeur ajoutée, mais projet livrable sans) :

- Les archives des éditions passées ont été écartées faute de temps sur ce cycle de développement, mais restent envisageables par la suite.
- Les statistiques de fréquentation, de leur côté, sont déjà disponibles via les données de vente de billets fournies par HelloAsso.

II.3 Périmètre fonctionnel

Le périmètre fonctionnel a été délimité avec l'association organisatrice afin de concentrer les efforts sur les besoins prioritaires.

In scope. Site public (accueil, programmation, détail artiste, actualités, informations pratiques, formulaire de contact), espace d'administration sécurisé (authentification, CRUD artistes, CRUD actualités, CRUD utilisateurs), upload d'images, envoi d'e-mails transactionnels, pipeline CI/CD.

Out of scope. La billetterie en ligne reste gérée par HelloAsso, partenaire historique du festival, qui s'occupe également de la vente d'articles physiques sur place pendant l'événement aucune fonctionnalité de paiement n'est donc intégrée à l'application.

Dans les fonctionnalités de l'ancien site non reconduites on retrouve la boutique de merchandising en ligne, car les produits dérivés devront eux aussi être repensés avant qu'une boutique en ligne ait du sens.

Le site est prévu uniquement en français aucune internationalisation n'a été retenue.

Les fonctionnalités écartées (boutique de merchandising) constituent des pistes d'évolution envisageables dans une version ultérieure de l'application, une fois le socle actuel stabilisé.

II.4 Acteurs et rôles utilisateurs

L'application distingue quatre profils d'utilisateurs, chacun avec un périmètre d'action différent.

Visiteur public. Toute personne accédant au site sans authentification. Il peut consulter toutes les pages publiques, accueil, programmation, actualités publiées, informations pratiques, et envoyer un message via le formulaire de contact.

Administrateur (rôle admin). Membre de l'association disposant d'un accès complet à l'espace d'administration: Gestion des artistes, des actualités et des comptes utilisateurs. Seul rôle habilité à créer, modifier ou supprimer des comptes.

Gestionnaire artistes (rôle artists). Membre responsable de la programmation. Accès limité à la création, la modification et la suppression des artistes et de leurs concerts associés.

Gestionnaire actualités (rôle news). Membre responsable de la communication. Accès limité à la création, la modification et la suppression des actualités.

Ces trois rôles authentifiés sont portés par une colonne unique de type ENUM sur la table users dans la BDD, un compte ne peut donc avoir qu'un seul rôle à la fois. Un membre devant cumuler les responsabilités « artists » et « news » se voit attribuer le rôle admin.

II.5 Fonctionnalités attendues

II.5.1 Espace public

L'espace public regroupe l'ensemble des pages accessibles sans authentification.

Page d'accueil. Présentation des artistes mis en avant par l'organisation du festival (deux maximum) et des deux dernières actualités publiées, avec un lien vers la billetterie HelloAsso.

Programmation. Liste de l'ensemble des artistes avec leur scène et leur horaire de concert, accessible sans authentification.

Détail d'un artiste. Page dédiée présentant la biographie, le genre musical, l'origine, les liens YouTube et Spotify, ainsi que les horaires complets du concert associé.

Actualités. Liste des articles publiés triés par date, avec accès à la page de détail de chaque article (titre, contenu, image, auteur, date).

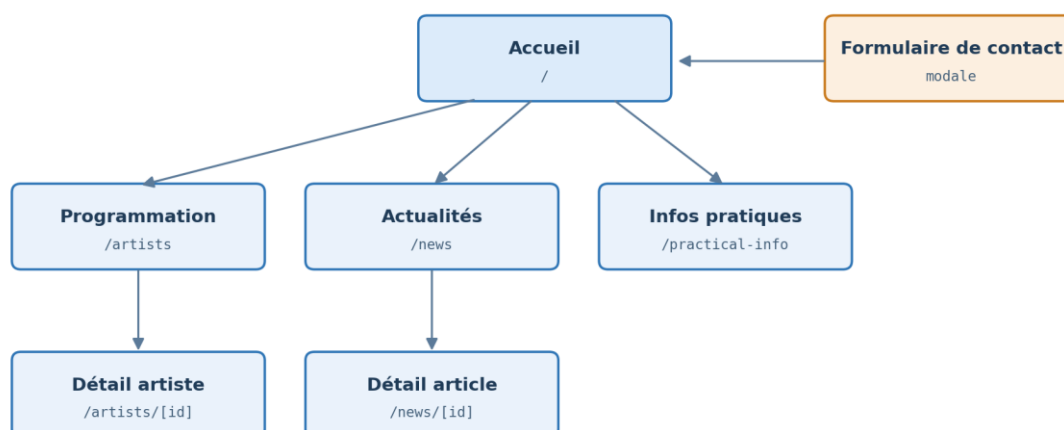
Détail d'une actualité. Page dédiée présentant le titre, l'image, le contenu complet, l'auteur et la date de publication.

Informations pratiques. Page statique présentant les modalités d'accès au site, les options d'hébergement et les informations utiles pour les festivaliers.

Formulaire de contact. Formulaire permettant à tout visiteur d'envoyer un message à l'organisation. Le message est transmis par e-mail via SMTP. Les champs (nom, e-mail, sujet, message) sont validés côté frontend et backend.

Pour l'édition actuelle, ni plafond d'affichage ni pagination ne sont mis en place, le volume d'artistes et d'actualités reste limité. Si la programmation ou le nombre d'actualités venait à s'étoffer lors des prochaines éditions, une pagination ou un chargement progressif deviendrait nécessaire, cette évolution est identifiée comme une amélioration future.

Les pages publiques disposent chacune de leurs propres métadonnées qui affichent un titre et une description propres à chaque entité plutôt qu'un intitulé générique partagé. Les balises Open Graph et Twitter Card sont également définies et génèrent un aperçu de partage fonctionnel sur les réseaux sociaux, mais uniquement au niveau du layout racine. Plusieurs pistes d'amélioration restent ouvertes ; rendre les balises Open Graph et Twitter Card dynamiques par page (reprendre le titre et la photo propres à chaque fiche artiste ou actualité, à l'image de ce qui a été fait pour title/description), ajouter une directive robots/noindex sur les pages d'administration afin d'empêcher leur indexation par les moteurs de recherche, et créer un sitemap.xml ainsi qu'un robots.txt absents à ce jour.



Toutes les pages sont accessibles sans authentification.
Le formulaire de contact est une modale accessible depuis l'accueil et le menu de navigation.

Figure — Plan du site, espace public

II.5.2 Espace administration

L'espace d'administration regroupe les fonctionnalités réservées aux membres authentifiés de l'association.

Authentification. Connexion par e-mail et mot de passe. Réinitialisation du mot de passe possible depuis la page de connexion. Changement de mot de passe obligatoire à la première connexion.

Gestion des artistes. Création d'un artiste avec upload d'image. Attribution d'une scène (MainStage ou Tremplin) et d'un horaire de concert. Possibilité de mettre en avant au maximum deux artistes sur la page d'accueil. Modification et suppression accessible aux rôles admin et artists.

Gestion des actualités. Création d'un article avec upload d'image, titre, contenu et statut (brouillon ou publié). Les brouillons ne sont visibles que par les utilisateurs authentifiés avec les rôles admin ou news. Modification et suppression accessible aux rôles admin et news.

Gestion des utilisateurs. Création d'un compte avec attribution d'un rôle (admin, artists, news) et envoi automatique du mot de passe provisoire par e-mail. Modification des informations et du rôle. Suppression des comptes. Réservé au seul rôle admin.

Tableau récapitulatif des droits par rôle

Fonctionnalité	admin	artists	news
Gestion des artistes (CRUD + upload image)	✓	✓	—
Mise en avant des artistes (is_featured)	✓	✓	—
Gestion des actualités (CRUD + brouillon/publié)	✓	—	✓
Gestion des utilisateurs (CRUD + attribution de rôle)	✓	—	—
Réinitialisation et changement de mot de passe	✓	✓	✓

✓ = accès autorisé — = accès non autorisé

II.6 Environnement et choix techniques

Stack technique. Next.js 16 (App Router), React 19, TypeScript et Tailwind CSS 4 côté frontend ; Express.js 5 et Node.js 20 côté backend ; PostgreSQL 16 pour la persistance des données.

Containerisation. L'application est orchestrée via Docker Compose avec trois services (db, backend, frontend) sur un réseau interne. Le démarrage est conditionné à la disponibilité de PostgreSQL (healthcheck).

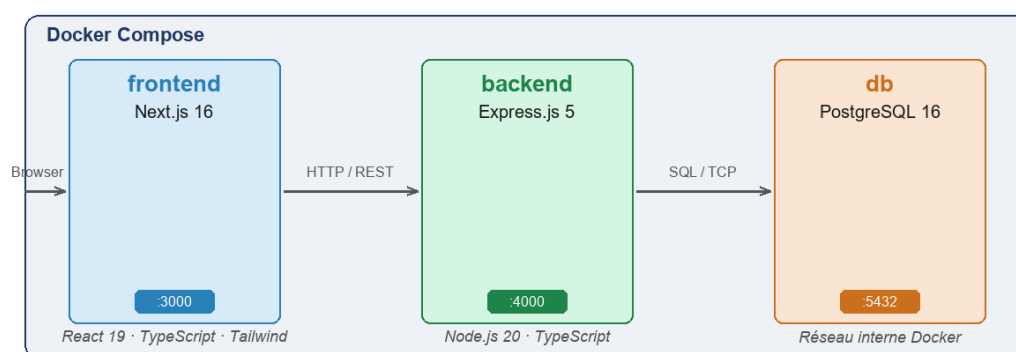


Figure — Architecture Docker Compose : trois services sur réseau interne

Sécurité. Authentification par JWT stockée en cookie httpOnly, sessions persistées en base de données. Validation des entrées par Zod côté backend. Hachage des mots de passe par bcrypt. TypeScript strict, sans any.

Tests. Couverture par des tests unitaires (Vitest) et des tests d'intégration (Vitest + Supertest côté backend, Vitest + React Testing Library côté frontend).

CI/CD. Pipeline GitHub Actions automatisé sur chaque push et pull request, lint ESLint et exécution des tests avec pour les tests d'intégration backend une base PostgreSQL réelle en environnement CI.

III. Gestion de projet

III.1 Méthode et organisation

Le projet a été mené en autonomie, en dehors de toute structure d'entreprise, dans le cadre de la formation CDA de l'école Hexagone. Cette configuration a nécessité une organisation personnelle et une autonomie dans la prise de décisions techniques et de gestion, du cahier des charges jusqu'à la mise en production.

Le projet a été découpé en quatre sous-projets interdépendants.

L'infrastructure Docker et le pipeline CI/CD forment l'enveloppe commune qui orchestre les trois autres, la base de données, le backend et le frontend.

Chaque sous-projet dispose de sa propre documentation (README.md, API.md, bd/README.md) et de ses propres tests, ce qui lui confère une autonomie de livraison claire.

La définition précise du périmètre de chaque sous-projet en amont a permis de développer le backend et le frontend en parallèle.

Le suivi de l'avancement repose sur un backlog tenu dans Notion, organisé sous forme de tableau kanban. L'avancement de chaque user story est suivi à travers cinq états, À faire, En cours, En révision, Terminé, Bloqué.

En parallèle, un rétroplanning, également tenu dans Notion, découpe le projet en cinq phases, la conception, l'infrastructure et la base de données, le backend, le frontend, la rédaction de la documentation technique.

Le code est versionné avec Git et hébergé sur GitHub, un pipeline GitHub Actions exécute automatiquement le linting et les tests à chaque push et pull request.

III.2 Découpage en tâches / backlog

Le backlog produit est tenu dans Notion, sous la forme de tableau kanban. Chaque carte représente une user story et est rattachée à une couche fonctionnelle (Authentification, Présentation, Programme & Artistes, Contact, Administration, Tests, DevOps), une ou plusieurs couches techniques (Infrastructure, Base de données, Backend, Frontend, Qualité & Docs), une priorité (P1, P2, P3) et une estimation de complexité. Chaque carte détaille une liste de critères d'acceptation sous forme de checklist, ainsi que des notes techniques précisant les fichiers, endpoints et composants concernés.

Le backlog complet (toutes les cartes, avec leurs critères d'acceptation et notes techniques) reste consultable directement dans Notion seul un exemple de carte est reproduit ci-dessous à titre d'illustration.

Connexion d'un utilisateur

Complexité 3

🔗 Couche fonction... **Authentification**

🔗 Couche technique **Backend** **Frontend**

🔗 Priorité **P1**

🔗 User Story En tant qu'utilisateur inscrit, je veux me connecter avec mon email et mon mot de passe afin d'accéder à mon espace personnel.

🔗 État **Terminé**

+ Add a property

Commentaires

Ajouter un commentaire...

Critères d'acceptation

- ✓ Formulaire avec champs **email** et mot de passe (labels **sr-only** pour l'accessibilité)
- ✓ Bouton de soumission désactivé si un champ est vide ou si la requête est en cours
- ✓ Message d'erreur générique affiché sous le formulaire en cas d'échec
- ✓ Redirection vers **/admin/dashboard** après connexion réussie
- ✓ Bouton "Mot de passe oublié" ouvrant une modale dédiée
- ✓ La modale "Mot de passe oublié" envoie un nouveau mot de passe par **email**
- ✓ Cookie **httpOnly** **JWT** créé et session enregistrée en base après connexion

Notes techniques

- Page : **src/app/(auth)/login/page.tsx** — composant client
- **Endpoint** connexion : **POST /admin/auth/login**
- **Endpoint** mot de passe oublié : **POST /admin/auth/forgot-password**
- **Backend** : **controllers/admin/auth/login.controller.ts**
- Session : table **sessions** (user_id, expires_at)
- Cookie : **httpOnly**, configuré via **COOKIE_ACCESS_TOKEN *** dans **.env.backend**
- Modale : composant **ForgotPassword** via **react-modal**

Figure — Exemple de carte backlog (user story « Connexion d'un utilisateur ») : propriétés, critères d'acceptation et notes techniques

III.3 Rétroplanning

Le projet s'est déroulé, du 3 novembre 2025 au 12 juillet 2026, découpé en cinq phases chronologiques et deux jalons, suivis dans Notion

La phase 1 Conception (3 au 21 novembre 2025) couvre l'analyse des besoins, la rédaction du cahier des charges, la modélisation MCD/MLD/MPD de la base de données, ainsi que les zonings et wireframes des principales pages.

La phase 2 Infrastructure et BDD (24 novembre au 19 décembre 2025) met en place l'environnement Docker Compose, les scripts SQL d'initialisation, le pipeline CI/CD GitHub Actions, ainsi que la configuration TypeScript strict, ESLint et Prettier.

Les phases 3 et 4 se chevauchent volontairement. La phase 3 Backend (5 janvier au 30 avril 2026) débute en premier afin d'initialiser le serveur et d'exposer les premiers endpoints REST, l'authentification JWT et les sessions, l'upload d'images et l'envoi d'e-mails, ainsi que la suite de tests backend. La phase 4 Frontend (19 janvier au 31 mai 2026) démarre deux semaines plus tard, dès que l'API est suffisamment stable pour être consommée, et se poursuit après la fin de la phase 3 pour les pages entièrement client ; elle couvre le développement des interfaces publique et administration, les design tokens Tailwind CSS 4 et les tests frontend.

La phase 5 Documentation et dossier (1er juin au 12 juillet 2026), est consacrée à la rédaction des documentations techniques, à la rédaction du dossier de projet, à sa relecture.

Deux jalons encadrent la fin du projet, la date limite de remise du dossier, fixée au 12 juillet 2026, et la soutenance orale du titre professionnel CDA, prévue du 27 au 30 juillet 2026.

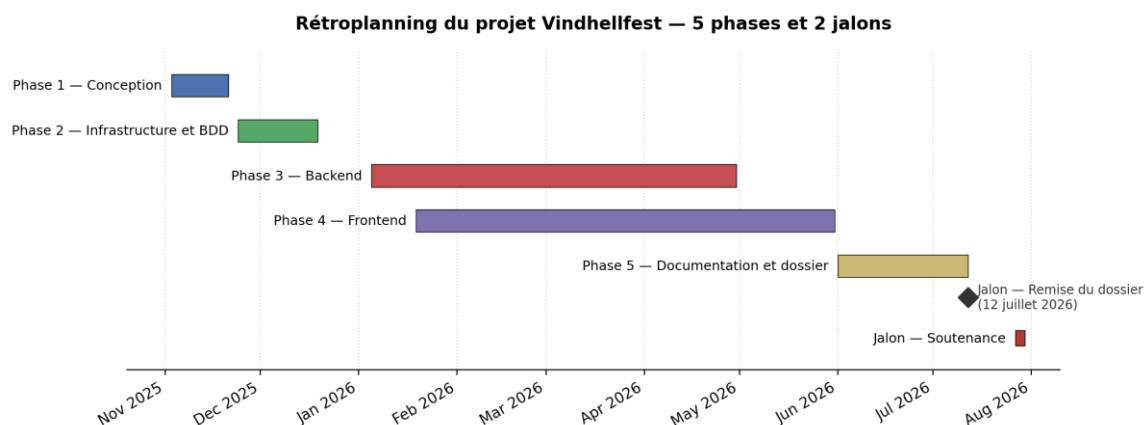


Figure — Diagramme de Gantt du rétroplanning (5 phases et 2 jalons)

III.4 Outils de gestion utilisés

Au-delà du backlog et du rétroplanning, le projet s'appuie sur un ensemble d'outils couvrant le versioning, l'intégration continue, la modélisation et l'environnement de développement.

- Versioning : Git et GitHub, dépôt unique regroupant frontend et backend.
- Gestion de projet : Notion, avec le backlog produit en tableau kanban et le rétroplanning.
- Intégration continue : GitHub Actions, deux jobs indépendants (backend et frontend), exécutant lint et tests à chaque push et pull request, sur toutes les branches.
- Conteneurisation : Docker Compose, avec 3 services (db, backend, frontend), garantissant un environnement identique en développement local et en CI.
- Modélisation de données : draw.io, pour les diagrammes MCD et MLD et pgAdmin pour la MPD de la base de données.
- Conception graphique : Figma, pour les zonings et wireframes.
- Éditeur de code : Visual Studio Code.
- Qualité de code : ESLint et Prettier.

III.5 Difficultés rencontrées et solutions apportées

Plusieurs difficultés techniques ont été rencontrées au cours du projet. En voici les principales, accompagnées des solutions retenues.

Contrainte d'exclusion PostgreSQL. Les règles métier interdisant à un artiste d'être programmé sur deux créneaux simultanés, et à une même scène d'accueillir deux concerts qui se chevauchent, devaient être garanties au niveau de la base de données elle-même plutôt que dans le seul code applicatif. La solution a été

l'activation de l'extension `btree_gist` via `CREATE EXTENSION IF NOT EXISTS btree_gist` dans les scripts d'initialisation SQL, et mise en place de deux contraintes `EXCLUDE USING gist` sur la table `concerts`, l'une sur la scène (deux concerts ne peuvent se chevaucher sur la même scène), l'autre sur l'artiste (un même artiste ne peut jouer sur deux scènes en même temps).

Absence de hot reload en environnement Docker. Une fois les applications conteneurisées, le hot reload ne fonctionnait plus. Solution, désactiver Turbopack (`NEXT_DISABLE_TURBOPACK=1`) et à forcer le retour à l'ancien moteur Webpack via la commande de démarrage, au prix d'un rechargement plus lent mais sans impact en production.

Double URL d'API selon le contexte d'exécution. Dans une architecture Docker, le frontend Next.js s'exécute dans un conteneur isolé sur le réseau interne, tandis que le navigateur de l'utilisateur se trouve sur la machine hôte, ces deux contextes n'adressent pas le backend de la même façon, localhost pointant, depuis l'intérieur d'un conteneur, vers le conteneur lui-même plutôt que vers le backend. Solution, deux variables d'environnement distinctes `API_URL_SERVER` (`http://backend:4000`), utilisée par les Server Components exécutés côté conteneur, et `NEXT_PUBLIC_API_URL` (`http://localhost:4000`), utilisée par les Client Components exécutés dans le navigateur, le préfixe `NEXT_PUBLIC_` étant la convention Next.js pour exposer une variable au navigateur.

IV. Spécifications fonctionnelles

IV.1 Cas d'utilisation

IV.1.1 Diagramme de cas d'utilisation global

Il existe deux types d'acteurs : le Visiteur, qui accède au site sans authentification, et l'Administrateur, authentifié via l'un des trois rôles admin, artists et news. Le diagramme de cas d'utilisation ci-dessous regroupe les fonctionnalités accessibles à chacun, en distinguant par couleur celles communes à tous les administrateurs de celles réservées à un ou plusieurs rôles spécifiques.

Le Visiteur dispose de sept cas d'utilisation, tous accessibles sans authentification : consulter l'accueil, consulter la programmation, consulter le détail d'un artiste, consulter les actualités, consulter le détail d'une actualité, consulter les infos pratiques, et envoyer un message de contact.

L'Administrateur dispose de quatre cas d'utilisation communs aux trois rôles se connecter, consulter le dashboard, changer son mot de passe, se déconnecter et de trois cas d'utilisation restreints selon le rôle gérer les artistes (rôles admin et artists), gérer les actualités (rôles admin et news) et gérer les utilisateurs (rôle admin uniquement).

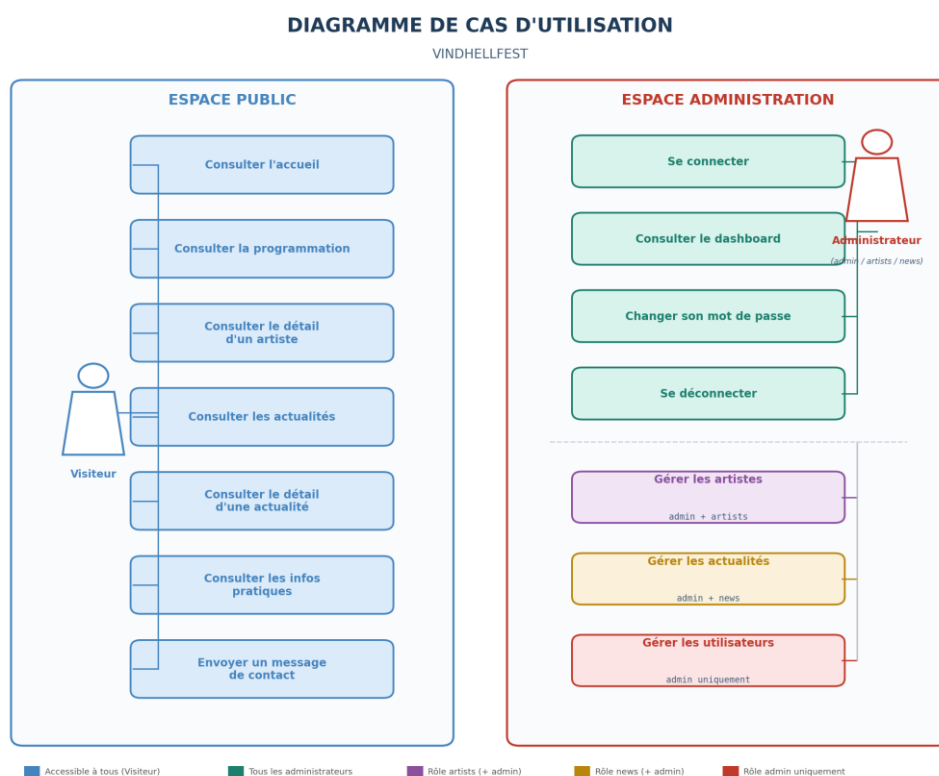


Figure — Diagramme de cas d'utilisation global — Vindhellfest

IV.1.2 Description des cas d'utilisation principaux

Cette section ne décrit pas l'intégralité des quatorze cas d'utilisation, elle se limite à cinq cas jugés principaux. Les cas d'utilisation non détaillés ici suivent des logiques

équivalentes et sont documentés, avec leurs critères d'acceptation détaillés, dans le backlog Notion.

Consulter la programmation. Permet à tout visiteur de découvrir les artistes et créneaux du festival, sans authentification.

Acteur : Visiteur.

Préconditions : aucune la route GET /artists est publique.

Scénario nominal : le visiteur ouvre la page Programmation ; le frontend appelle GET /artists, qui retourne la liste des artistes et de leurs concerts associés, le visiteur peut ensuite consulter le détail d'un artiste via GET /artists/:id.

Extensions : 404 ARTIST_NOT_FOUND si l'identifiant demandé ne correspond à aucun artiste . En l'absence d'artistes publiés, GET /artists renvoie une liste vide et affiche « Aucun artiste ».

Envoyer un message de contact. Permet à tout visiteur de transmettre une demande à l'organisation depuis le formulaire public.

Acteur : Visiteur.

Préconditions : aucune la route POST /contact/submit est publique.

Scénario nominal : le visiteur renseigne email, nom, sujet et message, le schéma Zod valide le format des champs, le contrôleur appelle le service d'envoi d'email et l'API répond 200 avec un message de confirmation.

Extensions : 400 VALIDATION_INVALID_BODY si un champ est absent ou hors bornes (email invalide, nom ou sujet trop courts/longs) ; 500 MAIL_SEND_ERROR si l'envoi SMTP échoue côté serveur.

Se connecter à l'administration. Permet à un administrateur (rôle admin, artists ou news) d'accéder au back-office.

Acteur : Administrateur, quel que soit son rôle.

Préconditions : posséder un compte créé au préalable par un administrateur.

Scénario nominal : l'administrateur soumet email et mot de passe à POST /admin/auth/login, les identifiants sont vérifiés, une session est ouverte en base et un JWT est déposé dans un cookie httpOnly.

Extensions : 401 AUTH_INVALID_CREDENTIALS si l'email ou le mot de passe est incorrect, 401 AUTH_MISSING_COOKIE, AUTH_MISSING_ACCESS_TOKEN ou AUTH_INVALID_ACCESS_TOKEN si une route protégée est appelée sans cookie ou avec un token absent/invalide ; 401 SESSION_NOT_FOUND ou SESSION_ALREADY_CLOSED si la session a été révoquée ou a expiré ; 403 FORBIDDEN si le rôle connecté n'a pas accès à la ressource demandée ; 429 RATE_LIMIT_TOO_MANY_ATTEMPTS après plusieurs tentatives de connexion échouées (rateLimitLogin).

Créer un artiste. Permet à un administrateur habilité d'ajouter un artiste et son concert associé à la programmation.

Acteur : Administrateur de rôle admin ou artists.

Préconditions : être authentifié, session ouverte, rôle admin ou artists

Scénario nominal : l'administrateur soumet POST /admin/artists en multipart (image et champs du formulaire), le middleware upload vérifie le type MIME de l'image, validateBody valide le corps selon createArtistSchema (Zod), puis le contrôleur createArtist convertit l'image en WebP via sharp, ouvre une transaction SQL, insère

l'artiste puis le concert associé, et valide la transaction, l'API répond 201 avec l'artiste créé.

Extensions : 400 VALIDATION_INVALID_BODY si le corps ne respecte pas le schéma, 400 ARTIST_FILE_REQUIRED si aucune image n'est fournie, 400 ARTIST_INVALID_FILE_TYPE si le type de fichier n'est ni jpeg, ni png, ni webp, 400 ARTIST_INVALID_YOUTUBE_URL ou ARTIST_INVALID_SPOTIFY_URL si un lien fourni est invalide, 409 ARTIST_FEATURED_LIMIT si deux artistes sont déjà mis en avant sur la page d'accueil ; 401/403 via la chaîne d'authentification en cas de session absente ou de rôle insuffisant. En cas d'échec après écriture du fichier, la transaction est annulée et l'image déjà écrite sur le disque est supprimée. Le diagramme de séquence détaillant ce scénario est fourni en Annexe I.

Gérer les utilisateurs. Permet à un administrateur du rôle admin de créer, modifier ou supprimer les comptes des autres administrateurs.

Acteur : Administrateur de rôle admin exclusivement.

Préconditions : être authentifié, session ouverte, rôle admin.

Scénario nominal : à la création (POST /admin/users), le contrôleur createUser vérifie la disponibilité de l'email et du nom d'affichage, génère un mot de passe temporaire, l'enregistre (haché) en base et envoie un email de bienvenue contenant les identifiants.

Extensions : 400 VALIDATION_INVALID_BODY si le corps ne respecte pas le schéma, 409 USER_EMAIL_ALREADY_USED ou 409 USER_DISPLAY_NAME_ALREADY_USED si l'email ou le nom d'affichage est déjà utilisé, 500 MAIL_SEND_ERROR si l'envoi de l'email de bienvenue échoue, 403 FORBIDDEN si le rôle connecté n'est pas admin.

IV.2 Maquettes et design

IV.2.1 Wireframes (basse fidélité)

Chaque écran de l'application a fait l'objet, avant sa réalisation, d'un zoning puis d'un wireframe basse fidélité, réalisés avec Figma. Cette étape a permis de valider l'organisation des pages indépendamment de la charte graphique.

Treize wireframes et treize zonings couvrent l'ensemble des écrans principaux de l'application, aussi bien côté public (accueil, programmation, détail d'un artiste, actualités, détail d'une actualité, infos pratiques, connexion) que côté administration (dashboard, gestion des artistes, gestion des actualités, gestion des utilisateurs).

L'ensemble des zonings et wireframes est reproduit intégralement en Annexe B.

IV.3 Parcours utilisateur

IV.3.1 Parcours visiteur public

Le parcours ci-dessous décrit l'enchaînement des pages publiques telles qu'accessibles sans authentification. Contrairement aux autres étapes, l'envoi d'un message de contact n'est pas une page dédiée le formulaire s'ouvre dans une modale déclenchée par le lien « Nous contacter » du pied de page, présent sur l'ensemble des pages publiques via le layout commun il est donc accessible à tout moment du parcours plutôt qu'en dernière étape imposée.

Accueil (/). page servie en ISR (revalidation toutes les 60 secondes) via GET /public/home ; elle agrège un bandeau d'ouverture Hero, un teaser de la programmation limité aux artistes mis en avant, un teaser des actualités récentes, un rappel des infos pratiques et la liste des partenaires.

Programmation (/artists). liste complète des artistes via GET /public/artists, filtrable par jour de festival (navigation générée dynamiquement depuis les dates réelles, sidebar sur desktop, menu modal dédié sur mobile) et triée par heure de passage décroissante, l'artiste programmé le plus tard apparaissant en tête de liste, chaque carte renvoie vers la fiche détaillée via un lien « Voir plus ».

Fiche artiste (/artists/[id]). détail d'un artiste via GET /public/artists/:id (photo, créneau, biographie, liens YouTube/Spotify le cas échéant) redirige vers la page 404 si l'identifiant ne correspond à aucun artiste.

Actualités (/news). liste des actualités publiées, avec un tri chronologique (par défaut « Plus récent ») piloté par la même combinaison sidebar desktop / menu modal mobile que la programmation.

Détail actualité (/news/[id]). détail d'une actualité via GET /public/news/:id, redirige vers la page 404 si l'actualité n'existe pas ou n'est pas publiée (les actualités non publiées restent réservées aux rôles admin et news).

Infos pratiques (/practical-info). contenu statique organisé en trois blocs (accès et transport, restauration, informations sur place), chacun affiché sous forme de liste à puces cochées.

Nous contacter (modale, toutes pages). formulaire nom/email/sujet/message soumis à POST /contact/submit, validé côté client puis côté serveur, avec confirmation « votre message est envoyé » affichée en cas de succès.

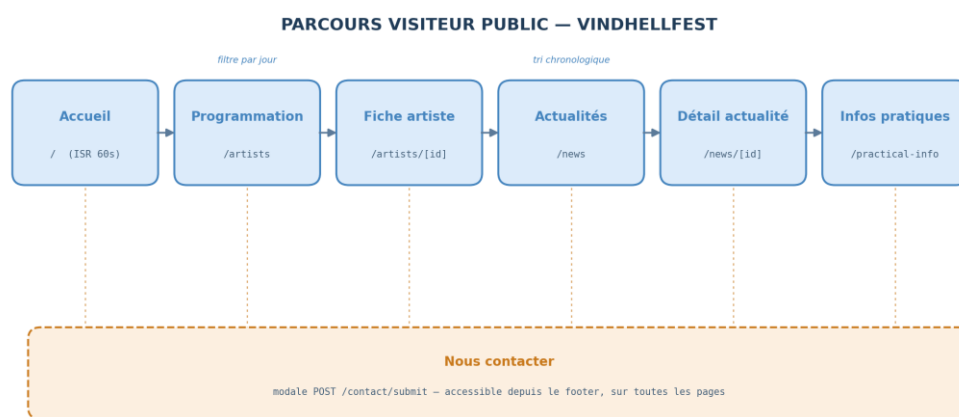


Figure — Parcours visiteur public — Vindhellfest

IV.3.2 Parcours administrateur

Le parcours administrateur diffère du parcours visiteur par une vérification de session à chaque navigation et par des sections de gestion dont l'accès dépend du rôle (admin, artists ou news).

Connexion (/login). formulaire email/mot de passe soumis à POST /admin/auth/login, en cas de succès, une session est ouverte en base et un cookie JWT httpOnly est déposé.

Vérification de session (AdminLayout). chaque page /admin/* est un layout serveur qui interroge GET /admin/auth/me avant de rendre le contenu, et redirige vers /login si la session est absente, invalide ou si l'API est inaccessible. Si mustChangePassword est vrai (première connexion), la modale de changement de mot de passe s'ouvre automatiquement sur le dashboard, en mode forcé.

Dashboard (/admin/dashboard). affiche la carte profil (nom, rôle, email, modification du mot de passe), un compte à rebours jusqu'au festival, et des raccourcis d'accès rapide filtrés selon le rôle connecté (filterNavByRole) Programmation pour les rôles admin et artists, Actualités pour les rôles admin et news, Utilisateurs pour le rôle admin uniquement.

Gestion de la programmation (/admin/artists). création et suppression d'artistes (et de leur concert associé) via POST/DELETE /admin/artists, depuis la liste, la modification (PATCH) s'effectue depuis la page de détail /admin/artists/[id], via le bouton « Modifier ».

Gestion des actualités (/admin/news). création et suppression d'actualités via POST/DELETE /admin/news, depuis la liste, avec gestion de leur statut de publication, la modification (PATCH) s'effectue depuis la page de détail /admin/news/[id], via le bouton « Modifier ».

Gestion des utilisateurs (/admin/users). création, modification et suppression des comptes administrateurs via POST/PATCH/DELETE /admin/users, le tout directement depuis la liste la création génère un mot de passe temporaire envoyé par email et impose son changement à la première connexion.

Déconnexion (header, toutes pages admin). révoque la session courante en base. Le cookie JWT httpOnly n'est pas explicitement supprimé, mais toute requête ultérieure utilisant ce cookie est rejetée par le middleware sessionIsOpen.

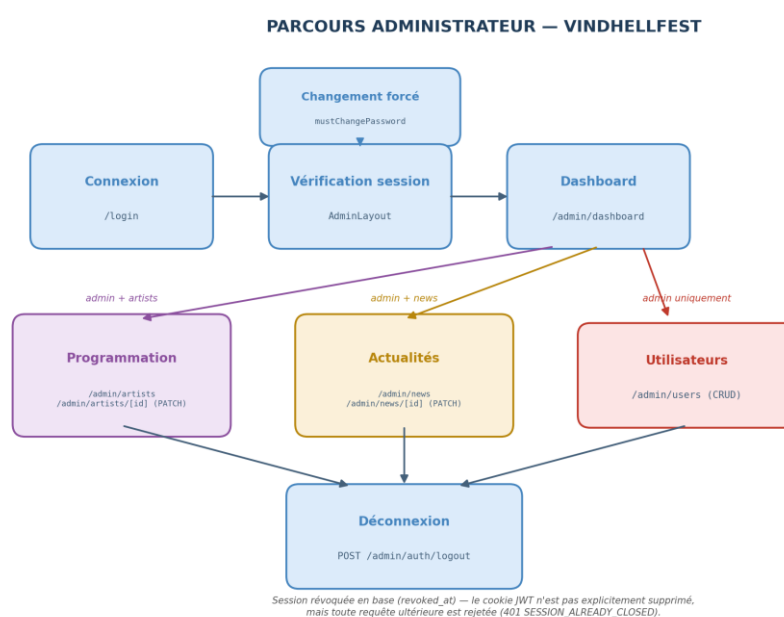


Figure — Parcours administrateur — Vindhellfest

V. Architecture logicielle

V.1 Vue d'ensemble de l'architecture

L'application repose sur une architecture 3 tiers, orchestrée par Docker Compose, un frontend Next.js, une API REST Express, et une base de données PostgreSQL, chacune isolée dans son propre conteneur et communiquant via un réseau Docker interne (app-net).

Architecture 3 tiers. Le frontend est exposé sur le port 3000 et le backend sur le port 4000. La base de données communique en interne avec le backend via le nom d'hôte db sur le réseau app-net, son port 5432 est par ailleurs mappé sur l'hôte dans la configuration actuelle, ce qui la rend accessible en développement avec un client SQL (V.5).

Double URL d'appel API. Le frontend appelle l'API selon deux chemins distincts, pilotés par deux variables d'environnement, `API_URL_SERVER` (`http://backend:4000`) pour le code exécuté côté serveur, dans le conteneur Next.js, et `NEXT_PUBLIC_API_URL` (`http://localhost:4000`) pour le code exécuté dans le navigateur (`apiRequest.ts`), qui n'a pas accès au réseau Docker interne. Le détail de cette contrainte et de sa résolution est exposé en III.5.

Rôle de chaque couche. Le frontend Next.js gère le rendu et l'interactivité client. Le backend Express expose une API REST sans état, chaque requête admin authentifiée par cookie JWT `httpOnly` vérifié en base.

Séquence de démarrage. Un healthcheck Docker sur le conteneur db garantit que le backend, dont le démarrage dépend de la condition `service_healthy`, n'essaie de se connecter à la base qu'une fois celle-ci prête.

Architecture en couches plutôt que MVC. Le projet ne suit pas un MVC monolithique classique où le serveur rend directement la vue. Front et back sont deux applications distinctes communiquant par HTTP : c'est la Vue (React) qui interroge l'API pour obtenir ses données de façon asynchrone, plutôt que de recevoir un rendu poussé par le serveur. En interne, l'API est elle-même structurée en couches : routes → middlewares → contrôleurs → base de données.

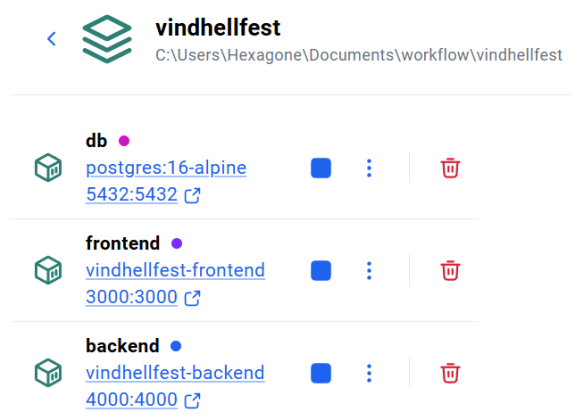


Figure — Conteneurs Docker en cours d'exécution (Docker Desktop) — Vindhellfest

V.2 Frontend — Next.js 16 (App Router)

Le frontend distingue Server Components (rendu sur le serveur) et Client Components (rendu côté navigateur).

Les pages publiques restent des Server Components pour le SEO, la vérification de session en admin/layout.tsx (V.2.7) est aussi un Server Component asynchrone qui redirige avant tout rendu.

Le code suit des couches croissantes : config/ → fonctions/ → hooks/ → components/ → app/, avec type.ts transversal.

V.2.1 Système de types — src/type.ts :

src/type.ts centralise les types métier. Les types principaux (UserItem, NewsItem, ArtistItem) reproduisent la forme des réponses API, les variantes sont créées par composition (HomeNews/HomeArtist) via Pick ou via Omit pour propager automatiquement toute évolution du type source. Les types propres à un seul composant restent locaux.

```
/* === NEWS === */

/** Type représentant une ligne news retournée par l'API. */
export type NewsItem = {
  id: string;
  title: string;
  content: string | null;
  is_published: boolean;
  created_at: string;
  url_media: string;
  description_media: string;
  author_name: string | null;
};

/** Type représentant les données news retournées par l'endpoint home. */
export type HomeNews = Pick<
  NewsItem,
  "id" | "title" | "url_media" | "description_media" | "created_at"
>;
```

Extrait — src/type.ts, dérivation de HomeNews par composition (Pick) à partir de NewsItem

V.2.2 Système de styles :

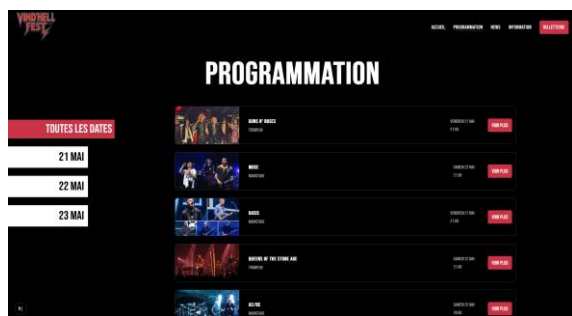
globals.css importe tailwindcss, puis tokens.css et animations.css (@keyframes). Le thème actif (--color-text, --color-bg) bascule via [data-theme], sans JavaScript.

```
[data-theme="admin"] {
  --color-text: var(--color-text-admin);
  --color-bg: var(--color-bg-admin);
}

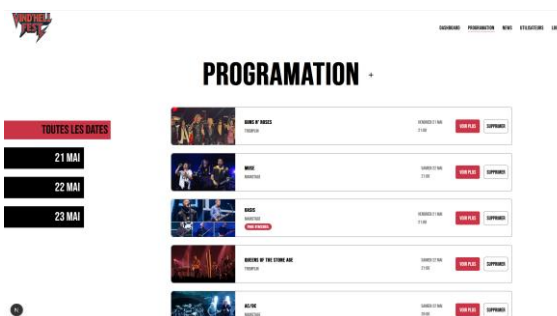
[data-theme="visitor"] {
  --color-text: var(--color-text-visitor);
  --color-bg: var(--color-bg-visitor);
}
```

Extrait — globals.css, redéfinition de --color-text / --color-bg selon [data-theme]

Les classes composants réutilisables sont déclarées sous `@layer` composants. Aucune valeur de couleur brute n'apparaît dans les `.tsx` tout passe par ces tokens Tailwind standard.



Page /artists — thème visitor (fond sombre)



Page /admin/artists — thème admin (fond clair)

V.2.3 Couche utilitaire — `src/functions/` :

Le dossier `functions/` regroupe cinq utilitaires, sans état ni JSX :

apiRequest.ts : appels API navigateur, retourne `{data, error}`,

fetchPublic.ts : Server Components, ISR 60 s,

getApiErrorMessage.ts : Affiche le message API si présent, sinon message par défaut selon le code HTTP

formatDate.ts : dates en français,

validation.ts : vérifications client `isEmpty/isEmail/isMaxLength`, la validation qui fait foi restant Zod côté backend.

	apiRequest.ts	fetchPublic.ts
Contexte d'exécution	Navigateur (Client Components)	Serveur (Server Components)
URL de base	NEXT_PUBLIC_API_URL (localhost:4000)	API_URL_SERVER (backend:4000)
Cookies	credentials: "include"	Aucun (endpoints publics, pas de session)
Cache	Aucune consigne (fetch standard)	ISR, revalidate: 60s
Retour	<code>{ data, error }</code> typé (<code>ApiResponseResult<T></code>)	T null

V.2.4 Hooks personnalisés — `src/hooks/` :

Hook	Rôle	Paramètres	Retour (principal)
<code>useFetch<T></code>	Charge des données au montage (GET)	endpoint: string	<code>{ data, isLoading, error }</code>
<code>useMutation<T></code>	Gère un appel de création/modification (POST/PATCH)	endpoint, method	<code>{ mutate, isLoading, error, reset }</code>
<code>useDelete</code>	Gère une suppression (DELETE)	endpoint: string	<code>{ handleDelete, isSubmitting, isDeleted, error, reset }</code>
<code>useModal<T></code>	Gère l'ouverture d'une modale, avec item optionnel (édition/suppression)	initialOpen?: boolean	<code>{ isOpen, item, open, close }</code>
<code>useNavPath</code>	Expose le chemin courant et s'il s'agit d'une route admin	—	<code>{ pathname, isAdminPath }</code>

useRoleGuard	Redirige vers /admin/dashboard si le rôle connecté ne permet pas d'accéder à la route courante	— (lit le contexte AdminAuthProvider)	aucun (effet de bord uniquement)
--------------	--	---------------------------------------	----------------------------------

useRoleGuard complète côté client la vérification déjà faite côté serveur dans admin/layout.tsx, la première protège contre l'affichage d'un contenu non autorisé lors d'une navigation interne, la seconde contre l'accès direct à une URL non autorisée.

```

/** Redirige vers /admin/dashboard si le rôle de l'utilisateur ne lui permet pas d'accéder à la route courante.
 * Doit être appelé uniquement dans les enfants de AdminAuthProvider.
 * @fonction useAdminUser récupère l'utilisateur admin depuis le contexte
 * @fonction useNavPath récupère la route courante
 */
export function useRoleGuard() {
  // on récupère l'utilisateur admin et la route courante
  const adminUser = useAdminUser();
  const { pathname } = useNavPath();
  const router = useRouter();

  // on vérifie si l'utilisateur a le droit d'accéder à la route courante
  useEffect(() => {
    // admin user est fait appel a un contexte
    if (!adminUser) return;

    // voir si la route courante a une restriction de rôle
    const currentItem = navAdminItem.find(
      (item) => item.path === pathname || pathname.startsWith(item.path + "/"),
    );

    // si la route courante n'a pas de restriction de rôle, on ne fait rien
    if (!currentItem?.role) return;

    // redirect si le rôle de l'utilisateur ne lui permet pas d'accéder à la route courante
    const allowedRoles = currentItem.role.split(",");
    if (!allowedRoles.includes(adminUser.user.role)) {
      router.replace("/admin/dashboard");
    }
  }, [adminUser, pathname, router]);
}

```

Extrait — src/hooks/useRoleGuard.ts

Les hooks useModal.ts et useMutation.ts suivent une logique similaire gestion de l'état d'ouverture/fermeture, et mutation générique avec état de chargement. (cf. Annexe D).

V.2.5 Configuration applicative — src/config/ :

Ils regroupent des données statiques, sans logique React ni appel réseau,

ui.ts: navVisitorItems, navAdminItem, filtres par entité, filterNavByRole()

festival.ts : FESTIVAL_DAYS, FESTIVAL_STAGES, TICKETING_URL, FESTIVAL_LOCATION, chacune source de vérité unique réutilisée par plusieurs composants.

```

/** Liens affichés dans la navigation de l'espace administrateur. */
export const navAdminItem: NavItem[] = [
  {
    label: "Dashboard",
    path: "/admin/dashboard",
    role: "admin, artists, news",
  },
  {
    label: "programmation",
    path: "/admin/artists",
    role: "admin, artists",
    desc: "Gérer la programmation artistique",
  },
  {
    label: "News",
    path: "/admin/news",
    role: "admin, news",
    desc: "Gérer les news et actualités",
  },
  {
    label: "Utilisateurs",
    path: "/admin/users",
    role: "admin",
    desc: "Gérer les comptes utilisateurs",
  },
  { labelBtn: "Logout", active: false },
];

```

Extrait — src/config/ui.ts, navAdminItem

V.2.6 Composants partagés — src/components/ :

Les vingt composants se répartissent en quatre catégories :

mise en page : Banner, Navigation, Footer, SideBarTool

Contenu : ArtistsContent, NewsContent, ArtistDetailContent, NewsDetailContent, SectionCta, LegalMention

Interactifs : ContactUs, ForgotPassword, MobileFiltersButton, modales AddArtistModal/AddNewsModal/DeleteModal

Utilitaires : LoadingLine, ModalCloseButton, ModalSetup, AdminUserProvider
Plusieurs sont réutilisés tels quels entre public et admin (ArtistsContent/NewsContent, DeleteModal).

```
/** Affiche un séparateur avec un bouton CTA centré entre deux lignes horizontales.
 * @param {string} props.href Chemin de destination au clic.
 * @param {string} [props.label="Voir plus"] Texte du bouton.
 */
export default function SectionCta({
  href,
  label = "Voir plus",
}): {
  href: string;
  label?: string;
} {
  return (
    <div className="flex items-center justify-center gap-6 w-full mt-6 group/cta">
      <span
        className="flex-1 h-0.5 bg-(--color-3) group-has-[.btn-cta:hover]/cta:animate-[line-reload_1.4s_ease_forwards]"
        style={{ transformOrigin: "right" }}
      />
      <Link href={href} className="btn-cta uppercase">
        {label}
      </Link>
      <span
        className="flex-1 h-0.5 bg-(--color-3) group-has-[.btn-cta:hover]/cta:animate-[line-reload_1.4s_ease_forwards]"
        style={{ transformOrigin: "left" }}
      />
    </div>
  );
}
```

Extrait — src/components/SectionCta.tsx

V.2.7 Structure des layouts et routage :

L'App Router distingue trois zones, chacune avec son layout et son thème :

(public)/ : accueil, artistes, actualités, infos pratiques, thème visitor

(auth)/ : /login, thème admin dès le serveur

admin/ : dashboard, artistes, actualités, utilisateurs, thème admin.

Le thème repose uniquement sur data-theme (CSS) posé par chaque layout, sans JavaScript

Vérification de session dans admin/layout.tsx : Server Component asynchrone qui lit les cookies, appelle GET /admin/auth/me et redirige vers /login avant tout rendu en cas d'échec. Si la session est valide, les données utilisateur sont injectées dans AdminUserProvider, accessibles via useAdminUser().

Ce même mécanisme de garde de rôle encapsule l'ensemble des pages d'administration. (cf. Annexe D).

V.2.8 Gestion des erreurs (frontend) :

Le traitement des erreurs suit une chaîne unique, un composant appelle un hook (useFetch, useMutation, useDelete) qui délègue à apiRequest.ts. Si la réponse HTTP n'est pas ok, apiRequest.ts construit une ApiRequestError (étend Error avec le

champ status), la rattrape, et retourne toujours `{ data, error }`, aucune exception ne remonte brute au composant.

`ApiRequestError` reconstruit ainsi une vraie erreur JS typée (message + status) à partir du JSON brut renvoyé par l'API, ce qui permet aux composants de faire `if (error) { error.status, error.message }`.

(cf. Annexe E : captures de la console lors d'une connexion valide et invalide.)

En cas d'erreur, celle-ci est transmise à `getApiErrorMessage.ts` qui retourne le message d'erreur de l'API s'il est exploitable (ex : 401 → «Email ou mot de passe incorrect», message renvoyé par le backend), sinon un message par défaut selon le code HTTP. Le composant se contente d'un rendu conditionnel. `data`, `isLoading` et `error` sont des états locaux créés par le hook lui-même qui se contente de les exposer au composant.

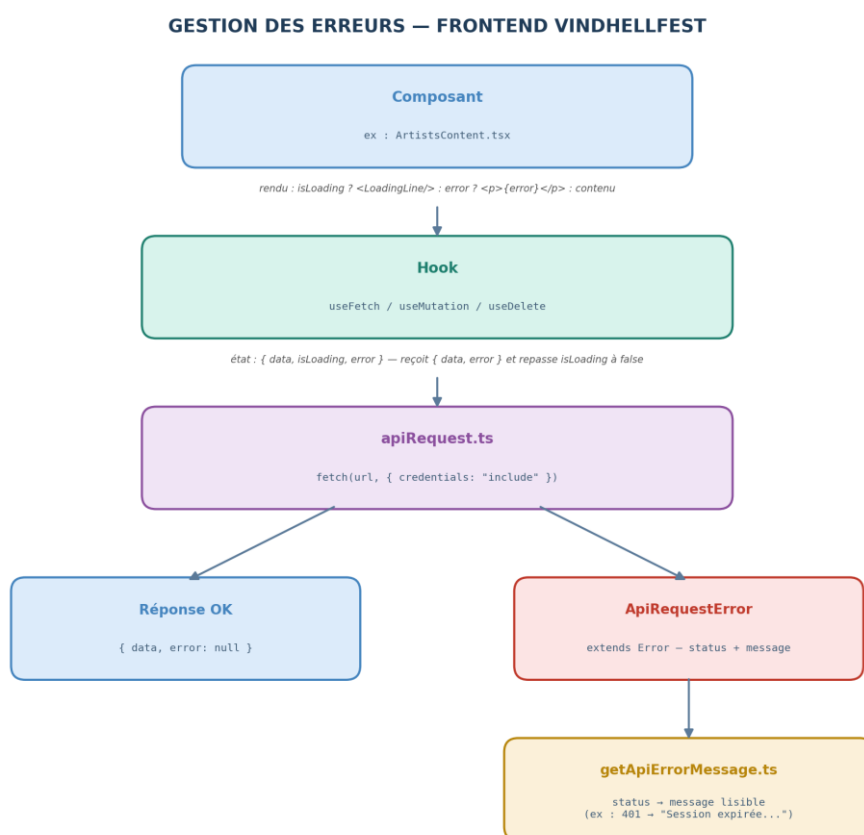


Figure — Gestion des erreurs, frontend — Vindhellfest

V.3 Backend — Express.js 5 (API REST)

Le backend adopte une architecture en couches : routes → middlewares → contrôleurs → services (optionnel) → base de données. Chaque couche ne connaît que la suivante, cette isolation se traduit par un fichier dédié à chaque responsabilité plutôt que des classes multi-responsabilités.

FLUX DE REQUÊTE — BACKEND VINDHELLFEST

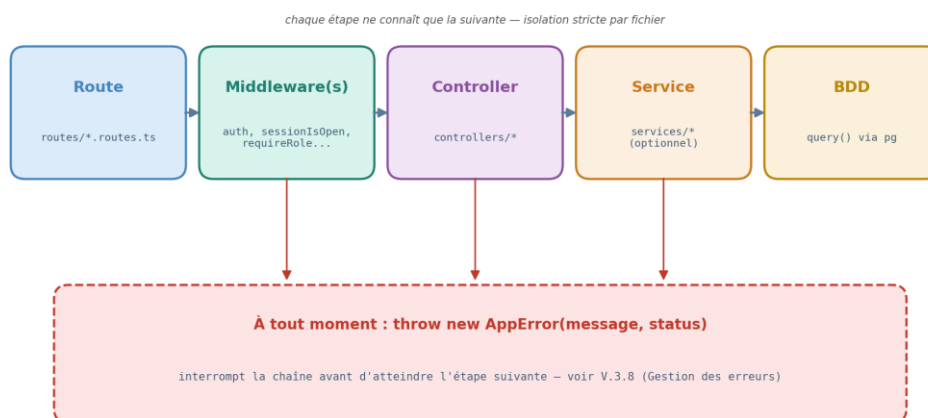


Figure — Flux de requête, backend — Vindhellfest

V.3.1 Système de types — `src/type.ts` :

`src/type.ts` est organisé en six sections :

EXPRESS augmente `Express.Locals` (`userId`, `userRole`, `userDisplayName`, `sessionId`).

USERS, SESSIONS, NEWS, ARTISTS, CONCERTS distinguent types base de données (lignes brutes pg) et types métier exposés par l'API (ex : `UserItem`, sans le hash). Comme côté frontend, les variantes sont dérivées par composition via `Pick` ou `Omit`.

```

type.ts

/* === EXPRESS === */

/** Augmentation du type Express.Locals pour typer res.locals dans les middlewares et controllers. */
declare global {
  // eslint-disable-next-line @typescript-eslint/no-namespace
  namespace Express {
    interface Locals {
      userId?: string;
      userRole?: UserRole;
      userDisplayName?: string;
      sessionId?: string;
    }
  }
}

/* === USERS === */

/** Type représentant une ligne retournant uniquement l'id -
  utilise pour les verifications d'existence en BDD. */
export type IdRow = { id: string };

/** Type représentant une ligne de la table users retournée par la base de données. */
export type UserCredentialsRow = {
  id: string;
  email: string;
  password_hash: string;
  display_name: string;
};
  
```

Figure — Extrait de `src/type.ts` (sections **EXPRESS** et **USERS**) — Vindhellfest

V.3.2 Fichiers racine — `src/`

index.ts : orchestre le démarrage, `dotenv.config()` → `validateEnv()` (arrête le processus si une variable obligatoire manque) → `createApp()` → `app.listen()`

app.ts : exporte `createApp()` séparément pour être utilisé avec Supertest, la fonction configure le CORS, sert /uploads, monte les routeurs, puis `notFoundHandler` et `errorHandler`.

db.ts : est le point d'accès unique à PostgreSQL.

env.ts : valide au démarrage, via un schéma Zod, les variables d'environnement obligatoires, une variable manquante lève une erreur explicite.

utils.ts : regroupe les fonctions partagées entre middlewares et contrôleurs (`initToken`, `serializeCookie`, `userExists`, `sessionExists`, `sessionRevoked`...).

```

/** Cree et configure l'application Express - CORS, routes API, handlers d'erreur. */
export function createApp() {
  const app = express();
  const allowedOrigin = process.env.FRONTEND_ORIGIN || "http://localhost:3000";

  // En production, l'app est derriere un proxy trust proxy permet a express-rate-limit de lire la vraie IP client.
  if (process.env.NODE_ENV === "production") { ...
  }

  /** Permet de lire le JSON envoye par le client dans le body des requetes HTTP. */
  app.use(express.json());

  /** Autorise le frontend a appeler l'API backend en local. ...
  });

  /** Sert les fichiers images uploades (artistes, etc.) */
  app.use("/uploads", express.static(path.join(__dirname, "../uploads")));

  /** Routes API (auth, admin, public, etc.) ...
  app.use("/admin", adminArtists);
  app.use("/admin", adminNews);
  app.use("/admin", adminAuth);
  app.use("/admin", adminUsers);
  app.use("/contact", contact);
  app.use("/public", publicHome);
  app.use("/public", publicArtists);
  app.use("/public", publicNews);

  /** Handlers globaux de fin de chaîne ...
  app.use(notFoundHandler);
  app.use(errorHandler);

  return app;
}

```

Figure — Extrait de `app.ts` (`createApp` — CORS, routes, handlers) — Vindhellfest

V.3.3 Middlewares — `src/middlewares/`

Onze fichiers, chacun avec une responsabilité unique :

Auth : vérifie le cookie JWT et lève une `AppError` 401 si l'utilisateur n'existe pas

optionalAuth : suit la même logique sans jamais interrompre la requête (routes semi-publiques, ex : `/public/news`).

sessionIsOpen : vérifie que la session existe en base et n'est pas révoquée, puis renouvelle le token d'accès.

requireRole : restreint l'accès selon le rôle stocké dans `res.locals` ;

authChain : compose `auth` + `sessionIsOpen` + `requireRole(...roles)` en une seule chaîne réutilisable.

validateBody et **validateUuidParam** : valident respectivement le corps de la requête et les paramètres d'URL via un schéma Zod

Upload : configure `multer` en `memoryStorage` (Buffer transmis à `sharp`), limité à 5 Mo et aux types `image/jpeg`, `image/png`, `image/webp`.

rateLimitLogin : limite à 5 tentatives par 10 minutes et par IP les routes de connexion et de réinitialisation.

Les trois fichiers restants — `asyncHandler`, `errorHandler` et `hashPassword` — sont décrits respectivement en V.3.8 (gestion des erreurs) et VII.3 (composants métier backend).

Le point commun à tous, une `AppError` levée à n'importe quelle étape interrompt immédiatement la chaîne le contrôleur n'est jamais atteint. Extrait réel de `auth.ts` consultable en Annexe D.

V.3.4 Validation des données — `src/schemas/`

Sept schémas Zod : authentification (`loginSchema`, `changePasswordSchema`, `forgotPasswordSchema`), utilisateurs (`createUserSchema`), actualités (`createNewsSchema`), artistes (`createArtistSchema`), contact (`contactSchema`). Pas de schéma distinct pour la modification : le même sert création (POST) et modification (PATCH), l'id dans l'URL déterminant l'opération. Chaque schéma est branché via `validateBody(schema)` directement dans la déclaration de route (V.3.7). Extrait réel de `schema.ts` consultable en Annexe D.

V.3.5 Couche services — `src/services/`

imageUpload.service : centralise `saveImage` (nom de fichier unique, redimensionnement 1600px max, conversion WebP qualité 80 via sharp) et `deleteImage` (suppression appelée après le commit SQL).

mailer.service : expose `sendMail` (convertit toute erreur SMTP en `AppError`) et trois envois spécifiques, `sendWelcomeEmail`, `sendPasswordResetEmail`, `sendContactEmail`.

user.service : regroupe les vérifications d'unicité (`checkEmailAvailable`, `checkDisplayNameAvailable`), `checkUserExists`, `generateTemporaryPassword`, `hashPassword`, et `isNewsPrivileged` (affichage des brouillons dans news).

```
> /** Genere un nom de fichier unique, cree le dossier si absent, redimensionne a 1600px max, ...
export async function saveImage(
  buffer: Buffer,
  uploadsDir: string,
  urlPrefix: string,
): Promise<string> {
  // Génère un nom de fichier unique et construit les chemins disque et URL publique.
  const uuid = randomUUID();
  const filename = `${uuid}.webp`;
  const filepath = path.join(uploadsDir, filename);
  const url_media = `${urlPrefix}/${filename}`;

  // Crée le dossier si absent, redimensionne à 1600px max et convertit en WebP qualité 80.
  await mkdir(uploadsDir, { recursive: true });
  await sharp(buffer)
    .resize(1600, undefined, { fit: "inside", withoutEnlargement: true })
    .webp({ quality: 80 })
    .toFile(filepath);

  return url_media;
}
```

Figure — Extrait de `imageUpload.service.ts` (`saveImage`) — Vindhellfest

V.3.6 Controllers — `src/controllers/`

Vingt contrôleurs se répartissent en trois groupes reflétant l'arborescence du dossier:

public/ : lecture seule, sans authentification, home, artists, news

admin/ : CRUD complet, chaque route protégée par `adminAuth`

contact/ : envoi d'email uniquement, aucune écriture en base

Aucune logique métier ne reste dans les fichiers de routes, ceux-ci se contentent d'assembler middlewares et contrôleur, toute la logique (validation métier, appels

SQL, appels aux services) est déléguée aux contrôleurs et, quand elle est réutilisée, à la couche services/.

```
> /** Retourne une news par son identifiant. ...
export async function getNews(req: Request, res: Response) {
  // Extrait l'UUID et vérifie si l'utilisateur a accès aux brouillons.
  const { id } = req.params;
  const isPrivileged = isNewsPrivileged(res.locals.userRole);

  // Récupère la news complète avec le nom de l'auteur.
  const rows = await query<NewsItem>(
    `SELECT a.id, a.title, a.content, a.is_published, a.created_at,
      a.url_media, a.description_media,
      u.display_name AS author_name
    FROM news a
    LEFT JOIN users u ON u.id = a.user_id
    WHERE a.id = $1`,
    [id],
  );

  // Lève une 404 si introuvable, ou si brouillon non accessible par cet utilisateur.
  if (!rows[0]) throw new AppError(ERRORS.NEWS_NOT_FOUND, 404);
  if (!isPrivileged && !rows[0].is_published)
    throw new AppError(ERRORS.NEWS_NOT_FOUND, 404);

  return res.status(200).json({ news: rows[0] });
}
```

Figure — Extrait de `get_news.controller.ts` (controllers/public/news) — Vindhellfest

V.3.7 Routes — src/routes/

Les routes publiques (home, artists) ne portent aucun middleware d'authentification, `news.routes.ts` fait exception avec `optionalAuth`, pour inclure les brouillons si l'appelant a un rôle privilégié, sans jamais bloquer un visiteur anonyme.

Les routes admin utilisent systématiquement `adminAuth(...roles)` (auth + sessionIsOpen + `requireRole`), déployée par décomposition directement dans la déclaration de route.

validateUuidParam s'applique à toutes les routes portant un paramètre `:id`, et `rateLimitLogin` uniquement aux routes de connexion et de mot de passe oublié.

```
const router = Router();

router.get("/users", ...adminAuth("admin"), asyncHandler(listUsers)); // Lister les utilisateurs

router.post(
  "/users",
  ...adminAuth("admin"),
  validateBody(createUserSchema),
  asyncHandler(createUser),
); // Créer un utilisateur

router.patch(
  "/users/:id",
  ...adminAuth("admin"),
  validateUuidParam(),
  validateBody(createUserSchema),
  asyncHandler(updateUser),
); // Modifier un utilisateur

router.delete(
  "/users/:id",
  ...adminAuth("admin"),
  validateUuidParam(),
  asyncHandler(deleteUser),
); // Supprimer définitivement un administrateur

export default router;
```

Figure — Extrait de `admin.users.routes.ts` — Vindhellfest

V.3.8 Gestion des erreurs — src/errors/

Le mécanisme suit le principe du V.2.8, côté backend : une `AppError` levée dans un middleware ou contrôleur (même classe qu'`ApiRequestError` côté frontend, champ `status`). Chaque contrôleur/middleware asynchrone est enveloppé par `asyncHandler`, qui capture l'erreur et la transmet via `next(error)`. `errorHandler`, monté en dernier avec `notFoundHandler`, renvoie le status et le message d'une `AppError` tels quels, ou une réponse générique 500 sinon, sans exposer de détail interne.

Ce mécanisme permet de lever une erreur avec un code HTTP (status) depuis n'importe quelle couche du code (service, middleware, contrôleur). Un seul endroit (`errorHandler`) centralise la transformation en réponse JSON, au lieu de répéter `res.status(...).json(...)` partout.

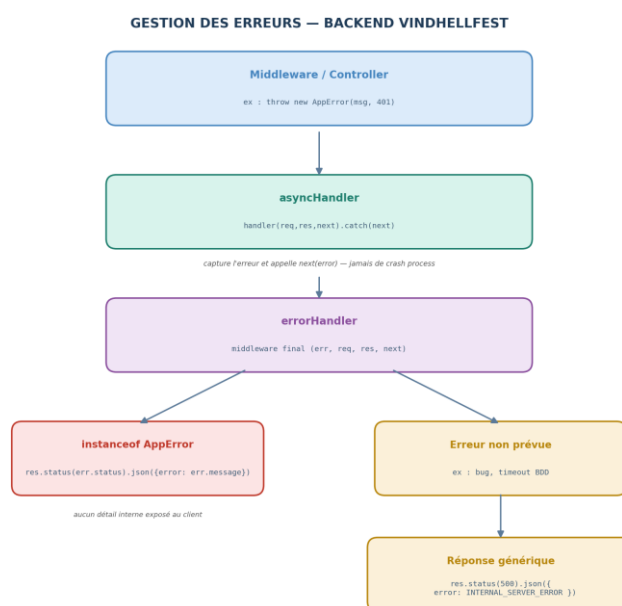


Figure — Gestion des erreurs, backend — Vindhellfest

V.4 Base de données — PostgreSQL 16

Cinq tables et cinq scripts. La base PostgreSQL est initialisée par 5 scripts SQL numérotés dans `bd/init/` : `01_user_schema.sql` à `05_concert_schema.sql`, exécutés dans l'ordre au premier démarrage : `users`, `sessions`, `news`, `artists`, `concerts`. Chaque script gère une seule table, ses ENUM et ses index. Quatre scripts complémentaires `06` à `09` insèrent ensuite des données de seed pour le développement.

Extensions PostgreSQL. Trois extensions :

Pgcrypto : `gen_random_uuid()`, base des UUID sur toutes les tables

Citext : dans `01_user_schema.sql`, rend `email` et `display_name` insensibles à la casse sans `LOWER()`

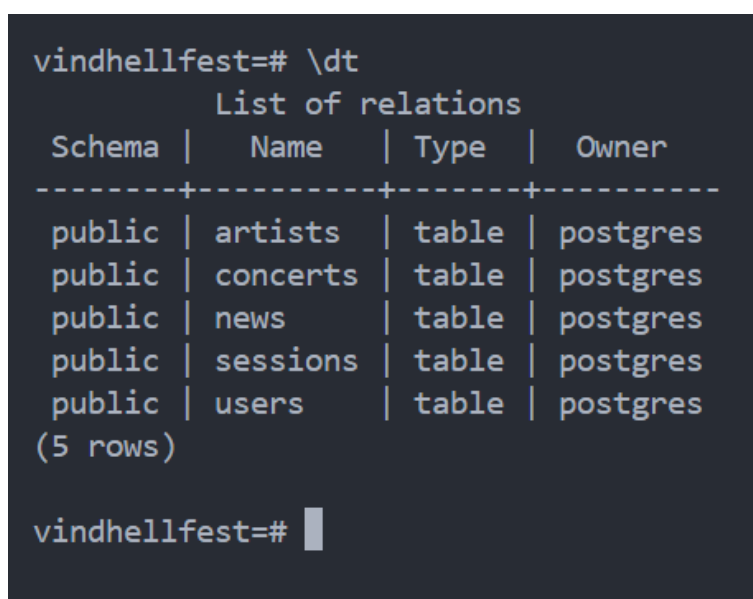
btree_gist : dans `05_concert_schema.sql`, nécessaire aux contraintes `EXCLUDE`

Deux types ENUM. `user_role` ('admin', 'artists', 'news') et `concert_stage` ('MainStage', 'Tremplin') encodent en base deux ensembles fermés de valeurs plutôt que des VARCHAR libres, l'intégrité est garantie par PostgreSQL lui-même

Contraintes métier au niveau base. La table concerts porte deux contraintes `no_overlap_stage`, `no_overlap_artist` qui interdisent respectivement deux concerts se chevauchant sur la même scène et un même artiste programmé sur deux scènes en même temps.

Un trigger sur la table artists (`trg_check_featured_limit`, BEFORE INSERT OR UPDATE) lève une exception si un troisième artiste tente de passer `is_featured` à TRUE.

Renvoi vers le détail complet. Le schéma complet, attributs, cardinalités, clés étrangères, index, données de seed



```
vindhellfest=# \dt
               List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | artists  | table | postgres
 public | concerts | table | postgres
 public | news     | table | postgres
 public | sessions | table | postgres
 public | users    | table | postgres
(5 rows)

vindhellfest=#
```

Figure — Session `psql`, `\dt` (liste des tables) — Vindhellfest

V.5 Infrastructure Docker

Trois services, trois responsabilités. `docker-compose.yml` déclare `db` (image `postgres:16-alpine`, port hôte 5432), `backend` (construit depuis `apps/backend/Dockerfile`, cible `dev`, port hôte 4000) et `frontend` (construit depuis `apps/frontend/Dockerfile`, cible `dev` également, port hôte 3000). `backend` dépend de `db` avec la condition `service_healthy`, `frontend` dépend seulement du démarrage du conteneur `backend`, sans condition de santé.

Réseau app-net. Les 3 services partagent un réseau Docker interne nommé `app-net`, qui permet au `backend` de joindre la base de données via le simple nom d'hôte `db` (et non une adresse IP), et au `frontend` de joindre le `backend` via `backend` en SSR (V.1, V.2.7). Seuls les ports 3000 (`frontend`) et 4000 (`backend`) sont pensés pour un accès utilisateur ; le port 5432 de `db` est lui aussi mappé vers l'hôte dans la configuration actuelle, ce qui le rend accessible depuis la machine locale en plus du réseau interne, pratique en développement pour s'y connecter avec un client SQL, mais à fermer dans une configuration de production stricte.

Volumes. `pgdata` est un volume Docker, donc persistant indépendamment du cycle de vie des conteneurs : supprimer et recréer le conteneur `db` ne fait pas perdre les données. À l'inverse, `apps/backend/uploads` est un `bind mount` vers un dossier de

l'hôte, les images uploadées (artistes, news) y sont donc bien persistées sur le disque local, mais rien ne garantit leur présence si l'application est redéployée sur une autre machine ou un autre environnement (ex. conteneur reconstruit sur un serveur distant sans ce dossier). Une évolution vers un stockage objet externe résoudrait cette limite.

Variables d'environnement. Le fichier `.env` à la racine ne contient que les identifiants PostgreSQL partagés entre db et backend (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`). Chaque service applicatif a ensuite son propre fichier dédié, `.env.backend` (connexion base de données, secrets JWT, configuration cookie, SMTP, validés au démarrage par `env.ts`, V.3.2) et `.env.frontend` (URLs d'API interne/externe, V.1).

Dockerfiles multi-stages. Le Dockerfile backend définit 3 étapes nommées (builder, runner, dev), builder compile le TypeScript, runner installe uniquement les dépendances de production et copie le résultat compilé (`dist/`), dev réinstalle toutes les dépendances et lance `npm run dev`. Le Dockerfile frontend en définit 5 (base, deps, builder, runner, dev) selon le même principe, avec une étape `deps` séparée pour isoler l'installation des dépendances Next.js. `docker-compose.yml` pointe actuellement vers `target dev` pour les deux services, le passage en production consisterait à basculer ce champ vers `target: runner`, sans modifier le reste de la configuration.

Le contenu complet de `docker-compose.yml` et des deux Dockerfiles est disponible en Annexe F (Configuration DevOps).

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
vindhellfest-backend	vindhellfest-backend	"docker-entrypoint.s..."	backend	2 minutes ago	Up 2 minutes	0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp
vindhellfest-db	postgres:16-alpine	"docker-entrypoint.s..."	db	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
vindhellfest-frontend	vindhellfest-frontend	"docker-entrypoint.s..."	frontend	2 minutes ago	Up 2 minutes	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

Figure — *docker compose ps, 3 conteneurs (db healthy) — Vindhellfest*

V.6 Pipeline CI/CD — GitHub Actions

Déclenchement. Le pipeline (`.github/workflows/ci.yml`) se déclenche à chaque push et à chaque pull request, sur toutes les branches. Il définit deux jobs, backend et frontend, ils s'exécutent donc en parallèle, sur deux machines virtuelles Ubuntu indépendantes.

Job backend : Après le checkout, il prépare les variables d'environnement un `.env.frontend`. Le job construit l'image backend, démarre le service db, attend sa disponibilité puis crée la base `vindhellfest_test`. Les dépendances sont installées, puis lint et tests s'exécutent dans un conteneur backend éphémère connecté à db via `app-net`. Un Shutdown final supprime conteneurs, réseaux et volumes, même en cas d'échec.

Job frontend. Même principe de fichiers `.env`. Après la construction de l'image, dépendances, lint et tests s'exécutent avec les tests frontend sont unitaires (Vitest + `jsdom` + `React Testing Library`) et ne nécessitent aucune API active. La commande utilisée est `npm run test:run` (vitest run), et non `npm test` qui resterait en mode Watch. Annexe F.

Le déploiement en production n'est pas encore automatisé à ce jour, une procédure de déploiement continu par runner est envisagée et détaillée en Annexe F.

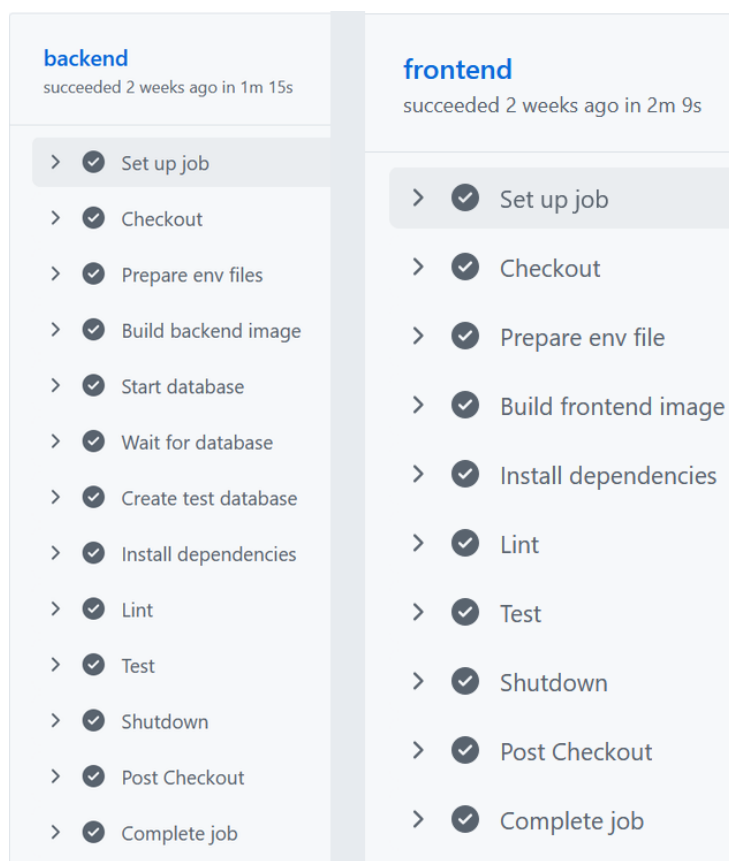


Figure — GitHub Actions, jobs backend et frontend réussis — Vindhellfest

V.7 Justifications des choix technologiques

Next.js 16 / React 19. Next.js avec App Router offre nativement SSR, SSG et ISR (page d'accueil en ISR 60 s, V.2.7). Le choix de Next.js plutôt qu'un SPA (Vite + React) est motivé par le SEO : un festival grand public doit être indexé et affiché immédiatement par les moteurs de recherche, ce qu'un rendu entièrement client ne permet pas nativement.

Express.js 5 / Node.js 20. Express reste minimaliste et laisse le contrôle sur l'organisation du code plutôt que d'imposer une structure rigide (NestJS) cohérent avec l'architecture en couches retenue. TypeScript est configuré en mode strict sur les deux applications, pour détecter les erreurs de typage à la compilation.

PostgreSQL 16. Retenu pour sa robustesse, son respect des standards SQL et des fonctionnalités, pgcrypto (UUID), citext (emails/noms insensibles à la casse), btree_gist (contraintes EXCLUDE empêchant les chevauchements de concerts) une garantie qu'une base NoSQL type MongoDB n'aurait pas offerte nativement.

JWT + cookies httpOnly + sessions en base. Les tokens JWT sont stockés dans des cookies httpOnly, inaccessibles en JavaScript côté client, plutôt que dans le localStorage (exposé au vol en cas de faille XSS). Les sessions sont aussi persistées en base plutôt que de reposer sur un JWT seul, permettant une révocation explicite (déconnexion, expiration forcée).

Zod. Zod est choisi pour la validation à l'exécution du corps de requête (messages d'erreur clairs via `.safeParse()`).

Vitest + Supertest + React Testing Library. Vitest est rapide, avec une API proche de Jest mais sans son temps de démarrage. Supertest teste les routes Express sans ouvrir de port réseau réel. React Testing Library encourage des tests centrés sur le comportement perçu par l'utilisateur plutôt que sur les détails d'implémentation.

Docker Compose. Garantit un environnement reproductible, identique entre développement et déploiement, plutôt qu'une installation manuelle des 3 services, source fréquente d'erreurs de configuration. Les Dockerfiles multi-stages partagent une base commune entre développement et production (target: runner), sans dupliquer la configuration.

```
"dependencies": {  
  "bcrypt": "^6.0.0",  
  "cookie": "^1.1.1",  
  "dotenv": "^17.2.3",  
  "express": "^5.1.0",  
  "express-rate-limit": "^8.2.1",  
  "jsonwebtoken": "^9.0.2",  
  "multer": "^2.1.1",  
  "nodemailer": "^8.0.2",  
  "pg": "^8.16.3",  
  "sharp": "^0.34.5",  
  "zod": "^4.2.1"  
},  
  
"dependencies": {  
  "@fortawesome/fontawesome-svg-core": "^7.2.0",  
  "@fortawesome/free-brands-svg-icons": "^7.2.0",  
  "@fortawesome/free-regular-svg-icons": "^7.2.0",  
  "@fortawesome/free-solid-svg-icons": "^7.2.0",  
  "@fortawesome/react-fontawesome": "^3.1.1",  
  "next": "^16.1.6",  
  "react": "19.2.3",  
  "react-dom": "19.2.3",  
  "react-modal": "^3.16.3"  
},
```

Figure — package.json backend et frontend, dépendances — Vindhellfest

VI. Spécifications techniques

VI.1 Structure du projet

Un seul dépôt, pas d'outillage workspace. Le projet réunit frontend et backend dans un unique dépôt Git, ce qui garantit un versionnement cohérent entre les deux applications (un commit peut modifier une route backend et son appel frontend en une seule fois) et permet de partager une configuration Docker unique à la racine. apps/frontend et apps/backend restent deux projets Node.js indépendants, chacun avec son propre package.json et son propre node_modules, simplement colocalisés dans le même dépôt et orchestrés ensemble par Docker Compose.

Arborescence à la racine. apps/ contient les deux applications (frontend, backend, détaillées ci-dessous), bd/ regroupe les scripts SQL d'initialisation, docker-compose.yml orchestre les 3 services, .env, .env.backend et .env.frontend portent les variables d'environnement, exclues du versionnement (.gitignore). .github/workflows/ contient le pipeline CI/CD.

src/ du backend. apps/backend/src/ contient 6 fichiers racine (index.ts, app.ts, db.ts, env.ts, type.ts, utils.ts) et 6 dossiers, un par responsabilité : controllers/, routes/, middlewares/, schemas/, services/, errors/.

src/ du frontend. apps/frontend/src/ contient 2 fichiers racine (type.ts, declarations.d.ts) et 5 dossiers : app/ (pages App Router), components/ (composants réutilisables), config/ (navigation, rôles), functions/ (apiRequest, fetchPublic, getApiErrorMessage, formatDate, validation) et hooks/ (useFetch, useMutation, useDelete, useModal, useNavPath, useRoleGuard).

```

apps/
├── backend
│   ├── src
│   │   ├── controllers
│   │   ├── errors
│   │   ├── middlewares
│   │   ├── routes
│   │   ├── schemas
│   │   ├── services
│   │   ├── app.ts
│   │   ├── db.ts
│   │   ├── env.ts
│   │   ├── index.ts
│   │   └── ... (2 autres fichiers)
│   ├── tests
│   │   ├── helpers
│   │   ├── integration
│   │   ├── unit
│   │   ├── TEST.md
│   │   ├── setup.ts
│   │   └── tsconfig.json
│   ├── API.md
│   ├── Dockerfile
│   ├── README.md
│   ├── TYPE.md
│   ├── eslint.config.cjs
│   ├── package-lock.json
│   ├── package.json
│   └── ... (2 autres fichiers)
├── frontend
│   ├── public
│   │   ├── partners
│   │   ├── header_logo.png
│   │   ├── hero_bg.webp
│   │   └── hero_logo.webp
│   ├── src
│   │   └── app

```

```

├── components
├── config
├── functions
├── hooks
├── declarations.d.ts
├── type.ts
├── tests
│   ├── components
│   ├── functions
│   ├── hooks
│   ├── TEST.md
│   ├── setup.ts
│   └── tsconfig.json
├── Dockerfile
├── README.md
├── eslint.config.mjs
├── next-env.d.ts
├── next.config.ts
├── package-lock.json
├── package.json
├── ... (4 autres fichiers)
└── bd/
    ├── init
    │   ├── 01_user_schema.sql
    │   ├── 02_sessions_schema.sql
    │   ├── 03_news_schema.sql
    │   ├── 04_artist_schema.sql
    │   ├── 05_concert_schema.sql
    │   ├── 06_seed_users.sql
    │   ├── 07_seed_news.sql
    │   ├── 08_seed_artists.sql
    │   └── 09_seed_concerts.sql
    ├── README.md
    ├── mcd_vindhellfest.drawio
    ├── mld_vindhellfest.drawio
    └── mpd_vindhellfest.png

```

Figure — Arborescence du projet (apps/ et bd/, extrait) — Vindhellfest

VI.2 modèle Conceptuel de données (MCD)

Cinq entités :

UTILISATEUR : email, mot de passe haché, nom d’affichage, rôle, date de changement de mot de passe, date de création

SESSION : date d’expiration, date de révocation, date de création

NEWS : titre, contenu, statut de publication, date de création, média

ARTISTE : nom, genre, origine, biographie, média, liens YouTube/Spotify, statut mis en avant

CONCERT : scène, heure de début, heure de fin.

UTILISATEUR → **ouvre** → **SESSION (0,n) / (1,1)**. Un utilisateur peut ouvrir zéro ou plusieurs sessions (connexions successives), une session appartient à exactement un utilisateur et ne peut pas exister sans lui.

UTILISATEUR → **rédige** → **NEWS (0,n) / (0,1)**. Un utilisateur peut rédiger zéro ou plusieurs news, une news est rédigée par zéro ou un utilisateur, le (0,1) traduit le fait que l’auteur peut devenir inconnu si son compte est supprimé.

ARTISTE → **se produit** → **CONCERT (0,n) / (1,1)**. Un artiste peut se produire dans zéro ou plusieurs concerts (programmation à plusieurs créneaux), un concert est associé à exactement un artiste et ne peut pas exister sans lui.

Entités et relation délibérément écartées. ROLE et STAGE auraient pu être modélisées comme des entités Merise à part entière, mais les rôles (admin, artists, news) et les scènes (MainStage, Tremplin) forment chacun un ensemble fermé,

connu à l'avance et sans attribut propre, une entité séparée ne se justifie que si la valeur porte ses propres attributs ou évolue dynamiquement, ce qui n'est le cas ni pour l'un ni pour l'autre. Un type ENUM PostgreSQL garantit la même contrainte d'intégrité directement en base, sans jointure supplémentaire.

La relation UTILISATEUR → ARTISTE (un utilisateur crée techniquement un artiste) n'est pas modélisée non plus, l'application n'affiche pas l'auteur d'un artiste, ne filtre pas par créateur et ne gère aucun droit par propriété modéliser cette association ajouterait de la complexité sans réelle valeur métier.

Le diagramme MCD correspondant est fourni en Annexe A.

VI.3 modèle Logique de Données (MLD)

Types PostgreSQL retenus. Chaque entité du MCD devient une table, avec un type PostgreSQL choisi pour chaque attribut.

UUID (via pgcrypto) pour tous les identifiants.

CITEXT pour email et display_name (users) comparaison insensible à la casse sans LOWER() explicite.

VARCHAR borné pour les chaînes de longueur connue à l'avance (VARCHAR(255) pour password_hash/url_media/liens externes, VARCHAR(150) pour title/name, VARCHAR(80) pour origin, VARCHAR(60) pour genre) ;

TEXT pour le contenu long et imprévisible (bio, content).

BOOLEAN pour les états binaires (is_published, is_featured).

TIMESTAMPZ pour toutes les dates.

Deux ENUM PostgreSQL, user_role et concert_stage, encodent les deux ensembles fermés de valeurs.

Clés étrangères et suppression. Trois clés étrangères relient les tables.

sessions.user_id référence users.id en ON DELETE CASCADE : supprimer un utilisateur supprime ses sessions, qui n'ont aucun sens sans lui.

news.user_id référence users.id en ON DELETE SET NULL (colonne nullable) : une news reste publiée même si son auteur quitte l'association, seul le lien vers l'utilisateur disparaît l'alternative aurait été un soft delete (is_active) sur users, envisageable comme évolution future mais non retenue ici.

concerts.artist_id référence artists.id en ON DELETE CASCADE un concert n'a pas de sens sans l'artiste programmé.

Tables racines. users et artists ne portent aucune clé étrangère ce sont les deux racines du schéma, dont dépendent respectivement sessions/news et concerts. Cette absence de dépendance amont explique pourquoi elles sont aussi les deux seules tables sans contrainte ON DELETE à gérer en cascade depuis une autre table.

Le diagramme MLD correspondant est fourni en Annexe A.

VI.4 modèle Physique de Données (MPD)

Contraintes CHECK. Chaque table impose ses propres règles de cohérence directement en base.

sessions porte deux CHECK : `chk_session_expires_after_created` (`expires_at > created_at`) et `chk_session_revoked_after_created` (`revoked_at IS NULL OR revoked_at >= created_at`).

concerts porte `chk_concert_end_after_start` (`end_time > start_time`).

Contraintes EXCLUDE. La table `concerts` porte deux contraintes EXCLUDE USING gist, qui nécessitent l'extension `btree_gist`.

no_overlap_stage interdit à deux concerts de se chevaucher sur la même scène.

no_overlap_artist interdit à un même artiste de jouer sur deux scènes en même temps, selon le même principe appliqué à `artist_id`. L'intervalle '[' est fermé au début et ouvert à la fin, ce qui autorise un concert à commencer exactement à l'heure où le précédent se termine.

Trigger `trg_check_featured_limit`. Ce trigger BEFORE INSERT OR UPDATE FOR EACH ROW sur `artists` exécute la fonction `check_featured_limit()`, si la ligne en cours passe `is_featured` à TRUE, il compte les artistes déjà mis en avant en excluant la ligne courante (`id != NEW.id`, pour ne pas se compter soi-même lors d'une modification), si ce compte atteint déjà 2, il lève RAISE EXCEPTION 'featured_limit_reached'. Le backend intercepte cette exception PostgreSQL et la traduit en erreur métier 409

Dix index. Les index couvrent les colonnes utilisées en filtre ou en tri par l'API :

Sur `sessions`, `user_id`, `expires_at` et `revoked_at` (recherche des sessions d'un utilisateur, vérification d'expiration/révocation)

Sur `news`, `created_at` et `is_published` (tri chronologique, filtrage des brouillons) ;

Sur `artists`, `genre`, `name` et `is_featured` (filtrage par genre, recherche par nom, artistes mis en avant) ;

Sur `concerts`, un index composite (`stage`, `start_time`) pour le programme d'une scène triée par horaire, et `start_time` seul pour le tri global.

L'ensemble de ces règles (types, contraintes, trigger, index) est implémenté dans `bd/init/01_user_schema.sql` à `05_concert_schema.sql`, rejoués automatiquement au démarrage du conteneur db, les scripts 06 à 09 insèrent ensuite les données de seed de développement (V.4).

Column	Type	Collation	Nullable	Default	Storage	Compression	Stats target	Description
<code>id</code>	<code>uuid</code>		not null	<code>gen_random_uuid()</code>	plain			
<code>artist_id</code>	<code>uuid</code>		not null		plain			
<code>stage</code>	<code>concert_stage</code>		not null		plain			
<code>start_time</code>	<code>timestamp with time zone</code>		not null		plain			
<code>end_time</code>	<code>timestamp with time zone</code>		not null		plain			

Indexes:

- "concerts_pkey" PRIMARY KEY, btree (`id`)
- "idx_concerts_stage_start" btree (`stage`, `start_time`)
- "idx_concerts_start_time" btree (`start_time`)
- "no_overlap_artist" EXCLUDE USING gist (`artist_id` WITH =, `tstzrange(start_time, end_time, '[')::text`) WITH &&
- "no_overlap_stage" EXCLUDE USING gist (`stage` WITH =, `tstzrange(start_time, end_time, '[')::text`) WITH &&

Check constraints:

- "chk_concert_end_after_start" CHECK (`end_time > start_time`)

Foreign-key constraints:

- "concerts_artist_id_fkey" FOREIGN KEY (`artist_id`) REFERENCES `artists(id)` ON DELETE CASCADE

Access method: heap

Figure — Structure de la table `concerts` (psql \d+ concerts) — Vindhellfest

VI.5 API REST — description des endpoints

VI.5.1 Routes publiques

GET /public/home. Aucune authentification. Retourne en parallèle (Promise.all) les artistes `is_featured = TRUE` (2 maximum, limite garantie par le trigger) et les 2 dernières news publiées. 200 en succès, 500 en cas d'erreur serveur.

GET /public/artists. Aucune authentification. Liste de la programmation via un LEFT JOIN sur concerts stage et `start_time` valent null si aucun concert n'est encore associé. *bio*, *genre*, *origin*, *liens externes* et *end_time* ne sont pas renvoyés dans la liste. 200/500.

GET /public/artists/:id. Aucune authentification. Détail complet d'un artiste avec son concert associé. 200 en succès, 404 si l'artiste n'existe pas, 500 en cas d'erreur serveur.

GET /public/news. Middleware optionalAuth, un rôle privilégié (admin ou news) reçoit toutes les news, y compris les brouillons, sinon, seules les news `is_published = TRUE` sont retournées. 200/500.

GET /public/news/:id. Middleware optionalAuth : un brouillon n'est accessible qu'à un rôle privilégié, sinon la route répond 404 même si la news existe réellement, le comportement masque délibérément l'existence du brouillon plutôt que de renvoyer un 403. 200 en succès, 404 sinon.

VI.5.2 Authentification

POST /admin/auth/login. Middlewares rateLimitLogin (5 tentatives / 10 min / IP) puis validateBody. Pose un cookie httpOnly contenant le token d'accès et crée une session en base. 200 en succès, 400 si le corps est invalide, 401 si email/mot de passe incorrect, 429 en cas de dépassement du taux de tentatives.

POST /admin/auth/logout. Révoque la session courante (`revoked_at` renseigné en base). 200 en succès, 401 si non authentifié ou session déjà fermée.

GET /admin/auth/me. Retourne les informations de l'utilisateur connecté et renouvelle le cookie d'accès (Set-Cookie) si la session est ouverte c'est cette route qui alimente useRoleGuard côté frontend. `mustChangePassword` vaut true si `password_changed_at` est null en base (mot de passe provisoire jamais changé). 200 en succès.

PATCH /admin/auth/password. Middlewares auth, sessionIsOpen, validateBody, `hashPassword("newPassword")`. L'utilisateur est identifié via son propre token (`res.locals.user.id`), aucun paramètre d'URL. 200 en succès, 400 si le corps est invalide, 401 plus un cas dédié « Mot de passe incorrect », 404 si l'utilisateur a été supprimé entre-temps.

POST /admin/auth/forgot-password. Middlewares rateLimitLogin, validateBody. Génère un mot de passe temporaire (`generateTemporaryPassword`) et l'envoie par email (`sendPasswordResetEmail`, V.3.5) `mustChangePassword` sera true à la connexion suivante. 200 en succès, 400 si le corps est invalide, 404 si aucun compte n'est associé à l'email, 429 en cas de dépassement du taux de tentatives.

VI.5.3 Administration — Artistes

Trois routes CRUD (POST/PATCH/DELETE /admin/artists[:id]). Toutes protégées par `adminAuth("admin", "artists")`, un rôle `news` reçoit 403.

POST — création. Chaîne `upload.single("image")` puis `validateBody(createArtistSchema)`. L'image est convertie en WebP (sharp, qualité 80) et écrite sur le disque avant l'ouverture de la transaction SQL. L'artiste puis le concert associé sont ensuite insérés dans une seule transaction (BEGIN / deux INSERT / COMMIT), si l'insertion du concert échoue, la transaction est annulée (ROLLBACK) et le fichier image déjà écrit est supprimé, pour ne laisser aucun résidu orphelin. Le trigger `trg_check_featured_limit` (VI.4) peut lever une exception PostgreSQL `featured_limit_reached`, que le contrôleur intercepte et traduit en 409. 201 en succès ; 400 (données ou image manquante/invalides), 401, 409 (limite `featured` atteinte).

PATCH — modification. L'image reste optionnelle si elle est absente, l'image existante est conservée, si une nouvelle est fournie, l'ancienne est supprimée du disque après la mise à jour réussie. Artiste et concert sont modifiés dans la même transaction SQL que pour la création. 200 en succès, 400, 401, 409.

DELETE — suppression. Supprime l'artiste, le concert associé est supprimé automatiquement en cascade par PostgreSQL (ON DELETE CASCADE) sans requête SQL supplémentaire côté contrôleur. Le fichier image est supprimé du disque après la suppression en base. 200 en succès, 400, 401.

VI.5.4 Administration — Actualités

Trois routes CRUD (POST/PATCH/DELETE /admin/news[:id]). Toutes protégées par `adminAuth("admin", "news")` — un rôle `artists` reçoit 403.

Champs multipart. `title`, `description_media` et le fichier image (jpeg/png/webp, 5 Mo max) sont obligatoires à la création, `content` est optionnel. `is_published` est transmis comme chaîne `"true"/"false"` (le multipart ne supporte pas les booléens natifs) `false` par défaut si absent. À la modification, l'image redevient optionnelle absente, l'image existante est conservée, fournie, l'ancienne est supprimée du disque. La suppression retire la news et son fichier image. 201 (création) ou 200 (modification/suppression) en succès, 400, 401, selon l'opération.

VI.5.5 Administration — Utilisateurs

Quatre routes CRUD (GET/POST/PATCH/DELETE /admin/users[:id]). Toutes protégées par `adminAuth("admin")` uniquement — un rôle `artists` ou `news` reçoit 403, quel que soit l'endpoint.

Création. `generateTemporaryPassword` génère un mot de passe provisoire, haché (`hashPassword`) avant l'insertion en base, le mot de passe en clair n'est jamais stocké et n'est transmis qu'une fois, par email (`sendWelcomeEmail`). `checkEmailAvailable` et `checkDisplayNameAvailable` sont appelées avant l'insertion.

Modification. Les mêmes vérifications d'unicité s'appliquent, mais en excluant l'id de l'utilisateur en cours de modification (`checkEmailAvailable(email, userId)`) sans cette exclusion, un utilisateur ne pourrait jamais enregistrer son propre email sans déclencher un faux conflit.

Suppression. Le contrôleur exécute un simple DELETE FROM users la suppression en cascade des sessions de l'utilisateur n'est pas codée explicitement dans le contrôleur, elle est garantie par la contrainte sessions.user_id ON DELETE CASCADE au niveau base de données. 200/201 en succès.

VI.5.6 Contact

POST /contact/submit. Aucune authentification. Seul middleware : `validateBody(contactSchema)`. Le contrôleur ne persiste aucune donnée en base il relaie directement le message vers l'adresse de l'association par email (`sendContactEmail`), sans laisser de trace dans la base de données. 200 en succès, 400 si le corps est invalide, 500 en cas d'échec d'envoi SMTP.

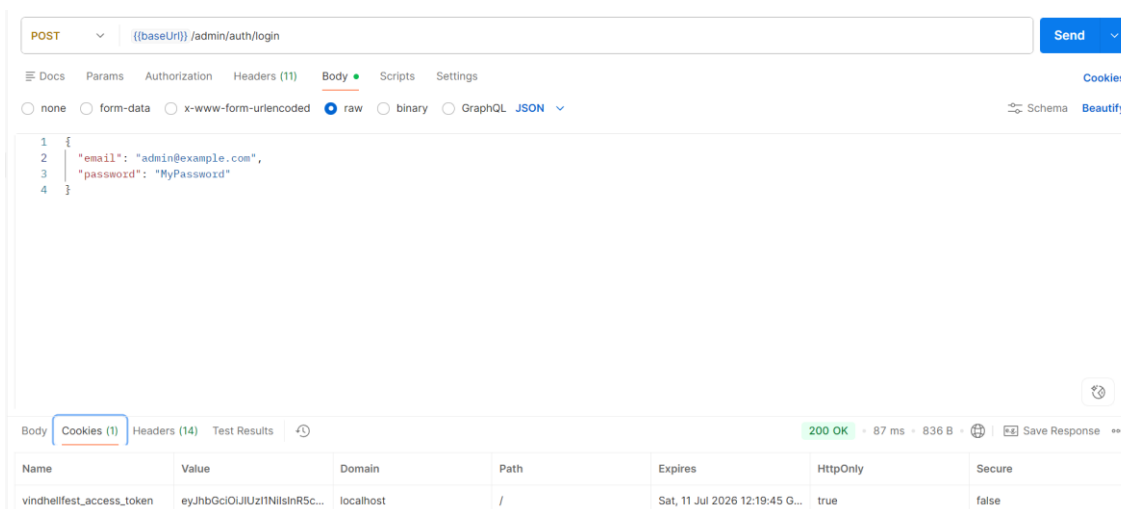


Figure — POST /admin/auth/login testé dans Postman — Vindhellfest

VI.6 Authentification et gestion des sessions

Pourquoi une session en base en plus du JWT. Un JWT signé reste valide jusqu'à sa date d'expiration, sans qu'aucune action côté serveur ne puisse l'invalider avant terme. Le projet associe donc chaque JWT à une ligne sessions en base, identifiée par un sessionId inclus dans le token. Cette ligne devient la source de vérité réelle, une déconnexion (POST /admin/auth/logout) renseigne simplement revoked_at, et toute requête ultérieure présentant ce même token est rejetée si le JWT lui-même n'a pas encore expiré.

Signature et contenu du JWT. `initToken` (`utils.ts`) signe le token via `jsonwebtoken` avec un secret partagé (`JWT_ACCESS_SECRET`), en algorithme `HS256` par défaut de la bibliothèque. Le payload ne contient que deux champs, `userId` et `sessionId` aucune donnée métier (rôle, email) n'y est stockée, elle est relue en base à chaque requête authentifiée. La durée de vie du JWT (`JWT_ACCESS_EXPIRES_IN`, 1h en développement) est volontairement plus courte que celle de la session en base (`SESSION_EXPIRES_IN`, 12h) le JWT est un accès court terme renouvelé en continu, la session est la fenêtre réelle pendant laquelle l'utilisateur reste connecté.

Attributs du cookie. serializeCookie (utils.ts) fixe httpOnly à true de façon non configurable le token n'est jamais accessible en JavaScript côté client, seule protection réellement efficace contre le vol par XSS. secure et sameSite sont en revanche lus depuis l'environnement (COOKIE_ACCESS_TOKEN_SECURE, COOKIE_ACCESS_TOKEN_SAME_SITE) secure=false et sameSite=lax en

développement local (HTTP sans TLS), à passer à `secure=true` en production (HTTPS obligatoire pour que le cookie soit transmis). `path` est fixé à `/` et `maxAge` correspond à la durée du JWT (1h), pas à celle de la session en base.

Renouvellement automatique. Sur toute route protégée par sessionIsOpen (routes admin authentifiées), un nouveau JWT et un nouveau cookie sont générés à chaque requête réussie ce qui glisse la fenêtre d'expiration de 1h en continu tant que la session en base reste valide et non révoquée, l'utilisateur n'est déconnecté que s'il reste inactif plus longtemps que SESSION_EXPIRES_IN, jamais simplement parce que le JWT de 1h est arrivé à expiration. optionalAuth (routes semi-publiques, V.3.3) vérifie la même validité de session mais ne renouvelle jamais le cookie seules les routes réellement authentifiées bénéficient du renouvellement.

mustChangePassword. Ce champ, retourné par GET /admin/auth/me, vaut true si password_changed_at est null en base, cas d'un compte fraîchement créé ou dont le mot de passe vient d'être réinitialisé avec un mot de passe temporaire jamais encore changé par son propriétaire. Le frontend s'appuie sur ce booléen pour forcer une modale de changement de mot de passe avant de laisser l'utilisateur accéder au reste de l'espace admin.

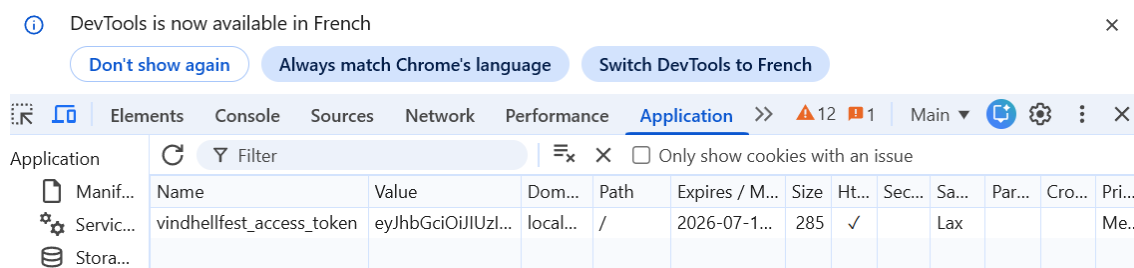


Figure — DevTools. attributs du cookie vindhellfest access token — Vindhellfest

VI.7 Upload de fichiers

multer + sharp. upload (middlewares/upload.ts) configure multer en memoryStorage le fichier reste en mémoire sous forme de Buffer, jamais écrit tel quel sur le disque, pour être transmis directement à sharp. Un fileFilter personnalisé n'accepte que image/jpeg, image/png et image/webp, et rejette toute autre valeur via une AppError 400 la taille est limitée à 5 Mo (limits.fileSize). saveImage (services/imageUpload.service.ts) redimensionne ensuite l'image à 1600px de large maximum et la convertit systématiquement en WebP qualité 80 via sharp, quel que soit le format d'origine envoyé.

Nommage et stockage par domaine. Chaque fichier reçoit un nom unique via randomUUID() (ex. 3f2a91d4-....webp) aucune dépendance au nom original envoyé par le client, qui pourrait contenir des caractères invalides ou entrer en collision. ARTISTS_UPLOADS_DIR et NEWS_UPLOADS_DIR pointent vers deux dossiers distincts.

Remplacement et suppression. `deleteImage` est systématiquement appelée après un COMMIT réussi plutôt qu'avant, pour ne pas perdre l'ancienne image si l'écriture en base venait à échouer entre-temps à la création d'un artiste, l'image est écrite avant l'ouverture de la transaction SQL et supprimée en cas de ROLLBACK, à la modification, l'ancienne image n'est supprimée qu'après le succès de la mise à jour, et seulement si une nouvelle image a été fournie.

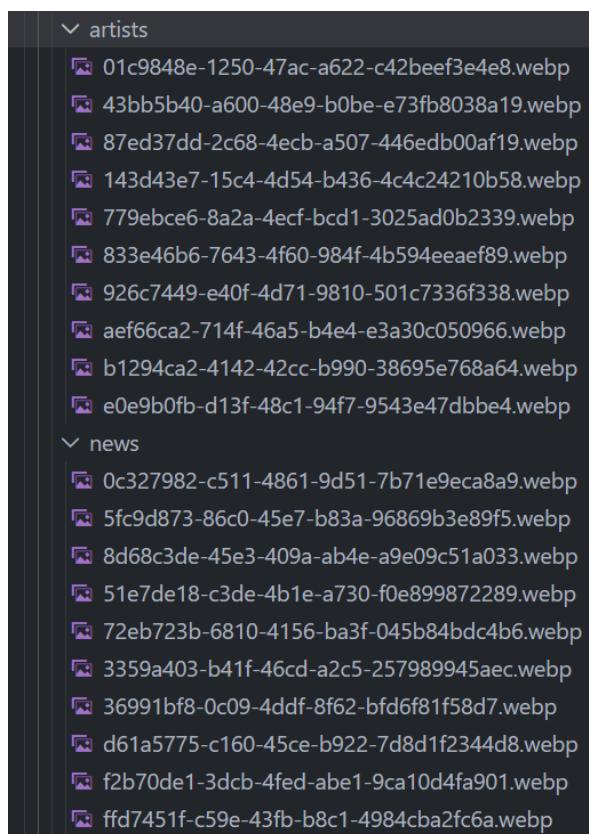


Figure — Dossier `apps/backend/uploads/` (`artists/`, `news/`) — Vindhellfest

VI.8 Envoi d'emails transactionnels

nodemailer et configuration SMTP. `mailer.service.ts` crée une unique instance de transporteur SMTP (`nodemailer.createTransport`) au chargement du module, configurée depuis les variables `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER` et `SMTP_PASS` le transporteur est donc partagé entre tous les envois plutôt que recréé à chaque appel.

Trois cas d'usage. `sendWelcomeEmail` transmet l'identifiant et le mot de passe provisoire en clair à la création d'un compte par un administrateur. `sendPasswordResetEmail` joue le même rôle après une demande de réinitialisation. `sendContactEmail` relaie le formulaire de contact public vers l'adresse de l'association en plaçant l'adresse du visiteur en `replyTo` une réponse directe depuis la messagerie de l'association part donc vers le visiteur, sans exposer son adresse comme expéditeur apparent.

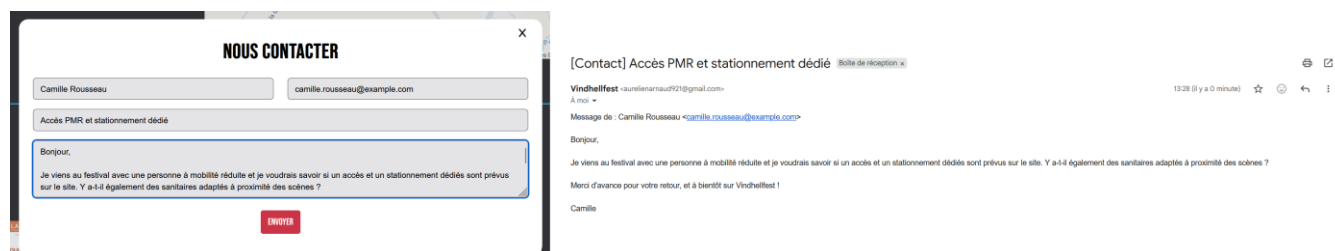


Figure — Modale de contact remplie et email reçu correspondant — Vindhellfest

VI.9 RGPD et protection des données personnelles

Données traitées. L'application manipule deux catégories de données à caractère personnel : les comptes internes (table users : email, nom d'affichage, mot de passe haché, rôle) réservés au personnel du festival (admin, artists, news) et non aux visiteurs publics, et les données du formulaire de contact (email, nom, message) rempli volontairement par un visiteur. Ce second cas ne donne lieu à aucun stockage en base et transmet directement le message par email via `sendContactEmail` sans jamais l'écrire en base de données.

Base légale et minimisation. Les comptes internes reposent sur l'intérêt légitime lié à l'administration du site (accès restreint à un rôle professionnel, pas de finalité commerciale externe). Le formulaire de contact repose sur la démarche volontaire de la personne qui choisit de le remplir pour être recontactée. Aucun cookie de mesure d'audience ou de publicité tiers n'est posé ; le seul cookie utilisé est le cookie de session `HttpOnly` strictement nécessaire à l'authentification, ce qui l'exempte du bandeau de consentement RGPD applicable aux cookies non essentiels.

Sécurité des données. Les mots de passe sont hachés avec `bcrypt` (coût 10, `hashPassword.ts`) et ne sont jamais stockés ni journalisés en clair. Le cookie d'accès est `HttpOnly` de façon non configurable, tandis que `Secure` et `SameSite` sont pilotés par variables d'environnement (`serializeCookie`, `utils.ts`) : `Secure` est à `false` en développement local (HTTP sans TLS) et doit être positionné à `true` en production (HTTPS). Ces attributs limitent son exposition au vol par script (XSS) ou par un site tiers (CSRF). Les sessions sont stockées côté serveur (table sessions) avec une date d'expiration et un mécanisme de révocation explicite au logout.

Conservation et droits des personnes. Les sessions expirent automatiquement (`SESSION_EXPIRES_IN`) et sont révocables individuellement. Les comptes internes sont conservés tant qu'ils sont actifs et peuvent être supprimés définitivement par un administrateur via l'interface de gestion des utilisateurs (`deleteUser` effectue une suppression physique en base, `DELETE FROM users`, pas une simple désactivation), ce qui couvre le droit à l'effacement. Les messages du formulaire de contact n'étant jamais stockés côté application, leur conservation dépend uniquement de la messagerie de l'organisation qui les reçoit, hors périmètre du code applicatif.

VI.10 Accessibilité et RGAA

Démarche suivie. Le cahier des charges fixait un objectif de test d'accessibilité avec l'outil WAVE, visant 0 erreur et 0 erreur de contraste. Au-delà de cet outil de mesure, plusieurs pratiques ont été intégrées au code : attributs `alt` sur les images (dont certains générés dynamiquement à partir du champ `description_media`), attributs `aria-label` et `aria-hidden` sur les éléments de navigation et les fenêtres modales, et associations `htmlFor` entre les champs de formulaire et leurs libellés.

Résultat de l'audit Lighthouse. Un audit Lighthouse réalisé sur le build de production (`npm run build`) obtient un score de 100/100 sur les quatre catégories Performance, Accessibility, Best Practices et SEO (Performance : First Contentful

Paint 0,2 s, Largest Contentful Paint 0,5 s, Total Blocking Time 10 ms, Cumulative Layout Shift 0, Speed Index 0,4 s). Le score de 100/100 en Accessibility confirme que les 20 règles automatisées d'axe-core passent sans erreur.

Résultat du test WAVE. Le test WAVE (WebAIM) réalisé sur la page d'accueil relève 0 erreur, 0 erreur de contraste, 2 alertes, 53 fonctionnalités, 18 éléments de structure et 17 attributs ARIA, conforme à l'objectif « 0 erreur et 0 erreur de contraste » fixé au cahier des charges. Les 2 alertes signalées correspondent en réalité à une répétition volontaire d'un même lien, le bouton d'appel à l'action « Billetterie » apparaît à la fois dans la section héro et dans la barre de navigation, cette dernière restant affichée en permanence lors du défilement (navbar sticky). Il ne s'agit donc pas d'une erreur mais d'un choix d'ergonomie assumé, destiné à garder l'accès à la billetterie visible à tout moment.

Constat de code. Une revue du code frontend dénombre 11 attributs alt, 17 attributs aria-* (12 aria-label et 5 aria-hidden), et 32 associations htmlFor entre labels et champs de formulaire. Le total d'attributs ARIA relevé dans le code concorde donc avec les 17 attributs ARIA comptabilisés par WAVE.

Limite identifiée. Lighthouse relève, à titre informatif (critère non noté), un point de vigilance rattaché au WCAG 2.4.9 (niveau AAA) : deux liens au texte identique « Voir plus » pointent vers des destinations différentes (/artists et /news). Une correction possible consisterait à leur attribuer des aria-label distincts (par exemple « Voir plus d'artistes » et « Voir plus d'actualités »).

Périmètre de la démarche. Un audit RGAA complet couvre 106 critères et représente une charge hors de portée du calendrier de ce projet. La démarche mise en œuvre ici, bonnes pratiques de code, test WAVE et audit Lighthouse ne remplace pas un audit RGAA formel, mais répond concrètement à l'exigence du Guide projet de traiter l'accessibilité (RGAA) dans les Spécifications techniques.

VII. Réalisations

VII.1 Interface publique

VII.1.1 Page d'accueil

Server Component `app/(public)/page.tsx` est un composant serveur asynchrone qui appelle `fetchPublic<...>("/public/home")` (V.2.7).

Cinq sections composées. `HomeHero` (dates du festival, bouton billetterie externe), `HomeProgrammation` (2 artistes mis en avant, lien « Voir plus » vers `/artists` via `SectionCta`), `HomeNews` (2 dernières news publiées, même pattern `SectionCta`), `HomeInfosPratiques` (adresse, carte Google Maps intégrée) et `HomePartenaires` (deux rangées de logos en défilement infini, sans appel API). `Programmation` et `News` ne s'affichent pas du tout si la liste reçue est vide (return null).

Voir Annexe C pour une capture d'écran de cette page.

VII.1.2 Programmation — liste des artistes

Client Component avec filtre par jour. `app/(public)/artists/page.tsx` est un composant client qui gère un état `activeFilter` (jour sélectionné, issu de `filterArtistsItems`) et le transmet à `ArtistsContent`.

Adaptation mobile/desktop. Les mêmes items de filtre alimentent `MobileFiltersButton` (bouton qui ouvre une modale de navigation, visible uniquement sous `md:hidden`) et `SideBarTool` (navigation sticky visible seulement à partir de `md:`), pour éviter de dupliquer la logique de filtre entre les deux affichages.

ArtistsContent partagé. Ce composant charge `/public/artists` via `useFetch`, affiche `LoadingLine` pendant le chargement, trie les artistes par heure de concert décroissante et filtre par jour si `activeFilter` est défini. Il s'agit du même composant que celui utilisé en administration (VII.2.3) : `useNavPath` détermine s'il faut afficher les actions réservées à l'admin (bouton Supprimer, badge « Page d'accueil ») via un simple test `isAdminPath`.

Voir Annexe C pour une capture d'écran de la page programmation.

VII.1.3 Détail d'un artiste

Server Component. `app/(public)/artists/[id]/page.tsx` récupère l'artiste via `fetchPublic(`/public/artists/${id}`)` et transmet les données à `ArtistDetailContent`.

Contenu affiché. Image héro pleine largeur avec le nom en overlay, bandeau scène/horaire, biographie, puis liens YouTube et Spotify affichés uniquement s'ils sont renseignés. Les champs genre et origin ne sont en réalité pas affichés par ce composant seuls nom, image, horaire, biographie et liens sociaux le sont.

Gestion du cas 404. Si `fetchPublic` retourne null (artiste inexistant, backend renvoyant 404), la page appelle `notFound()`, qui délègue le rendu au `not-found.tsx` global de l'application il n'existe pas de page 404 spécifique aux artistes, la même page 404 est réutilisée pour toute route introuvable.

VII.1.4 Actualités — liste et détail

Liste — NewsContent. Même structure que ArtistsContent (client component partagé public/admin, filtre sur useNavPath) activeFilter contrôle le tri, "Plus récent" (par défaut, ordre DESC natif de l'API) ou "Plus ancien" (liste inversée côté client). Une news non publiée n'apparaît que si isAdminPath est vrai.

Détail — gestion des brouillons. app/(public)/news/[id]/page.tsx est un composant serveur qui appelle notFound() si fetchPublic renvoie null ce qui se produit pour un brouillon consulté par un visiteur non authentifié le comportement observable est un simple 404, sans distinction visible entre « n'existe pas » et « existe mais brouillon ».

VII.1.5 Informations pratiques

Page statique sans appel API. app/(public)/practical-info/page.tsx ne fait aucune requête réseau accès et localisation, restauration/buvette et informations sur place sont des listes codées en dur dans le composant, pas des données venant de la base.

Voir Annexe C pour une capture d'écran de la page infos pratiques.

VII.1.6 Formulaire de contact

ContactUs — Client Component en modale. Formulaire avec 4 champs (nom, email, sujet, message), rendu dans une modale plutôt que sur une page dédiée, il n'existe pas de route /contact, le composant ContactUs est inséré directement dans le Footer (accessible depuis n'importe quelle page publique).

Validation avant envoi. isEmail et isMaxLength (src/functions/validation.ts) valident chaque champ côté client avant d'autoriser la soumission (bouton désactivé tant que isFormInvalid est vrai).

Succès et erreur. En cas de succès, le formulaire est remplacé par un message de confirmation ("votre message est envoyé") et les champs sont réinitialisés. En cas d'échec, le message traduit par getApiErrorMessage (V.2.8) s'affiche sous le bouton, le formulaire restant rempli pour permettre une nouvelle tentative.

Voir Annexe C pour une capture d'écran de la modale de contact ouverte.

VII.2 Interface d'administration

VII.2.1 Connexion / Déconnexion

Formulaire de connexion. app/(auth)/login/page.tsx envoie email/mot de passe vers POST /admin/auth/login puis redirige vers /admin/dashboard en cas de succès.

ForgotPassword. Un bouton « Mot de passe oublié » ouvre une modale contenant ForgotPassword, un formulaire à un seul champ (email) qui appelle POST /admin/auth/forgot-password la page de connexion elle-même reste affichée en arrière-plan, seule la modale change de contenu.

Déconnexion. Gérée côté backend par POST /admin/auth/logout, qui révoque la session en base le cookie d'accès expire alors naturellement côté client au prochain rendu, faute d'être renouvelé.

Protection de route. useRoleGuard est appelé en tête de chaque page admin cliente (dashboard, artistes, news, utilisateurs) et redirige vers /login si la session est

absente ou si le rôle ne correspond pas à la page. `admin/layout.tsx` effectue en complément une vérification serveur avant même le premier rendu client.

CONNEXION

The screenshot shows a login interface with the title 'CONNEXION'. It features two input fields: the first contains the email 'admin@example.com' and the second contains masked characters '.....'. Below the password field is a red message: 'TROP DE TENTATIVES, REESSAYER PLUS TARD'. Underneath this message is a red button labeled 'ENVOYER'. At the bottom of the form area is a link that says 'MOT DE PASSE OUBLIE'.

Figure — Page de connexion, message de limitation de tentatives — Vindhellfest

VII.2.2 Dashboard

Cards de raccourcis filtrées par rôle. `DashboardContent` récupère l'utilisateur admin via `useAdminUser` puis filtre `navAdminQuickLinks` avec `filterNavByRole(items, user.role)` seuls les raccourcis autorisés pour le rôle courant (Programmation, News, Utilisateurs) sont affichés.

Chargement des infos utilisateur. `GET /admin/auth/me` est en réalité appelé côté serveur dans `admin/layout.tsx`, pas par `DashboardContent` lui-même, les données sont injectées dans `AdminUserProvider` et lues ici via le contexte React (`useAdminUser`), sans requête réseau supplémentaire au chargement du tableau de bord.

VII.2.3 Gestion des artistes (CRUD)

Création, modification, suppression. `AddArtistModal` est utilisée à la fois pour la création (bouton + sur `/admin/artists`, formulaire vide) et pour la modification (bouton « Modifier » sur la fiche détail, `ArtistEditButton`, formulaire pré-rempli via la prop `artistToEdit`), un seul composant modal pour les deux cas, différencié par la présence ou l'absence de cette prop. `DeleteModal`, générique et partagé avec la gestion des news et des utilisateurs, gère la confirmation de suppression via `useDelete`.

Gestion de l'image. Le remplacement ou la suppression de l'ancienne image est entièrement géré côté backend le formulaire frontend se contente d'envoyer le nouveau fichier en `multipart/form-data` si l'utilisateur en sélectionne un.

Voir Annexe C pour les captures des écrans de gestion des artistes.

VII.2.4 Gestion des actualités (CRUD)

Création, modification, suppression. Même principe que pour les artistes : `AddNewsModal` sert à la fois à la création (bouton + sur `/admin/news`) et à la modification, `DeleteModal` gère la suppression avec confirmation.

Statut et image optionnelle. Le statut brouillon/publié (`is_published`) est un champ du formulaire, transmis comme chaîne `"true"/"false"` en `multipart`, `NewsContent`

affiche un badge « Brouillon » sur les news non publiées, invisible côté public. L'image reste optionnelle à la modification l'ancienne est conservée si aucun nouveau fichier n'est sélectionné.

Voir Annexe C pour les captures des écrans de gestion des actualités.

VII.2.5 Gestion des utilisateurs (CRUD)

Page réservée au rôle admin. `useRoleGuard` redirige tout utilisateur `artists` ou `news` qui tenterait d'accéder directement à `/admin/users`.

Tableau avec statut du mot de passe. `UsersContent` affiche, pour chaque utilisateur, la date de création et soit « Mot de passe provisoire » (si `password_changed_at` est null) soit la date de dernière modification reflet direct du champ `mustChangePassword`.

Création, modification, suppression. `AddUserModal`, réutilisée pour la création et la modification du rôle. Un point notable, différent de l'intention initialement prévue, rien n'empêche explicitement un administrateur de supprimer son propre compte, ni côté frontend ni côté backend, le code gère simplement ce cas en redirigeant vers `/login` après coup, puisque la session de l'utilisateur qui vient de se supprimer devient de toute façon invalide.

Voir Annexe C pour les captures des écrans de gestion des utilisateurs.

VII.2.6 Changement de mot de passe

Changement forcé au premier login. `ChangePasswordModal` s'ouvre automatiquement dès l'arrivée sur le tableau de bord si le mot de passe est encore le mot de passe provisoire généré à la création ou à une réinitialisation. En mode `forced`, le bouton de fermeture est masqué et la modale ne peut être fermée qu'après un changement réussi.

Changement volontaire. Le même composant est accessible à tout moment via le bouton « Modifier » du mot de passe sur la carte profil du tableau de bord, cette fois sans le mode `forced` l'utilisateur peut fermer la modale à tout instant. Dans les deux cas, l'appel API est identique `PATCH /admin/auth/password`.

VII.3 Backend — composants métier

L'architecture backend (middlewares, services, gestion des erreurs) est détaillée en V.3. Trois précisions complémentaires : le middleware `hashPassword` hache un champ configurable du corps de requête (utilisé pour le changement de mot de passe) ; les messages d'erreur sont centralisés dans `errorMessages.ts` (constante `ERRORS`) plutôt que codés en dur dans chaque contrôleur, et le détail d'une erreur inattendue (hors `AppError`) n'est jamais journalisé en console en dehors de l'environnement de production, ni exposé au client.

VIII. Sécurité de l'application

VIII.1 Authentification JWT et cookies httpOnly

Signature JWT. Le token est signé avec un secret lu en variable d'environnement pour une durée de 1h en développement. Le payload ne contient que `userId` et `sessionId`, jamais de donnée sensible ou de rôle directement exploitable.

Cookie httpOnly, Secure, SameSite. `httpOnly` est fixé à `true` de façon non configurable (protection XSS le token n'est jamais accessible en JavaScript côté client), `secure` et `sameSite=lax` sont pilotés par variables d'environnement, à passer en `secure=true` en production sur HTTPS. `sameSite=lax` offre une protection partielle contre le CSRF (le cookie n'est pas envoyé sur une requête cross-site déclenchée par un lien tiers, mais l'est sur une navigation directe).

Renouvellement automatique. `sessionIsOpen` régénère un nouveau JWT et un nouveau cookie à chaque requête admin réussie, glissant la fenêtre d'expiration tant que la session en base reste valide.

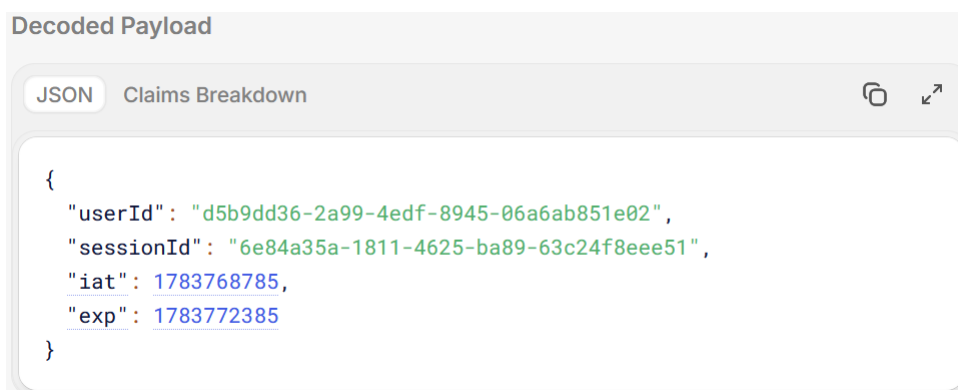


Figure — Payload JWT décodé (jwt.io) — `userId` et `sessionId` uniquement — Vindhellfest

VIII.2 Sessions en base de données

Pourquoi une session en base. Un JWT ne peut pas être invalidé avant sa date d'expiration naturelle ; associer chaque token à une ligne sessions en base permet une révocation explicite, impossible avec un JWT seul.

Vérification et révocation. `sessionIsOpen` vérifie à chaque requête que la session existe et n'est ni révoquée ni expirée. La déconnexion (POST `/admin/auth/logout`) renseigne `revoked_at`. La suppression d'un compte utilisateur entraîne l'invalidation en cascade de toutes ses sessions au niveau base de données (`sessions.user_id ON DELETE CASCADE`) sans code applicatif dédié.

VIII.3 Contrôle d'accès par rôle (RBAC)

Trois rôles, périmètres distincts. `admin`, `artists` et `news` ont des périmètres d'action différents, appliqués via `requireRole` sur chaque route sensible `artists` et `news` sont chacun cantonnés à leur domaine, `admin` a accès à la gestion des utilisateurs en plus.

La protection frontend n'est qu'un confort. `useRoleGuard` et `filterNavByRole` évitent d'afficher des pages ou des liens inaccessibles à l'utilisateur courant, mais n'empêchent rien côté serveur.

Un appel direct à l'API avec un rôle insuffisant reçoit un 403 (`requireRole`), quelle que soit l'interface utilisée pour l'émettre.

VIII.4 Validation des entrées

Validation Zod backend — le verrou réel. `validateBody` applique un schéma Zod à chaque route qui accepte un corps de requête, rejetant toute donnée non conforme avec un 400 avant d'atteindre le contrôleur cette validation s'applique quel que soit l'appelant (frontend officiel ou requête directe à l'API).

Validation frontend — confort UX uniquement. `src/functions/validation.ts` (`isEmail`, `isMaxLength`, `isEmpty`...) désactive les boutons de soumission avant l'envoi, mais n'a aucune valeur de sécurité elle peut être contournée en appelant l'API directement, d'où la nécessité de la validation Zod backend en dernier rempart.

Protection contre l'injection SQL. Toutes les requêtes passent par la fonction `query<T>()`, qui utilise systématiquement des requêtes paramétrées (`$1`, `$2`...) via `node-postgres` aucune concaténation de chaîne SQL n'est utilisée nulle part dans le code, ce qui élimine structurellement le risque d'injection.

VIII.5 Rate limiting

rateLimitLogin. Limite à 5 tentatives par tranche de 10 minutes et par adresse IP, appliqué sur `POST /admin/auth/login` et `POST /admin/auth/forgot-password` les deux points d'entrée les plus exposés à une attaque par force brute. Au-delà de la limite, l'API répond 429 avec un message dédié plutôt que de continuer à traiter les tentatives.

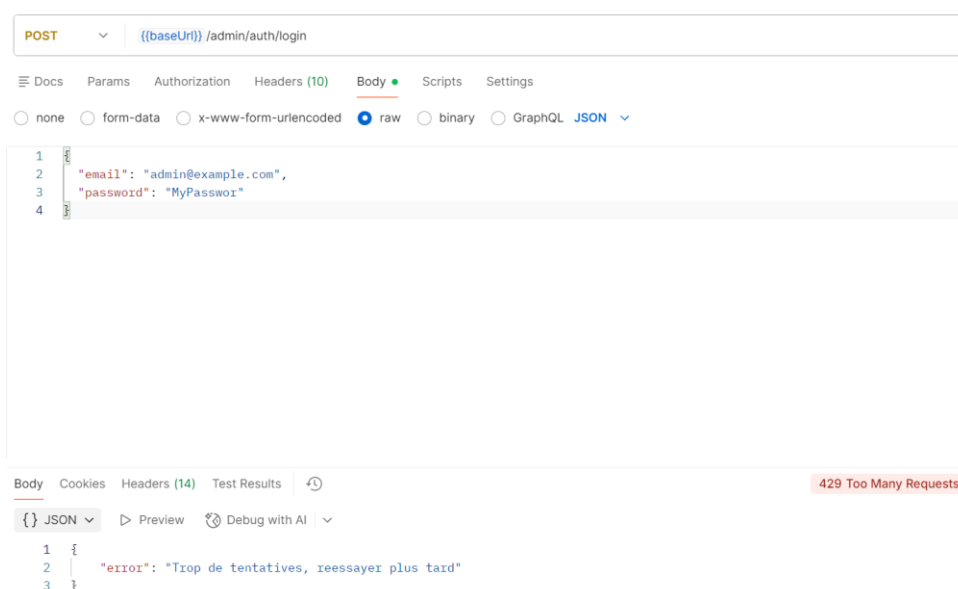


Figure — Réponse 429 après dépassement de la limite de tentatives — Vindhellfest

VIII.6 Upload sécurisé

Filtre MIME, taille et conversion. upload n'accepte que image/jpeg, image/png et image/webp, limite chaque fichier à 5 Mo, et `savelsImage` convertit systématiquement le fichier en WebP via `sharp`.

Nommage et stockage. Chaque fichier est renommé avec un UUID généré côté serveur le nom original envoyé par le client n'est jamais utilisé, ce qui empêche à la fois les collisions et toute tentative de path traversal via un nom de fichier malveillant.

VIII.7 Variables d'environnement

Aucun secret dans le code source. Toutes les valeurs sensibles (secrets JWT, identifiants base de données, SMTP) vivent exclusivement dans `.env`, `.env.backend` et `.env.frontend`, exclus du versionnement Git.

Validation fail-fast. `env.ts` (V.3.2) définit un schéma Zod listant 19 variables (18 obligatoires, `PORT` étant la seule optionnelle), `validateEnv()` est appelée avant la création de l'application (`index.ts`) et lève une erreur explicite listant les variables manquantes si la validation échoue le serveur ne démarre jamais dans un état de configuration incomplet.

Variables concernées. Base de données (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`), JWT (`JWT_ACCESS_SECRET`, `JWT_ACCESS_EXPIRES_IN`), cookie (`COOKIE_ACCESS_TOKEN_NAME`, `COOKIE_ACCESS_TOKEN_SECURE`, `COOKIE_ACCESS_TOKEN_SAME_SITE`), session (`SESSION_EXPIRES_IN`), CORS (`FRONTEND_ORIGIN`) et SMTP (`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `CONTACT_EMAIL`).

VIII.8 CORS

Origine restreinte. `app.ts` implémente un CORS restreint à une seule origine toute requête provenant d'une autre origine ne reçoit pas l'en-tête `Access-Control-Allow-Origin` et est bloquée par le navigateur. `Access-Control-Allow-Credentials: true` est ajouté uniquement quand l'origine correspond, condition nécessaire pour que le navigateur transmette le cookie `httpOnly` en cross-origin.

Protection complémentaire contre le CSRF. En limitant les origines autorisées et en couplant `credentials:true` à `sameSite=lax`, une requête forgée depuis un site tiers ne peut ni recevoir de réponse lisible (bloquée par CORS) ni transmettre le cookie (bloqué par `SameSite`).

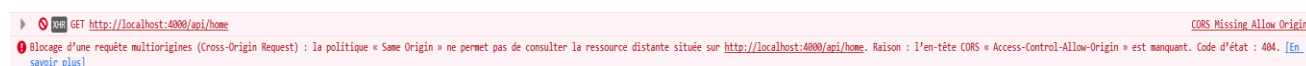


Figure — Erreur CORS dans la console du navigateur (`Access-Control-Allow-Origin` manquant) — Vindhellfest

Requête effectuée depuis une origine (`http://127.0.0.1:3000`) différente de l'origine autorisée par le backend (`http://localhost:3000`, valeur de `FRONTEND_ORIGIN`) : le serveur ne renvoie pas l'en-tête `Access-Control-Allow-Origin` attendu, et le navigateur bloque la réponse.

VIII.9 Hachage des mots de passe

bcrypt et salage automatique. Deux fonctions appellent `bcrypt.hash(password, 10)` un coût de 10 rounds, avec un sel généré et intégré automatiquement par `bcrypt` dans le hash produit, sans gestion manuelle du sel.

Mot de passe en clair jamais persisté. Le mot de passe temporaire généré à la création d'un compte ou lors d'une réinitialisation (`generateTemporaryPassword`) n'est jamais écrit en base il est haché avant l'insertion SQL et n'existe en clair que le temps de sa transmission unique par email.

```
vindhellfest=# SELECT email, password_hash FROM users;
```

email	password_hash
admin@example.com	\$2b\$10\$3Br0yYg6p5EcLXJaHT/mp00qq6A5niWuCpT8hM2FXlkl2Yj0x.A7.
artists@example.com	\$2b\$10\$3Br0yYg6p5EcLXJaHT/mp00qq6A5niWuCpT8hM2FXlkl2Yj0x.A7.
news@example.com	\$2b\$10\$3Br0yYg6p5EcLXJaHT/mp00qq6A5niWuCpT8hM2FXlkl2Yj0x.A7.

```
(3 rows)
```

```
vindhellfest=#
```

Figure — Colonne `password_hash`, format `bcrypt` (`$2b$10$...`) — Vindhellfest

IX. Plan de tests et jeu d'essai

IX.1 Stratégie de test

Approche pyramidale. La stratégie de test du projet suit une pyramide de tests classique, une base de tests unitaires (middlewares et services isolés, dépendances mockées), une couche intermédiaire de tests d'intégration (API testée de bout en bout via Supertest, avec une vraie base de données PostgreSQL de test), et pas de tests end-to-end.

Deux niveaux distincts côté backend.

Les tests unitaires (dossier tests/unit/) vérifient le comportement interne d'une fonction ou d'un middleware en isolation, sans démarrage de serveur, la base de données, nodemailer et sharp sont mockés.

Les tests d'intégration (dossier tests/integration/) font traverser chaque requête toute la chaîne Express (middlewares, contrôleur, base de données réelle) via Supertest, seuls l'envoi d'email et le traitement d'image restent mockés globalement.

Frameworks utilisés. Vitest sert de runner et de bibliothèque d'assertions/mocks (vi.mock, vi.fn, vi.mocked) à la fois côté backend et côté frontend. Supertest est utilisé côté backend pour simuler des requêtes HTTP contre le serveur Express. Côté frontend, React Testing Library complète Vitest pour tester le rendu et les interactions des composants.

Volume de tests. Le projet compte au total 39 fichiers de test : 7 fichiers de tests unitaires et 6 fichiers de tests d'intégration côté backend, et 26 fichiers côté frontend (18 composants, 4 hooks, 4 fonctions utilitaires).

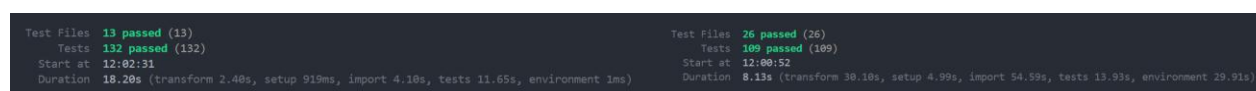


Figure — `npm test` (backend, 132 tests) et `npm run test:run` (frontend, 109 tests), suites passées avec succès — Vindhellfest

Le détail complet de cette exécution (sortie de `npm test` côté backend et `npm run test:run` côté frontend) est disponible en Annexe E (Rapport d'exécution des tests).

IX.2 Tests unitaires — backend

Organisation. Les tests unitaires backend se trouvent dans tests/unit/ et se répartissent en deux familles : les middlewares (4 fichiers) et les services (3 fichiers). Aucun serveur Express ne démarre pour ces tests, les dépendances externes (base de données, bcrypt, JWT, sharp, fs/promises, nodemailer) sont mockées selon le besoin de chaque fichier.

IX.2.1 Middlewares

Quatre middlewares testés isolément. auth.middleware.test.ts, requireRole.middleware.test.ts, validateBody.middleware.test.ts et validateUuidParam.middleware.test.ts couvrent chacun un middleware indépendamment du serveur Express, en simulant manuellement les objets req/res/next.

auth et optionalAuth. Vérifie que auth bloque la requête (401) si le cookie est absent, si le token est invalide, ou si l'utilisateur n'existe plus en base, et qu'elle passe normalement si le token est valide et l'utilisateur existe (res.locals peuplé, next() appelé).

Il vérifie aussi que optionalAuth ne bloque jamais la requête dans ces mêmes situations d'échec elle continue simplement sans utilisateur identifié tout en peuplant res.locals normalement quand l'authentification réussit.

requireRole. Vérifie que la requête passe next() si le rôle de l'utilisateur fait partie des rôles autorisés, et qu'elle est bloquée par une erreur 403 Forbidden sinon, que ce soit parce que le rôle n'est pas autorisé, ou parce qu'aucun rôle n'est présent dans res.locals (utilisateur non authentifié en amont).

validateBody. Vérifie que la requête passe next() si le corps respecte le schéma Zod, avec req.body remplacé par la version validée et transformée par Zod (ex. espaces retirés via trim), et qu'elle est bloquée par une erreur 400 sinon, que ce soit à cause d'un champ invalide ou d'un corps vide.

validateUuidParam. Vérifie que la requête passe next() si le paramètre d'URL est un UUID valide, et qu'elle est bloquée par une erreur 400 sinon (paramètre absent ou valeur qui n'est pas un UUID). Teste aussi qu'un nom de paramètre personnalisé (ex. artistId) est correctement pris en compte.

IX.2.2 Services

Trois services testés. imageUpload.service.test.ts, user.service.test.ts et mailer.service.test.ts vérifient chacun le comportement d'un service en isolation, avec ses dépendances externes mockées (sharp, fs/promises, bcrypt, nodemailer).

imageUpload.service. Vérifie que saveImage crée le dossier de destination si besoin (mkdir récursive), convertit et redimensionne l'image via sharp avant de l'écrire au bon chemin, puis retourne une URL publique unique à chaque appel (UUID différent).

Il vérifie aussi que deleteImage supprime le fichier au bon chemin, reconstruit à partir de l'URL stockée.

user.service. Vérifie que l'email ou le display_name sont libres, déjà pris, ou exclus de la vérification via excludeId (cas d'une modification), puis que l'utilisateur existe ou non (404 si absent).

Il vérifie aussi que le mot de passe temporaire fait une longueur minimale de 12 caractères et qu'il est différent à chaque appel, ainsi que le hachage du mot de passe via Bcrypt, avec hash différent à chaque appel même pour un mot de passe identique.

mailer.service. Vérifie que sendPasswordResetEmail et sendWelcomeEmail envoient bien le mail au bon destinataire, avec le mot de passe et le nom de l'utilisateur inclus dans le corps du message.

Il vérifie aussi que sendContactEmail préfixe le sujet par [Contact], règle le replyTo sur l'email de l'expéditeur du formulaire.

IX.3 Tests d'intégration — backend (Vitest + Supertest)

Organisation. Les tests d'intégration backend se trouvent dans `tests/integration/` (6 fichiers, `admin/` et `public/`) et utilisent Supertest contre une vraie instance PostgreSQL de test. Nodemailer et sharp restent mockés (`setup.ts`) ; la base est réinitialisée entre chaque test (TRUNCATE ... RESTART IDENTITY CASCADE).

IX.3.1 Routes publiques

public.test.ts. Vérifie les listes et détails d'artistes et de news (cas nominal, liste vide, UUID inexistant → 404), ainsi que le filtrage des brouillons sur les news : un visiteur non connecté ne voit que les news publiées, un compte admin ou news voit aussi les brouillons.

contact.test.ts. Vérifie la soumission valide (200, email envoyé) ainsi que les rejets à 400 quand le courriel, le nom, le sujet ou le message ne respectent pas le schéma Zod, ou quand un champ obligatoire est manquant.

IX.3.2 Authentification

auth.test.ts Vérifie que POST `/login` pose le cookie de session en cas de succès (200), et le refuse (401) si l'email ou le mot de passe est incorrect, ou (400) si le corps est invalide. Que POST `/logout` révoque la session (200) et refuse la requête sans cookie (401) et que GET `/me` renvoie les informations de l'utilisateur connecté (200), en la refusant (401) si le cookie est absent, si la session a été révoquée, ou si elle a expiré.

Il vérifie aussi que POST `/forgot-password` envoie l'email de réinitialisation si le compte existe (200), répond 404 sinon et 400 si le corps est invalide ; et que PATCH `/password` met à jour le mot de passe (200), le refuse (401) si l'ancien mot de passe est incorrect ou si le cookie est absent, et (400) si le corps est invalide.

IX.3.3 Administration

artists.test.ts. Vérifie que POST `/admin/artists` crée l'artiste avec les bons champs (201), et le refuse si l'image est absente (400), si le corps est invalide (400), ou si la limite de 2 artistes mis en avant est atteinte (409), les trois routes (POST, PATCH, DELETE) partageant les mêmes contrôles d'accès : 403 pour le rôle news, 401 sans cookie.

Il vérifie aussi que PATCH `/admin/artists/:id` modifie l'artiste, avec ou sans nouvelle image (200), et que DELETE `/admin/artists/:id` le supprime (200) les deux renvoyant 404 si l'artiste n'existe pas et 400 si l'UUID est mal formé.

news.test.ts. Vérifie que POST `/admin/news` crée la news avec les bons champs (201), et la refuse si l'image est absente (400) ou si le corps est invalide (400). Les trois routes (POST, PATCH, DELETE) partageant les mêmes contrôles d'accès : 403 pour le mauvais rôle, 401 sans cookie.

Il vérifie aussi que PATCH `/admin/news/:id` modifie la news, avec ou sans nouvelle image (200), et que DELETE `/admin/news/:id` la supprime (200), les deux renvoyant 404 si la news n'existe pas et 400 si l'UUID est mal formé.

users.test.ts. Vérifie que GET `/admin/users` renvoie la liste (200) et que POST `/admin/users` crée l'utilisateur avec les bons champs (201), en le refusant si l'email

(409) ou le `display_name` (409) est déjà pris, ou si le corps est invalide (400), ces deux routes étant bloquées (403) pour les autres rôles.

Il vérifie aussi que `PATCH /admin/users/:id` modifie l'utilisateur (200) et que `DELETE /admin/users/:id` le supprime (200), les deux renvoyant 404 si l'utilisateur n'existe pas et 400 si l'UUID (ou, pour `PATCH`, le corps) est invalide, ces deux routes étant bloquées (403) pour le rôle `artists`.

Les quatre routes refusent (401) toute requête sans cookie.

IX.4 Tests de composants et fonctions — frontend (React Testing Library)

Organisation. Les tests frontend se trouvent dans `apps/frontend/tests/` (26 fichiers) et utilisent Vitest et React Testing Library pour le rendu et la simulation d'interactions utilisateur. Ils se répartissent en trois dossiers, `components/` (18 fichiers), `hooks/` (4 fichiers) et `functions/` (4 fichiers).

IX.4.1 Composants UI

Composants réutilisables testés (cf. V.2.6). `AddArtistModal`, `AddNewsModal`, `ArtistsContent`, `ContactUs`, `DeleteModal`, `Footer`, `ForgotPassword`, `NewsContent`.

Pages et composants propres à une route (dossier `app/**`, hors des vingt de V.2.6). `AddUserModal`, `AdminArtistDetailPage`, `AdminNewsDetailPage`, `ArtistsPage`, `ChangePasswordModal`, `DashboardContent`, `LoginPage`, `NewsPage`, `UsersContent`, `UsersPage`.

Cas couverts. Chaque fichier vérifie le rendu initial, les interactions utilisateur (saisie, soumission de formulaire, ouverture/fermeture de modale, clic sur les boutons d'action) et les états d'erreur (message d'erreur affiché en cas d'échec de l'appel API, état de chargement).

Isolation du réseau. Les appels API sont mockés via `vi.mock`, ce qui permet de tester chaque composant indépendamment du backend réel et de simuler précisément les réponses de succès comme d'erreur.

IX.4.2 Hooks

4 hooks testés. `useDelete`, `useFetch`, `useMutation` et `useRoleGuard` sont testés indépendamment des composants qui les consomment. Les tests couvrent le cycle de vie de chaque hook (état initial, déclenchement de la requête), la gestion des états `loading/error`, et pour `useRoleGuard` la redirection lorsque le rôle de l'utilisateur connecté ne correspond pas au rôle attendu par la page.

IX.4.3 Fonctions utilitaires

4 fonctions testées. `formatDate.test.ts`, `getApiErrorMessage.test.ts`, `validation.test.ts` et `filterNavByRole.test.ts` vérifient chacun les cas nominaux et les cas limites de leur fonction respective, formatage de dates dans différents formats, extraction d'un message d'erreur lisible depuis une réponse API, règles de validation de champs (email, longueur), et filtrage des liens de navigation selon le rôle de l'utilisateur.

IX.5 Pipeline CI/CD et exécution automatisée

Le déclenchement du pipeline et le détail des étapes de chaque job (backend, frontend) sont décrits en V.6. Deux nuances propres aux tests, le job backend exécute les tests d'intégration contre une vraie instance PostgreSQL, d'où l'attente de sa disponibilité avant de lancer npm test, le job frontend s'exécute sans démarrer db ni backend, car ses tests reposent entièrement sur des appels réseau mockés.

 lint fix CI #159: Commit 6e83694 pushed by aurelienrnd	test	 Jun 15, 22:44 GMT+2  2m 25s	...
 update commetaire et doc CI #158: Commit 853ada2 pushed by aurelienrnd	test	 Jun 14, 19:07 GMT+2  1m 58s	...
 update commetaire et doc CI #157: Commit 0430884 pushed by aurelienrnd	test	 Jun 14, 17:46 GMT+2  2m 17s	...
 Merge branch 'dev' CI #156: Commit 8af796d pushed by aurelienrnd	main	 Jun 12, 15:43 GMT+2  2m 15s	...
 update commetaire et doc CI #155: Commit 81e9ccd pushed by aurelienrnd	dev	 Jun 12, 15:32 GMT+2  2m 4s	...
 Merge branch 'main' into dev CI #154: Commit 2c323d8 pushed by aurelienrnd	dev	 Jun 12, 14:39 GMT+2  2m 11s	...
 Dev CI #153: Pull request #4 synchronize by aurelienrnd	dev	 Jun 12, 14:39 GMT+2  2m 21s	...

Figure — Historique des exécutions du workflow CI (GitHub Actions) — Vindhellfest

IX.6 Jeu d'essai — Création d'un artiste avec image

Fonctionnalité choisie. Le jeu d'essai de la fonctionnalité la plus représentative, la création d'un artiste avec upload d'image (POST /admin/artists) a été retenue, elle mobilise l'ensemble des couches techniques du projet, contrôle de rôle (adminAuth), upload multipart (multer), validation Zod, conversion d'image (sharp, WebP qualité 80, redimensionnement 1600 px), et écriture transactionnelle en base sur deux tables (artists puis concerts). Les données ci-dessous sont extraites telles quelles du test d'intégration réel (tests/integration/admin/artists.test.ts), exécuté avec Vitest + Supertest ; le résultat obtenu correspond à l'exécution effective de la suite de tests

Route : POST /admin/artists.

Token : cookie de session valide généré par createAuthSession("artists") — utilisateur et session réels insérés en base, JWT signé avec le vrai secret, rôle artists.

Donnée de test : name="Test Artist", genre="Metal", origin="France", bio="Une biographie de test suffisamment longue.", description_media="Photo de scene", stage="MainStage", start_time="2025-08-01T18:00:00.000Z", end_time="2025-08-01T19:00:00.000Z".

Donnée en entrée	Résultat attendu	Résultat obtenu	Analyse des écarts
Cas nominal — POST /admin/artists (multipart)	HTTP 201	201 obtenu, conforme	Aucun écart. Comportement conforme à la spécification.
Cas limite — Image manquante	HTTP 400, message ARTIST_FILE_REQUIRED.	HTTP 400, message ARTIST_FILE_REQUIRED.	Aucun écart. Le contrôleur vérifie la présence du fichier avant toute écriture ou insertion en base.
Cas limite — Corps de requête invalide stage="ScenesInexistante" (valeur hors de l'ENUM autorisé).	HTTP 400, message VALIDATION_INVALID_BODY.	HTTP 400, message VALIDATION_INVALID_BODY.	Aucun écart. validateBody (Zod) rejette la requête avant le contrôleur ; l'image reste en mémoire (memoryStorage) et n'est jamais écrite sur le disque.
Cas limite — 3^e artiste créé avec is_featured="true" alors que 2 artistes "featured" existent déjà en base	HTTP 409, message "Deux artistes sont déjà mis en avant..." (ARTIST_FEATURED_LIMIT)	HTTP 409, message "Deux artistes sont déjà mis en avant..." (ARTIST_FEATURED_LIMIT)	Écart mineur sur la couverture de test : le test actuel vérifie bien le code 409 et le message d'erreur, mais pas que le fichier image a réellement été supprimé du disque. Piste d'amélioration future.
Cas limite — Rôle non autorisé Authentification : cookie de session valide, mais rôle news	HTTP 403, message FORBIDDEN.	HTTP 403, message FORBIDDEN.	Aucun écart. requireRole bloque la requête avant qu'elle n'atteigne validateBody ou le contrôleur.
Cas limite — Authentification absente Authentification : aucun	HTTP 401, message AUTH_MISSING_COOKIE.	HTTP 401, message AUTH_MISSING_COOKIE.	Aucun écart. Le middleware auth intercepte la requête avant tout traitement.

Conclusion. Ce jeu d'essai illustre, sur une fonctionnalité représentative de la complexité du projet, le passage d'une donnée d'entrée réelle à un résultat obtenu conforme à l'attendu. Les exécutions réelles de ces cas (terminal Vitest) sont visibles en Annexe E.

X. Veille sécurité / Veille technologique

La veille a été organisée via Feedly, structuré en deux dossiers thématiques : « Veille sécurité » (CERT-FR, Node.js Blog — Vulnerability Reports) et « Veille technologique » (GitHub Changelog, Docker, Next.js Blog, Node.js Blog, PostgreSQL news, React Blog), afin de suivre les évolutions et alertes des principales briques technologiques du projet. Deux situations rencontrées pendant le développement illustrent la mise en pratique de cette veille.

▼ Vindhellfest Veille sécurité	44
CERT-FR	43
Node.js Blog: Vulnerability Reports	1
▼ Vindhellfest Veille technologique	72
Archive: 2026 - GitHub Changelog	34
Docker	9
Next.js Blog	3
Node.js Blog	8
PostgreSQL news	18
React Blog	

Figure — Organisation Feedly, dossiers Veille sécurité et Veille technologique — Vindhellfest

Absence de hot reload en environnement Docker (cf. III.5). La recherche de solution s'est appuyée sur l'issue GitHub officielle du dépôt vercel/next.js, [#80665](#) (« Docs: how to enable turbopack's polling feature for the file watcher? »), dans laquelle plusieurs développeurs rencontrant le même symptôme en environnement Docker confirment que le moteur Turbopack ne respecte pas correctement le mécanisme de polling, contrairement à Webpack, et documentent le même contournement que celui retenu ici, désactiver Turbopack et revenir à Webpack le temps que le problème soit corrigé en amont.

Alerte npm audit — vulnérabilité PostCSS (CVE-2026-41305).

npm audit a signalé une faille de sécurité modérée dans postcss (l'outil qui transforme les fichiers CSS au moment de la compilation). Cette dépendance n'a pas été installée directement : elle est embarquée automatiquement par Next.js. Nature de la faille. Dans certains cas, postcss pourrait laisser passer du code malveillant caché dans du CSS non fiable, ouvrant la porte à une injection de script (XSS) si ce CSS vient d'une source externe non contrôlée.

Analyse du risque pour le projet. Dans Vindhellfest, postcss ne traite que les fichiers CSS du projet, écrits en interne, uniquement au moment de la compilation : aucun utilisateur ne peut soumettre de CSS. Le scénario d'attaque décrit par l'alerte ne s'applique donc pas à l'application.

Raison de ne pas appliquer le correctif. Le correctif automatique (npm audit fix --force) obligerait à revenir à Next.js 9.3.3, une version très ancienne, ce qui casserait le projet pour un risque qui ne peut pas se produire dans ce contexte.

Décision retenue. Ne pas appliquer ce correctif, et surveiller une future version de Next.js qui embarquera une version corrigée de postcss.

XI. Annexes

A. Schémas de base de données

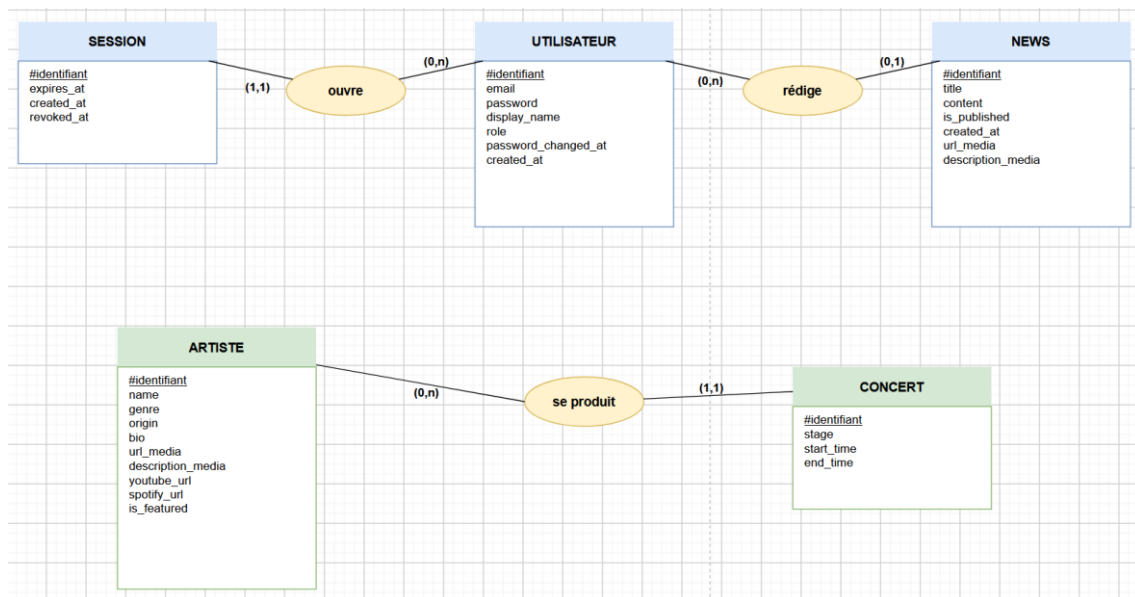


Figure — Modèle Conceptuel de Données (MCD) — Vindhellfest

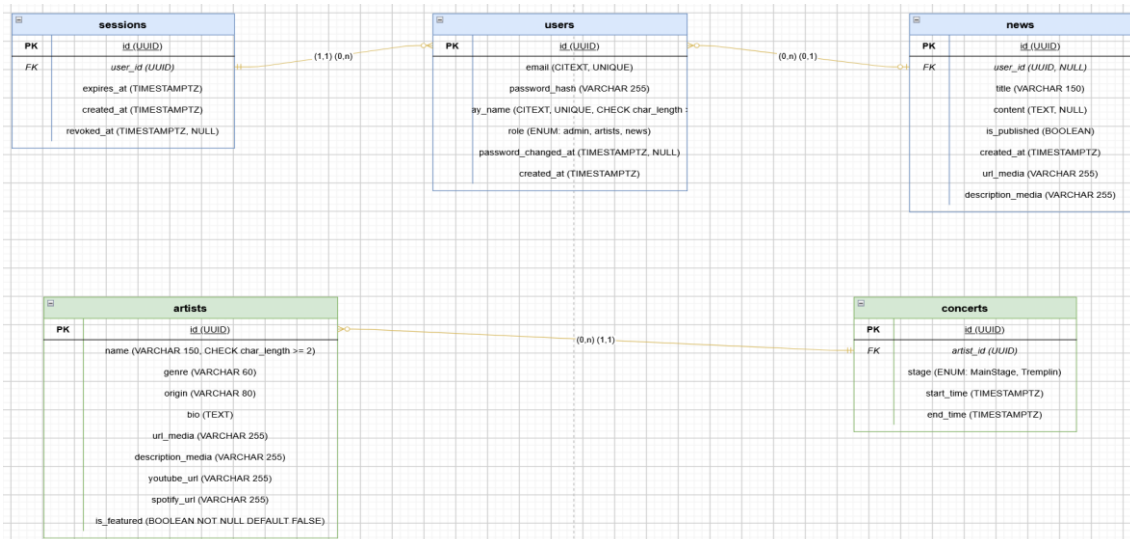


Figure — Modèle Logique de Données (MLD) — Vindhellfest

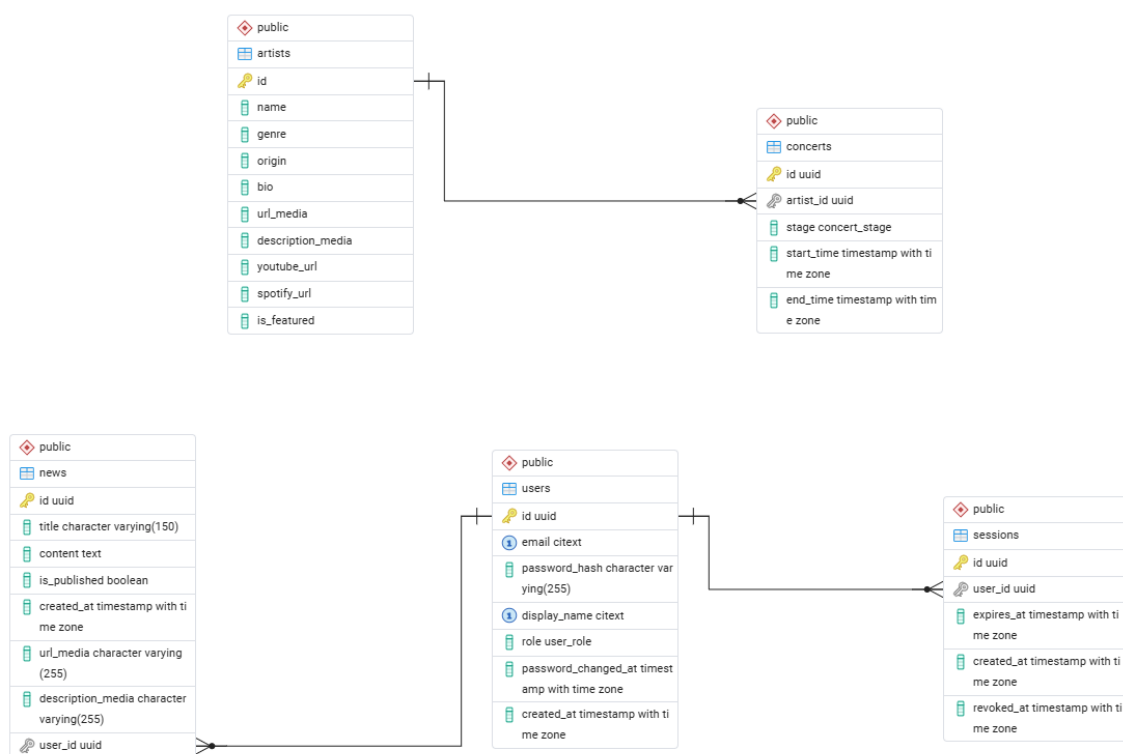


Figure — Modèle Physique de Données (MPD) — Vindhellfest

Scripts SQL d'initialisation (bd/init/, 01 à 05). Les cinq scripts ci-dessous créent respectivement les tables users, sessions, news, artists et concerts, exécutés dans cet ordre par le conteneur db au premier démarrage. Les quatre scripts de seed complémentaires (06 à 09) ne sont pas reproduits ici (voir V.4).

```
-- Extensions necessaires
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- Pour les UUID aleatoires (gen_random_uuid)
CREATE EXTENSION IF NOT EXISTS citext; -- Pour rendre les emails insensibles a la casse

-- //NOTE : Utiliser uniquement en phase de developpement.
-- Elle supprime la table si elle existe deja, afin d'éviter des erreurs lors des modifications du schema.
DROP TABLE IF EXISTS users CASCADE;
DROP TYPE IF EXISTS user_role CASCADE;

-- Type ENUM pour les roles utilisateur -- valeurs autorisees uniquement
CREATE TYPE user_role AS ENUM ('admin', 'artists', 'news');

-- Table des utilisateurs administrateurs
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email CITEXT NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  display_name CITEXT NOT NULL UNIQUE CHECK (char_length(display_name) >= 2),
  role user_role NOT NULL,
  password_changed_at TIMESTAMPTZ NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Figure — Script SQL — 01_user_schema.sql — Vindhellfest


```
-- Extensions necessaires
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- //NOTE : Utiliser uniquement en phase de developpement.
-- Elle supprime la table si elle existe deja, afin d'eviter des erreurs lors des modifications du schema.
DROP TABLE IF EXISTS sessions CASCADE;

-- Table des sessions utilisateur
CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- Identifiant unique genere automatiquement
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, -- Reference a l'utilisateur, suppression en cascade
  expires_at TIMESTAMPTZ NOT NULL, -- Date/heure d'expiration de la session
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), -- Date de creation
  revoked_at TIMESTAMPTZ NULL, -- Date/heure de revocation de la session (NULL si active)

  -- Contraintes de coherence
  CONSTRAINT chk_session_expires_after_created CHECK (expires_at > created_at), -- La session doit expirer apres sa creation
  CONSTRAINT chk_session_revoked_after_created CHECK (revoked_at IS NULL OR revoked_at >= created_at) -- La session revoquee doit l'etre apres sa creation
);

-- Indexes pour optimiser les requêtes courantes
CREATE INDEX idx_sessions_user_id ON sessions(user_id);
CREATE INDEX idx_sessions_expires_at ON sessions(expires_at);
CREATE INDEX idx_sessions_revoked_at ON sessions(revoked_at);
```

Figure — Script SQL — 02_sessions_schema.sql — Vindhellfest

```
-- Extensions necessaires
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- //NOTE : Utiliser uniquement en phase de developpement.
-- Elle supprime la table si elle existe deja, afin d'eviter des erreurs lors des modifications du schema.
DROP TABLE IF EXISTS news CASCADE;

-- Table des news
CREATE TABLE news (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- Identifiant unique genere automatiquement
  title VARCHAR(150) NOT NULL CHECK (char_length(title) >= 2), -- Titre affiche
  content TEXT NULL, -- Contenu de la news
  is_published BOOLEAN NOT NULL DEFAULT FALSE, -- Statut de publication
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(), -- Date de creation
  url_media VARCHAR(255) NOT NULL, -- URL/chemin du media associe
  description_media VARCHAR(255) NOT NULL, -- Texte alternatif / description courte
  user_id UUID NULL REFERENCES users(id) ON DELETE SET NULL -- Reference l'utilisateur createur, NULL si supprime
);

-- Indexes pour optimiser les requetes courantes
CREATE INDEX idx_news_created_at ON news(created_at);
CREATE INDEX idx_news_is_published ON news(is_published);
```

Figure — Script SQL — 03_news_schema.sql — Vindhellfest

```
Connect to PostgreSQL
-- Extensions necessaires
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- //NOTE : Utiliser uniquement en phase de developpement.
-- Elle supprime la table si elle existe deja, afin d'eviter des erreurs lors des modifications du schema.
DROP TABLE IF EXISTS artists CASCADE;

-- Table des artistes
CREATE TABLE artists (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- Identifiant unique genere automatiquement
  name VARCHAR(150) NOT NULL CHECK (char_length(name) >= 2), -- Nom de l'artiste, sensible a la casse
  genre VARCHAR(60) NOT NULL, -- Genre musical
  origin VARCHAR(80) NOT NULL, -- Origine/pays/ville
  bio TEXT NOT NULL, -- Biographie
  url_media VARCHAR(255) NOT NULL, -- URL/chemin du media associe
  description_media VARCHAR(255) NOT NULL, -- Texte alternatif / description courte
  youtube_url VARCHAR(255), -- Lien YouTube (optionnel)
  spotify_url VARCHAR(255), -- Lien Spotify (optionnel)
  is_featured BOOLEAN NOT NULL DEFAULT FALSE -- Mis en avant sur la page d'accueil (max 2)
);

-- Indexes pour optimiser les requetes courantes
CREATE INDEX idx_artists_genre ON artists(genre);
CREATE INDEX idx_artists_name ON artists(name);
CREATE INDEX idx_artists_is_featured ON artists(is_featured);

-- Trigger : limite a 2 le nombre d'artistes mis en avant simultanement
CREATE OR REPLACE FUNCTION check_featured_limit()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.is_featured = TRUE THEN -- Si l'artiste est mis en avant
    IF (SELECT COUNT(*) FROM artists WHERE is_featured = TRUE AND id != NEW.id) >= 2 THEN -- Verifie le nombre d'artistes deja mis en avant (exclut l'artiste en cours de modification)
      RAISE EXCEPTION 'featured_limit_reached'; -- Lève une exception si la limite est atteinte
    END IF;
  END IF;
  RETURN NEW; -- Retourne la ligne modifiée pour l'insertion ou la mise à jour
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_featured_limit -- Nom du trigger
BEFORE INSERT OR UPDATE ON artists
FOR EACH ROW EXECUTE FUNCTION check_featured_limit();
```

Figure — Script SQL — 04_artist_schema.sql — Vindhellfest

```
❖ Connect to PostgreSQL
-- Extensions nécessaires
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS citext;
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- //NOTE : Utiliser uniquement en phase de developpement.
-- Elle supprime la table si elle existe deja, afin d'éviter des erreurs lors des modifications du schema.
DROP TABLE IF EXISTS concerts CASCADE;
DROP TYPE IF EXISTS concert_stage CASCADE;

-- Type ENUM pour les scenes du festival – valeurs autorisees uniquement
CREATE TYPE concert_stage AS ENUM ('MainStage', 'Tremplin');

-- Table des concerts
CREATE TABLE concerts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), -- Identifiant unique genere automatiquement
  artist_id UUID NOT NULL REFERENCES artists(id) ON DELETE CASCADE, -- Reference l'artiste
  stage concert_stage NOT NULL, -- Scene du festival
  start_time TIMESTAMPTZ NOT NULL, -- Date/heure de debut
  end_time TIMESTAMPTZ NOT NULL, -- Date/heure de fin

  -- Contraintes de coherence
  CONSTRAINT chk_concert_end_after_start CHECK (end_time > start_time),
  CONSTRAINT no_overlap_stage EXCLUDE USING gist (
    stage WITH =,
    tstzrange(start_time, end_time, '[') WITH &&
  ),
  CONSTRAINT no_overlap_artist EXCLUDE USING gist (
    artist_id WITH =,
    tstzrange(start_time, end_time, '[') WITH &&
  )
);

-- Indexes pour optimiser les requetes courantes
CREATE INDEX idx_concerts_stage_start ON concerts(stage, start_time);
CREATE INDEX idx_concerts_start_time ON concerts(start_time);
```

Figure — Script SQL — 05_concert_schema.sql — Vindhellfest

B. Maquettes de l'application

Ensemble complet. L'application compte treize écrans, chacun décliné en zoning et en wireframe. Les treize pages sont reproduites ci-dessous, organisées par espace (public puis administration), chacune en version zoning (gauche) puis wireframe (droite).

Espace public



Figure — Page d'accueil, zoning (gauche) et wireframe (droite) — Vindhellfest

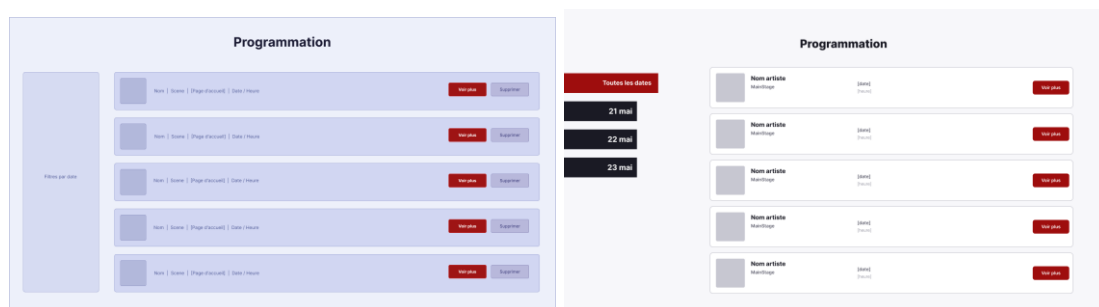


Figure — Page Programmation, zoning (gauche) et wireframe (droite) — Vindhellfest

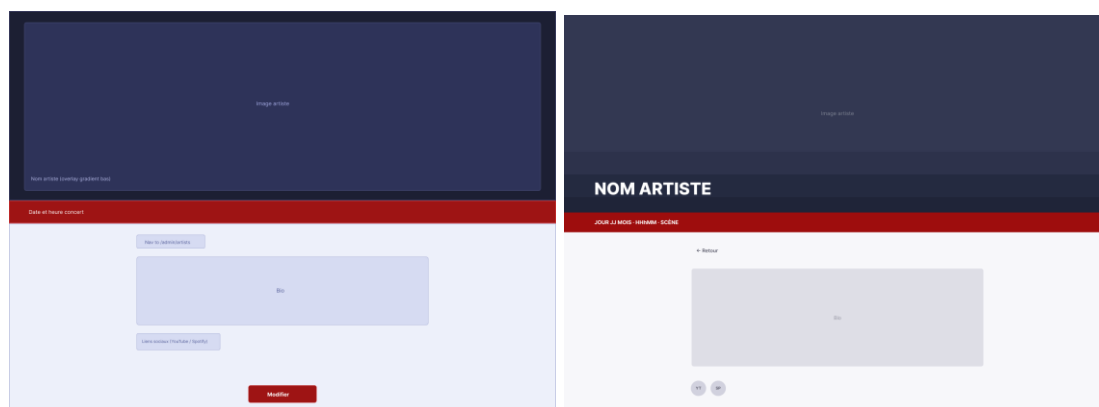


Figure — Fiche artiste (public), zoning (gauche) et wireframe (droite) — Vindhellfest

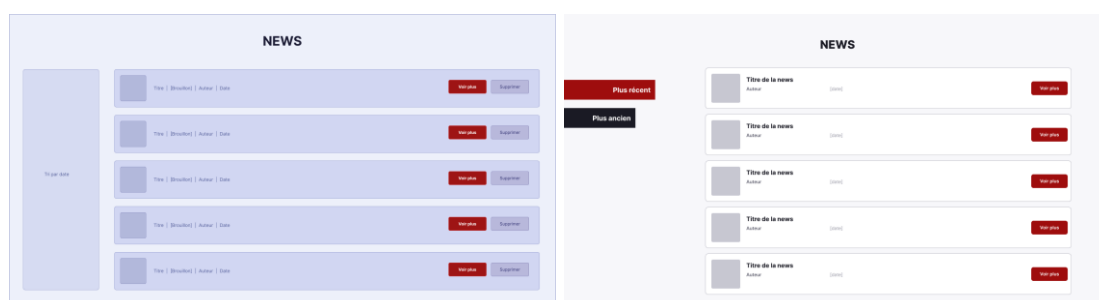


Figure — Liste des actualités (public), zoning (gauche) et wireframe (droite) — Vindhellfest

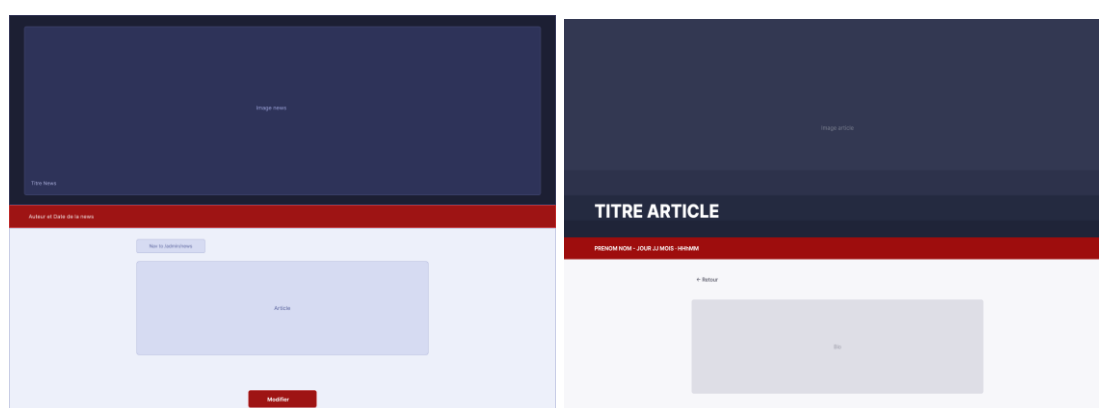


Figure — Détail d'une actualité (public), zoning (gauche) et wireframe (droite) — Vindhellfest

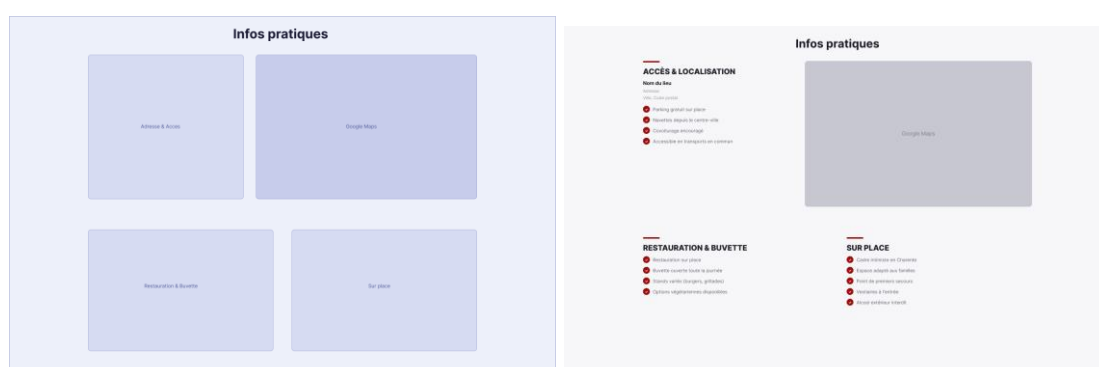


Figure — Infos pratiques (public), zoning (gauche) et wireframe (droite) — Vindhellfest

Espace administration

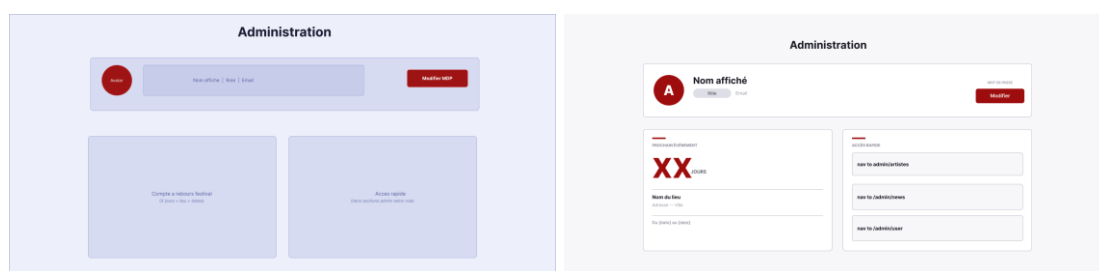


Figure — Tableau de bord admin, zoning (gauche) et wireframe (droite) — Vindhellfest

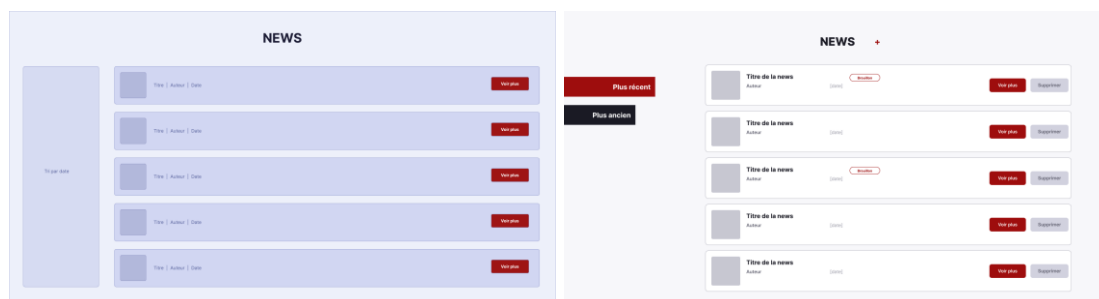


Figure — Gestion des news (admin), zoning (gauche) et wireframe (droite) — Vindhellfest

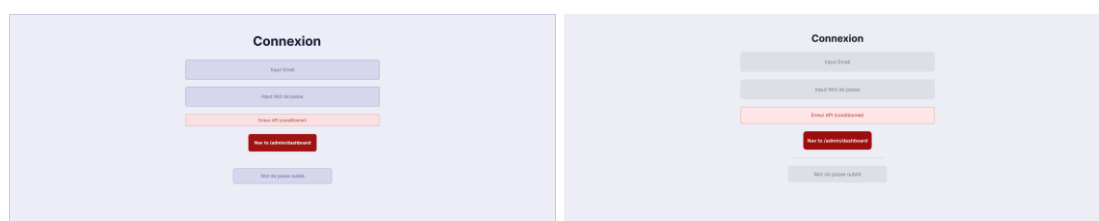


Figure — Connexion (admin), zoning (gauche) et wireframe (droite) — Vindhellfest

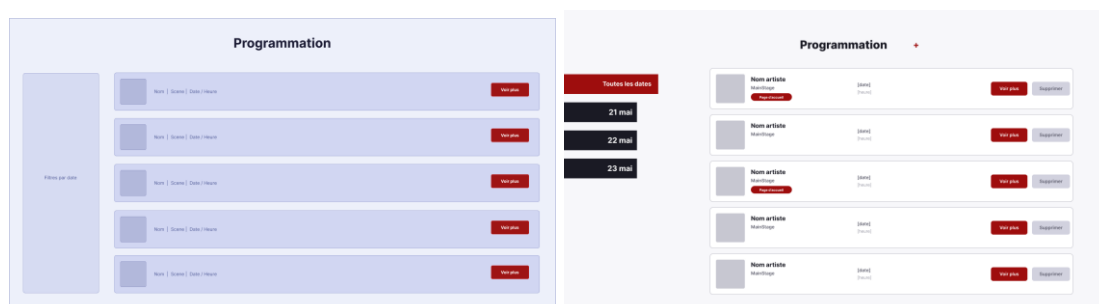


Figure — Liste des artistes (admin), zoning (gauche) et wireframe (droite) — Vindhellfest

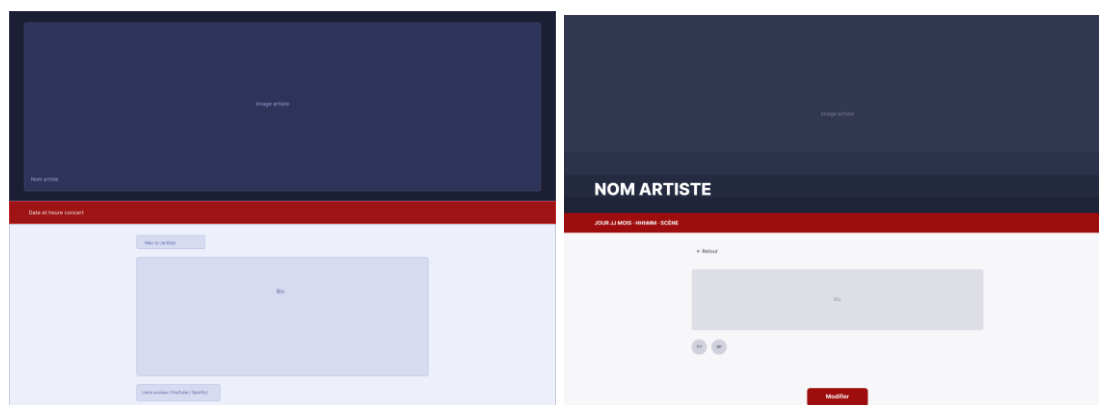


Figure — Fiche artiste (admin), zoning (gauche) et wireframe (droite) — Vindhellfest

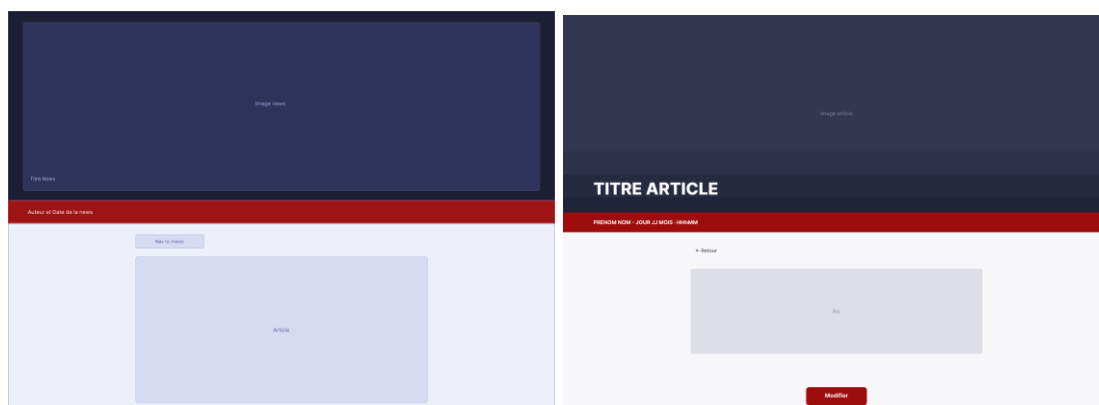


Figure — Détail d'une actualité (admin), zoning (gauche) et wireframe (droite) — Vindhellfest

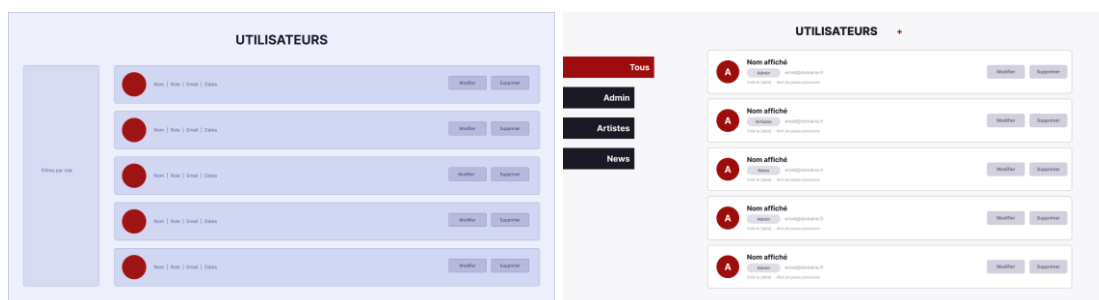


Figure — Liste des utilisateurs (admin), zoning (gauche) et wireframe (droite) — Vindhellfest

C. Captures d'écran de l'application

Captures réelles. Les captures ci-dessous proviennent de l'application Vindhellfest exécutée en local (docker compose up), avec les données de seed du dépôt (bd/init/06 à 09). Organisées par espace, elles couvrent les pages principales en vue desktop ; quelques vues mobile (iPhone) sont ajoutées en fin de section pour illustrer le responsive.

Espace public

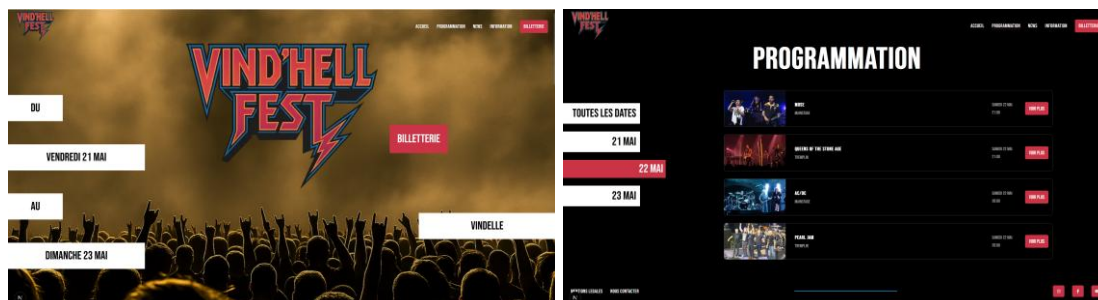


Figure — Accueil et Programmation — Vindhellfest (capture réelle)

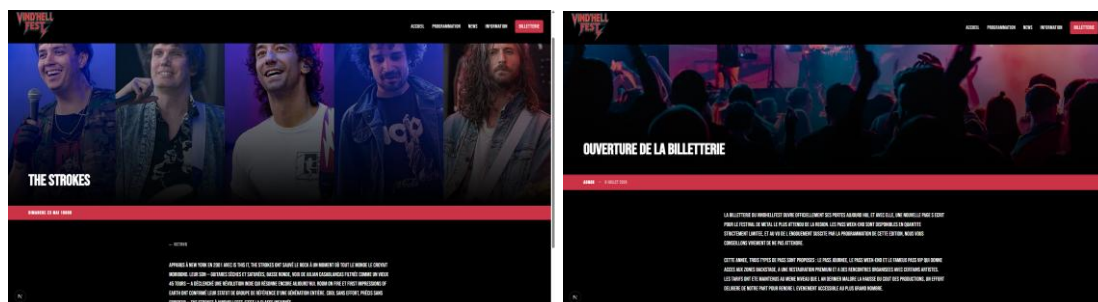


Figure — Fiche artiste (The Strokes) et détail d'une actualité — Vindhellfest (capture réelle)

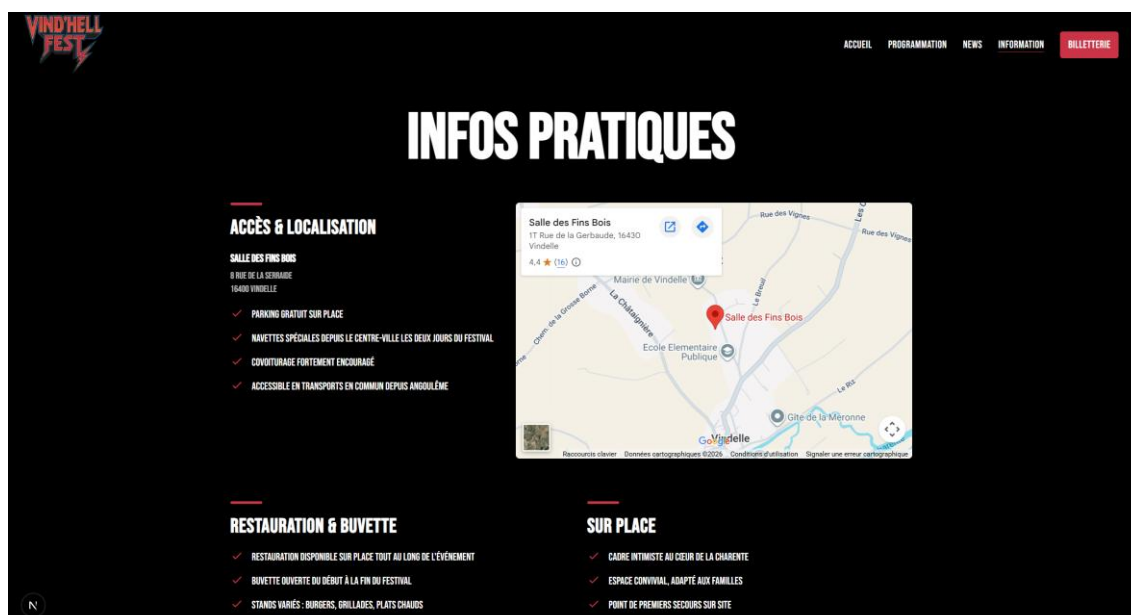


Figure — Infos pratiques — Vindhellfest (capture réelle)

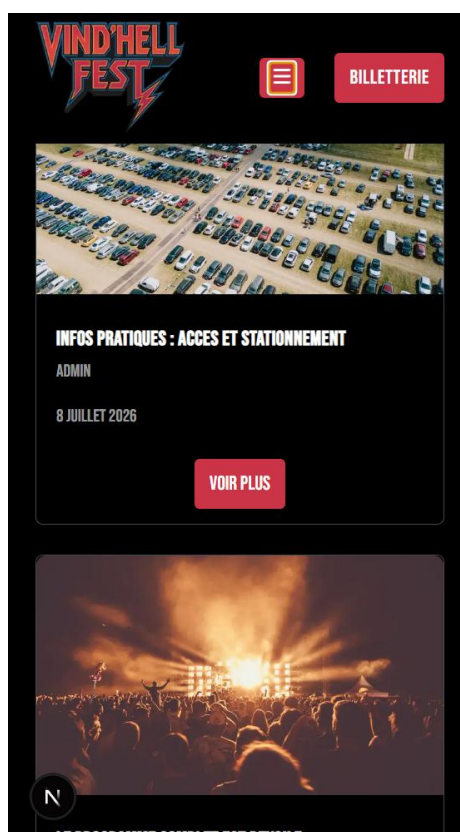


Figure — Liste des actualités, vue mobile — Vindhellfest (capture réelle)

Espace administration

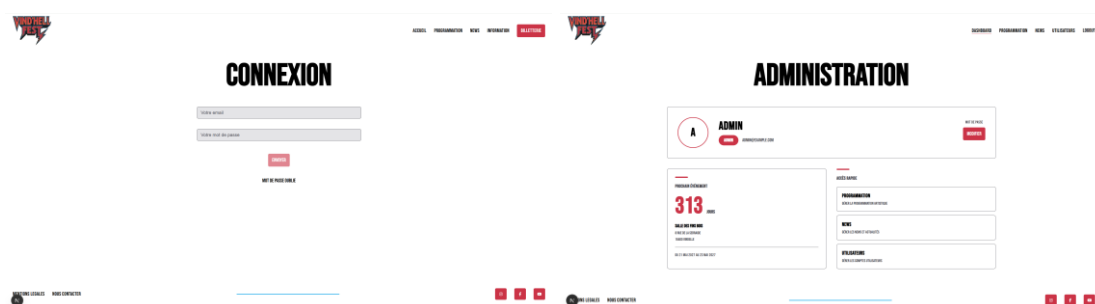


Figure — Connexion et tableau de bord admin — Vindhellfest (capture réelle)

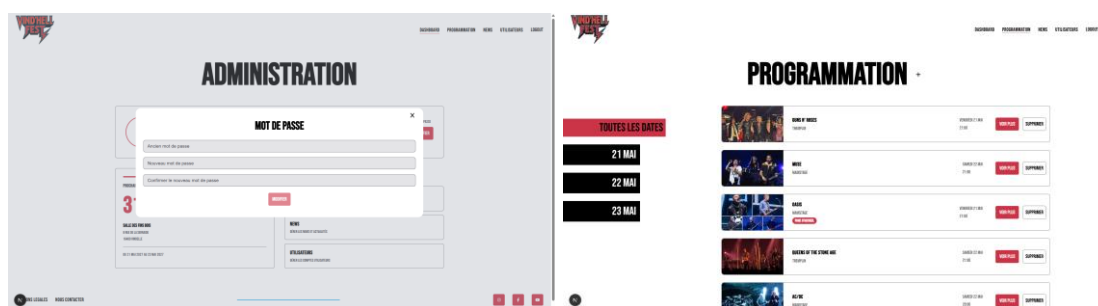


Figure — Modal changement de mot de passe et liste des artistes (admin) — Vindhellfest (capture réelle)

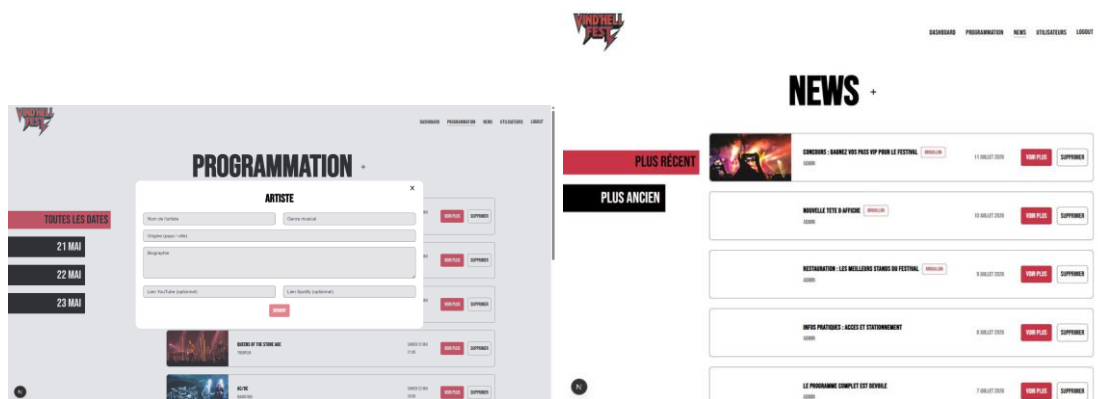


Figure — Formulaire d'ajout d'artiste et liste des news (admin) — Vindhellfest (capture réelle)

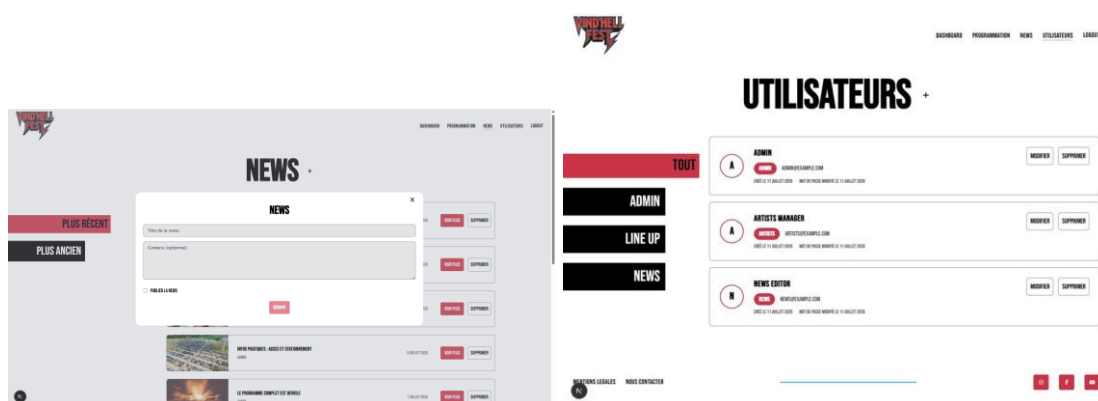


Figure — Formulaire d'ajout de news et liste des utilisateurs (admin) — Vindhellfest (capture réelle)

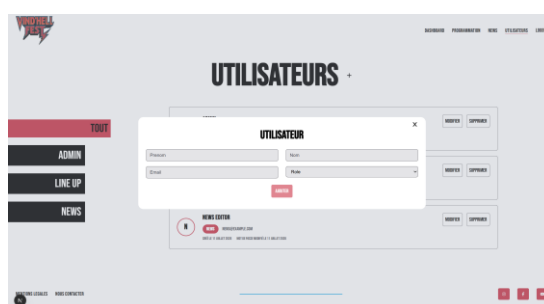


Figure — Formulaire d'ajout d'utilisateur (admin) — Vindhellfest (capture réelle)

Aperçu responsive (vue mobile)

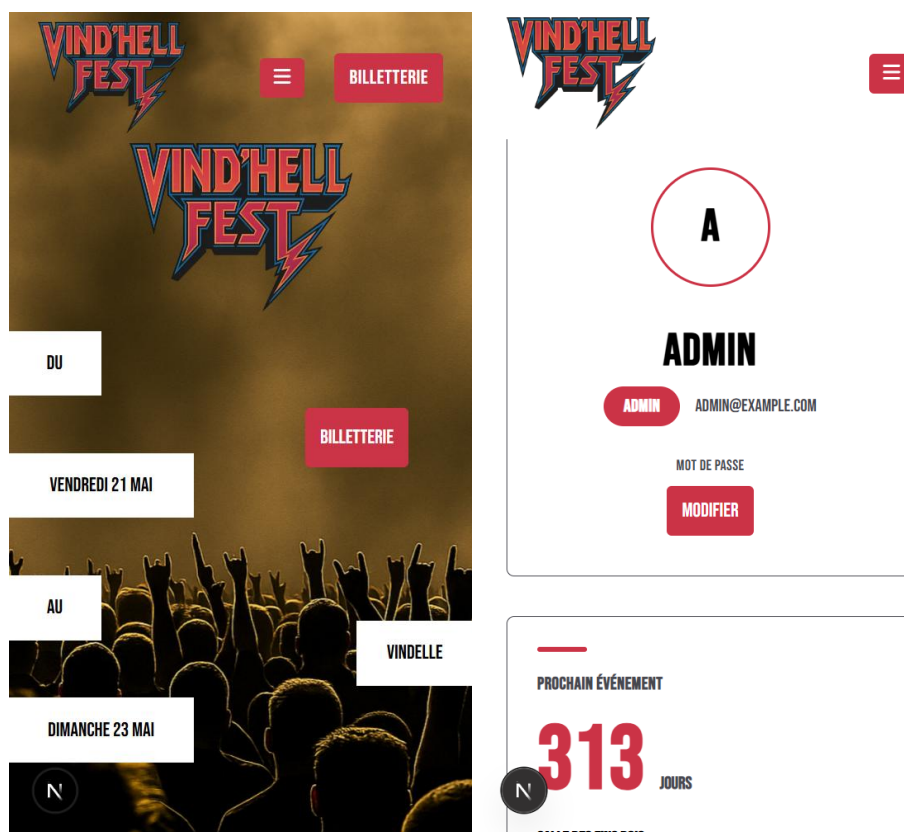


Figure — Accueil et dashboard admin, vue mobile — Vindhellfest (capture réelle)

The image shows a mobile app screen for adding a new artist. The form is titled 'ARTISTE' and includes several input fields: 'Nom de l'artiste', 'Genre musical', 'Origine (pays / ville)', 'Biographie', 'Lien YouTube (optionnel)', and 'Lien Spotify (optionnel)'. A red 'SUIVANT' button is at the bottom. The form is presented in a modal style with a close button (X) in the top right corner.

Figure — Formulaire d'ajout d'artiste, vue mobile — Vindhellfest (capture réelle)

D. Extraits de code significatifs

Extraits sélectionnés. Huit extraits illustrent les mécanismes transverses du backend et du frontend : authentification par JWT, renouvellement de session, gestion centralisée des erreurs async, validation des entrées, accès aux données côté client, gestion d'état de modale, mutation générique et protection des routes admin côté serveur. Les captures ci-dessous proviennent directement de l'éditeur (VS Code).

Middleware d'authentification — auth() (cf. V.3.3)

apps/backend/src/middlewares/auth.ts

```

/** Verifie que l'utilisateur est autorise a effectuer cette requete
 * Recupere et decode le token
 * Recherche l'utilisateur dans la BDD
 * Stocke le user et le sessionId dans res.locals pour les handlers suivants
 * @param req - la requête HTTP entrante
 * @param res - la réponse HTTP
 * @param next - la fonction de middleware suivante
 * @function getTokenFromCookie Verifie qu'un token est present dans le cookie de la requete
 * @function decodedToken Decode le token JWT pour recuperer le userId et le sessionId
 * @function query Execute une requete SQL sur la base de donnees
 */
export async function auth(req: Request, res: Response, next: NextFunction) {
  // Récupère les id depuis le token d'accès depuis les cookies,
  const token = getTokenFromCookie(req);
  const { userId, sessionId } = decodedToken(token);

  // Cherche l'utilisateur correspondant à l'userId extrait du token et renvoie son nom et son rôle
  const user = await query<AuthUserRow>(
    "SELECT id, display_name, role FROM users WHERE id = $1",
    [userId],
  );

  // Si l'utilisateur n'existe pas, on renvoie une erreur d'authentification
  if (!user[0]) {
    throw new AppError(ERRORS.AUTH_USER_NOT_FOUND, 401);
  }

  // // Stocke les infos d'authentification dans `res.locals`
  res.locals.userId = user[0].id;
  res.locals.userRole = user[0].role;
  res.locals.userDisplayName = user[0].display_name;
  res.locals.sessionId = sessionId;

  next();
}

```

Figure — Middleware auth() — apps/backend/src/middlewares/auth.ts — Vindhellfest

Ce middleware décode le token JWT depuis le cookie, recherche l'utilisateur en base et bloque la requête (401) si le token est absent, invalide ou si l'utilisateur n'existe plus, avant de peupler res.locals pour les handlers suivants.

Middleware de renouvellement de session — sessionIsOpen() (cf. V.3.3)

apps/backend/src/middlewares/sessionIsOpen.ts

```
export async function sessionIsOpen(
  _req: Request,
  res: Response,
  next: NextFunction,
) {
  // Récupère le reqSessionId: string on depuis 'res.locals'.
  const reqSessionId = requireSessionId(res.locals.sessionId);
  const reqUserId = requireUserId(res.locals.userId);

  // Recherche en base la session correspondant au identifiants
  const rows = await query<SessionRow>(
    "SELECT id, user_id, expires_at, revoked_at FROM sessions WHERE id = $1 AND user_id = $2",
    [reqSessionId, reqUserId],
  );
  const sessionBdd = rows[0];

  // Vérifie que la session existe bien et qu'elle n'a pas été révoquée.
  sessionExists(sessionBdd);
  sessionRevoked(sessionBdd);

  // Génère un nouveau token d'accès JWT puis le sérialise dans un cookie HTTP.
  const accessToken = initToken(
    reqUserId,
    "JWT_ACCESS_SECRET",
    "JWT_ACCESS_EXPIRES_IN",
    reqSessionId,
  );
  const accessCookie = serializeCookie(
    "COOKIE_ACCESS_TOKEN_NAME",
    "COOKIE_ACCESS_TOKEN_SECURE",
    "COOKIE_ACCESS_TOKEN_SAME_SITE",
    accessToken,
    "JWT_ACCESS_EXPIRES_IN",
  );

  // Répond avec le cookie
  res.setHeader("Set-Cookie", accessCookie);

  next();
}
```

Figure — Middleware sessionIsOpen() — apps/backend/src/middlewares/sessionIsOpen.ts — Vindhellfest

Ce middleware vérifie en base que la session associée au cookie existe et n'a pas été révoquée, puis régénère un token d'accès JWT et le cookie correspondant afin de prolonger la session de l'utilisateur.

Wrapper de gestion des erreurs asynchrones — asyncHandler() (cf. V.3.8)

apps/backend/src/middlewares/asyncHandler.ts

```
/** Wrappe un handler async et transfère les erreurs vers `next(error)`. */
export function asyncHandler(
  handler: (
    req: Request,
    res: Response,
    next: NextFunction,
  ) => Promise<unknown>,
): RequestHandler {
  return (req, res, next) => {
    void handler(req, res, next).catch(next);
  };
}
```

Figure — Wrapper asyncHandler() — apps/backend/src/middlewares/asyncHandler.ts — Vindhellfest

Ce wrapper générique enveloppe chaque handler async des routes Express : toute erreur levée dans la promesse est automatiquement transmise à next(), évitant de dupliquer un try/catch dans chaque contrôleur.

Validation des entrées — createArtistSchema (cf. V.3.4)

apps/backend/src/schemas/schema.ts

```
/** Schema Zod de creation et modification d'un artiste - valide les champs texte uniquement (l'image arrive via req.file).
 * Inclut les champs de programmation du concert associe (scene, heure de debut et de fin).
 * Utilise pour la creation (POST) et la modification (PATCH).
 */
export const createArtistSchema = z.object({
  name: z.string().min(2).max(100).trim(),
  genre: z.string().min(1).max(60).trim(),
  origin: z.string().min(1).max(80).trim(),
  bio: z.string().min(1).trim(),
  description_media: z.string().min(1).max(255).trim(),
  youtube_url: z
    .url()
    .max(255)
    .refine(
      (val) => /^https?:\/\/(www\.)?(youtube\.com|youtu\.be)\/.test(val),
      {
        message: "Le lien doit provenir de YouTube (youtube.com ouyoutu.be).",
      }
    )
    .optional()
    .or(z.literal("")),
  spotify_url: z
    .url()
    .max(255)
    .refine(
      (val) => /^https?:\/\/open\.spotify\.com\/.test(val), {
        message: "Le lien doit provenir de Spotify (open.spotify.com).",
      }
    )
    .optional()
    .or(z.literal("")),
  stage: z.enum(["MainStage", "Tremplin"]),
  start_time: z.iso.datetime(),
  end_time: z.iso.datetime(),
  is_featured: z.enum(["true", "false"]).optional(),
});
```

Figure — Schéma Zod createArtistSchema — apps/backend/src/schemas/schema.ts — Vindhellfest

Ce schéma Zod valide les champs texte d'un artiste (nom, genre, origine, biographie, liens YouTube et Spotify contrôlés par regex, scène, horaires ISO) avant toute création ou modification en bas, les 6 autres schémas du fichier (connexion, changement de mot de passe, réinitialisation, contact, actualité, utilisateur) suivent le même principe.

Hook personnalisé de récupération de données — useFetch() (cf. V.2.4)

apps/frontend/src/hooks/useFetch.ts

```
/** Charge des données depuis l'API au montage du composant.
 * @param {string} endpoint Chemin de l'endpoint (ex : "/public/artists").
 * @function apiRequest Fonction qui effectue la requête API et retourne une promesse avec les données ou l'erreur.
 * @function getApiErrorMessage Fonction qui récupère le message d'erreur partir de l'erreur retournée par apiRequest.
 * @returns { data: T | null, isLoading: boolean, error: string | null } - Les données chargées, l'état de chargement et l'erreur éventuelle.
 */
export function useFetch<T>(endpoint: string): {
  data: T | null;
  isLoading: boolean;
  error: string | null;
} {
  // On utilise useReducer pour gérer les états de chargement, succès et erreur de manière claire et prévisible.
  const [state, dispatch] = useReducer(
    (s: FetchState<T>, action: FetchAction<T>): FetchState<T> => {
      switch (action.type) {
        case "LOADING":
          return { ...s, isLoading: true, error: null };
        case "SUCCESS":
          return { data: action.data, isLoading: false, error: null };
        case "ERROR":
          return { ...s, isLoading: false, error: action.error };
      }
    },
    { data: null, isLoading: true, error: null },
  );

  // useEffect est utilisé pour déclencher la requête API au montage du composant et à chaque changement de l'endpoint.
  useEffect(() => {
    dispatch({ type: "LOADING" });

    // Effectue la requête API et retourne une promesse avec les données ou l'erreur.
    apiRequest<T>(endpoint).then((result) => {
      if (result.error) {
        dispatch({ type: "ERROR", error: getApiErrorMessage(result.error) });
        return;
      }

      dispatch({ type: "SUCCESS", data: result.data as T });
    });
  }, [endpoint]);

  return state;
}
```

Figure — Hook useFetch() — apps/frontend/src/hooks/useFetch.ts — Vindhellfest

Ce hook encapsule un appel à `apiRequest` via `useReducer` pour exposer un état `{ data, isLoading, error }` uniforme, évitant de dupliquer la logique de chargement, de succès et d'erreur dans chaque composant qui consomme l'API.

Hook personnalisé de gestion de modale — useModal() (cf. V.2.4)

apps/frontend/src/hooks/useModal.ts

```
/** Gère l'état d'ouverture d'une modale avec item optionnel.
 * Appeler open(item) pour ouvrir avec un élément (édition, suppression),
 * ou open() sans argument pour ouvrir sans contexte (création).
 * @param {boolean} initialOpen Ouvre la modale immédiatement si true (ex : mot de passe provisoire).
 * @return {Object} Un objet contenant :
 * - isOpen : booléen indiquant si la modale est ouverte.
 * - item : l'élément associé à la modale, ou null si aucun.
 * - open : ouvre la modale, accepte un item optionnel.
 * - close : ferme la modale et remet item à null.
 */
export function useModal<T = undefined>({
  initialOpen = false,
}): {
  isOpen: boolean;
  item: T | null;
  open: (item?: T | null) => void;
  close: () => void;
} {
  // initialise l'état de la modal
  const [isOpen, setIsOpen] = useState(initialOpen);
  const [item, setItem] = useState<T | null>(null);

  /** Ouvre la modale et associe un item optionnel.
   * @param {T | null} newItem Item à associer (édition/suppression), ou omis pour une création.
   */
  const open = (newItem?: T | null) => {
    setItem(newItem ?? null);
    setIsOpen(true);
  };

  /** Ferme la modale et remet l'item à null. */
  const close = () => {
    setIsOpen(false);
    setItem(null);
  };

  return { isOpen, item, open, close };
}
```

Figure — Hook useModal() — apps/frontend/src/hooks/useModal.ts — Vindhellfest

Ce hook gère l'état d'ouverture d'une modale ainsi que l'item optionnel associé : `open(item)` l'ouvre en contexte d'édition ou de suppression, `open()` sans argument l'ouvre pour une création, et `initialOpen` permet de l'ouvrir immédiatement (ex : mot de passe provisoire).

Hook personnalisé de mutation — useMutation() (cf. V.2.4)

apps/frontend/src/hooks/useMutation.ts

```
/** Gère l'état d'un appel API de création ou de modification.
 * @param {string} endpoint Chemin de l'endpoint (ex : "/admin/artists").
 * @param {'POST' | 'PATCH'} method Méthode HTTP à utiliser.
 * @function apiRequest Fonction utilitaire pour faire une requête API, qui retourne un objet { data, error }.
 * @function getApiErrorMessage Fonction utilitaire pour traduire un code d'erreur API en message lisible.
 * @return {Object} Un objet contenant :
 * - mutate: une fonction pour effectuer la mutation, prenant en paramètre le corps de la requête et une fonction de callback.
 * - isLoading: un booléen indiquant si la requête est en cours.
 * - error: un message d'erreur en cas d'échec de la requête, ou null si aucune erreur.
 * - reset: une fonction pour réinitialiser l'état de chargement et d'erreur (utile à la fermeture d'une modale).
 */
export function useMutation<T>({
  endpoint: string,
  method: "POST" | "PATCH",
}): {
  mutate: (
    body: FormData | Record<string, unknown> | null,
    onSuccess: (data: T) => void,
  ) => Promise<void>;

  isLoading: boolean;
  error: string | null;
  reset: () => void;
} {
  // initialisation de isLoading et error
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  /** Fonction pour effectuer la mutation
   * @param {FormData | Record<string, unknown> | null} body Corps de la requête - null si aucun body à envoyer.
   * @param {(data: T) => void} onSuccess Callback appelé avec les données en cas de succès.
   */
  const mutate = async (
    body: FormData | Record<string, unknown> | null,
    onSuccess: (data: T) => void,
  ) => {
    // réinitialisation de isLoading et error
    setIsLoading(true);
    setError(null);

    // préparation de la requête en fonction du type de body si il y en a un
    const init: RequestInit =
      body instanceof FormData
        ? { method, body }
        : body !== null
          ? {
              method,
              headers: { "Content-Type": "application/json" },
              body: JSON.stringify(body),
            }
          : { method };

    // Envoi de la requête et récupération du résultat typé
    const result = await apiRequest<T>(endpoint, init);

    // En cas d'erreur, on traduit le code d'erreur en message lisible et on arrête
    if (result.error) {
      setError(getApiErrorMessage(result.error));
      setIsLoading(false);
      return;
    }

    // Succès : on notifie le parent avec les données reçues
    onSuccess(result.data);
    setIsLoading(false);
  };

  // Fonction pour réinitialiser l'état de chargement et d'erreur (utile à la fermeture d'une modale)
  const reset = () => {
    setIsLoading(false);
    setError(null);
  };

  return { mutate, isLoading, error, reset };
}
```

Figure — Hook useMutation() (1/2) — apps/frontend/src/hooks/useMutation.ts — Vindhellfest

```
    // préparation de la requête en fonction du type de body si il y en a un
    const init: RequestInit =
      body instanceof FormData
        ? { method, body }
        : body !== null
          ? {
              method,
              headers: { "Content-Type": "application/json" },
              body: JSON.stringify(body),
            }
          : { method };

    // Envoi de la requête et récupération du résultat typé
    const result = await apiRequest<T>(endpoint, init);

    // En cas d'erreur, on traduit le code d'erreur en message lisible et on arrête
    if (result.error) {
      setError(getApiErrorMessage(result.error));
      setIsLoading(false);
      return;
    }

    // Succès : on notifie le parent avec les données reçues
    onSuccess(result.data);
    setIsLoading(false);
  };

  // Fonction pour réinitialiser l'état de chargement et d'erreur (utile à la fermeture d'une modale)
  const reset = () => {
    setIsLoading(false);
    setError(null);
  };

  return { mutate, isLoading, error, reset };
}
```

Figure — Hook useMutation() (2/2) — apps/frontend/src/hooks/useMutation.ts — Vindhellfest

Ce hook encapsule un appel API de création ou modification (POST/PATCH) via `apiRequest`, en adaptant automatiquement le corps de la requête selon son type (FormData ou JSON), et expose `{ mutate, isLoading, error, reset }` pour piloter l'état de chargement et d'erreur depuis le composant appelant.

Garde de session admin — AdminLayout (cf. V.2.7)

`apps/frontend/src/app/admin/layout.tsx`

```

/** Verifie la session via le backend avant de rendre les pages '/admin'.
 * Redirige vers '/login' si la session est absente, invalide ou si le backend est inaccessible.
 * Injecte les donnees utilisateur dans AdminUserProvider pour les rendre accessibles a toutes les pages admin
 * @children Banner
 * @children Footer
 * @children AdminUserProvider contexte
 */
export default async function AdminLayout({
  children,
}): Readonly<{
  children: React.ReactNode;
}> {
  // Definit l'URL de l'API
  const apiBaseUrl =
    process.env.API_URL_SERVER ?? process.env.NEXT_PUBLIC_API_URL;

  // Recupere les cookies de la requete en cours cote serveur et le convertit en string
  const cookieStore = await cookies();
  const cookieHeader = cookieStore.toString();

  // Verifie si l'URL de l'API est definie dans les variables d'environnement
  if (!apiBaseUrl) {
    throw new Error("Missing env var: API_URL_SERVER or NEXT_PUBLIC_API_URL");
  }

  // Verifie la session admin et redirige vers /login si la requete echoue ou renvoie une reponse non valide.
  let response: Response;
  try {
    response = await fetch(`${apiBaseUrl}/admin/auth/me`, {
      method: "GET",
      headers: { cookie: cookieHeader },
      cache: "no-store",
    });
  } catch {
    redirect("/login");
  }
  if (!response.ok) {
    redirect("/login");
  }

  // Parse les donnees utilisateur retournees par l'API
  const me = (await response.json()) as AdminAuthMeResponse;

```

Figure — AdminLayout (1/2) — `apps/frontend/src/app/admin/layout.tsx` — Vindhellfest

```

// Fournit les donnees utilisateur a toutes les pages enfants de la zone admin via le contexte
return (
  <AdminUserProvider value={me}>
    <div data-theme="admin" id="app-root" className="app-root">
      <Banner />
      <main className="flex flex-1 flex-col">{children}</main>
      <Footer />
    </div>
  </AdminUserProvider>
);

```

Figure — AdminLayout (2/2) — `apps/frontend/src/app/admin/layout.tsx` — Vindhellfest

Ce Server Component asynchrone protège l'ensemble des pages `/admin` : il transmet les cookies de la requête à `GET /admin/auth/me`, redirige vers `/login` si le backend est inaccessible, puis injecte les données utilisateur dans `AdminUserProvider` pour les rendre accessibles via `useAdminUser()`. Next.js applique automatiquement ce layout à toutes les routes admin, sans duplication de code.

E. Rapport d'exécution des tests

Suites de tests. Le projet est couvert par des tests automatisés côté backend (unitaires et intégration, avec Vitest) et côté frontend (tests de composants, hooks et fonctions utilitaires). Les captures ci-dessous montrent l'exécution complète des deux suites en local.

Backend — tests unitaires et d'intégration

```

✓ tests/integration/public/contact.test.ts (6 tests) 578ms
stdout | tests/unit/validateBody.middleware.test.ts
[dotenv@17.2.3] injecting env (0) from .env.backend -- tip: ⚙️ write to custom object with { processEnv: myObject }

✓ tests/unit/validateBody.middleware.test.ts (4 tests) 420ms
stdout | tests/unit/validateUuidParam.middleware.test.ts
[dotenv@17.2.3] injecting env (0) from .env.backend -- tip: 📖 add secrets lifecycle management: https://dotenvx.com/ops

✓ tests/unit/validateUuidParam.middleware.test.ts (4 tests) 385ms
stdout | tests/unit/requireRole.middleware.test.ts
[dotenv@17.2.3] injecting env (0) from .env.backend -- tip: 🔒 encrypt with Dotenvx: https://dotenvx.com

✓ tests/unit/requireRole.middleware.test.ts (3 tests) 354ms

Test Files 13 passed (13)
Tests 132 passed (132)
Start at 15:30:23
Duration 18.77s (transform 2.18s, setup 1.05s, import 4.13s, tests 11.81s, environment 1ms)

```

Figure — Exécution de la suite de tests backend (unitaires + intégration) — Vindhellfest

13 fichiers de tests, 132 tests passés, couvrant les middlewares, les services et les routes d'intégration publiques et admin, aucune régression.

Frontend — tests de composants, hooks et fonctions

```

✓ tests/hooks/useMutation.test.ts (5 tests) 36ms
✓ tests/hooks/useDelete.test.ts (4 tests) 40ms
✓ tests/components/AdminArtistDetailPage.test.tsx (2 tests) 423ms
  ✓ met à jour les informations affichées après une modification réussie 351ms
✓ tests/components/DeleteModal.test.tsx (4 tests) 514ms
  ✓ un clic sur Confirmer appelle handleDelete et getLabel est utilisé dans le texte 319ms
✓ tests/components/DashboardContent.test.tsx (6 tests) 538ms
  ✓ ouvre la modale au clic sur le bouton 'Modifier' 354ms
✓ tests/components/NewsContent.test.tsx (8 tests) 708ms
  ✓ ajoute la news à la liste localement après un ajout réussi 320ms
✓ tests/components/ArtistsContent.test.tsx (6 tests) 717ms
  ✓ ajoute l'artiste à la liste localement après un ajout réussi 396ms
✓ tests/components/LoginPage.test.tsx (5 tests) 1032ms
  ✓ désactive le bouton 'Envoyer' si email ou mot de passe est vide 396ms
  ✓ redirige vers /admin/dashboard après connexion réussie 461ms
✓ tests/components/NewsPage.test.tsx (2 tests) 746ms
  ✓ inverse l'ordre de la liste après clic sur le filtre 'Plus ancien' 593ms
✓ tests/components/ArtistsPage.test.tsx (2 tests) 774ms
  ✓ filtre les artistes par date après clic sur un filtre de jour 614ms
✓ tests/components/UsersPage.test.tsx (2 tests) 791ms
  ✓ filtre les utilisateurs par rôle après clic sur un filtre 617ms
✓ tests/components/UsersContent.test.tsx (7 tests) 1110ms
  ✓ ajoute l'utilisateur à la liste localement après un ajout réussi 368ms
✓ tests/components/AddNewsModal.test.tsx (5 tests) 1235ms
  ✓ le bouton de progression est désactivé à chaque étape si les champs requis sont vides 910ms
✓ tests/components/ChangePasswordModal.test.tsx (7 tests) 1864ms
  ✓ désactive le bouton 'Modifier' si les champs sont vides 365ms
  ✓ affiche une erreur locale si les mots de passe ne correspondent pas 528ms
  ✓ affiche le message de succès et masque le formulaire après changement réussi 459ms
  ✓ affiche le bouton 'Continuer' après succès en mode forced 324ms
✓ tests/components/AddUserModal.test.tsx (4 tests) 317ms
✓ tests/components/AddArtistModal.test.tsx (7 tests) 2394ms
  ✓ le bouton de progression est désactivé à chaque étape si les champs requis sont vides 1262ms
  ✓ affiche une erreur si l'URL YouTube ne correspond pas au domaine attendu 434ms
  ✓ affiche une erreur si l'URL Spotify ne correspond pas au domaine attendu 365ms
✓ tests/components/AdminNewsDetailPage.test.tsx (2 tests) 179ms
✓ tests/hooks/useRoleGuard.test.ts (4 tests) 22ms
✓ tests/functions/validation.test.ts (3 tests) 4ms
✓ tests/functions/getApiErrorMessage.test.ts (6 tests) 5ms
✓ tests/functions/formatDate.test.ts (5 tests) 26ms
✓ tests/functions/filterNavByRole.test.ts (4 tests) 9ms
✓ tests/hooks/useFetch.test.ts (3 tests) 127ms
✓ tests/components/ContactUs.test.tsx (3 tests) 536ms
  ✓ affiche un message de succès et masque le formulaire après envoi réussi 325ms
✓ tests/components/ForgotPassword.test.tsx (3 tests) 310ms
✓ tests/components/Footer.test.tsx (3 tests) 306ms

Test Files 26 passed (26)
Tests 109 passed (109)
Start at 15:31:46
Duration 7.72s (transform 30.31s, setup 4.87s, import 47.15s, tests 14.76s, environment 30.49s)

```

Figure — Exécution de la suite de tests frontend — Vindhellfest

26 fichiers de tests, 109 tests passés, couvrant les composants d'administration (artistes, actualités, utilisateurs), les modales, les hooks personnalisés (useFetch, useMutation, useDelete, useRoleGuard) et les fonctions utilitaires (validation, formatDate, gestion des erreurs API).

Chaîne de gestion d'erreur frontend — useMutation() (cf. V.2.8)

Deux captures de la console navigateur montrent la valeur réellement retournée par `apiRequest()` lors d'une tentative de connexion, selon que les identifiants sont valides ou non.

```
▼ {data: {...}, error: null} ⓘ
  ▼ data:
    message: "Authentication réussie"
    ▶ [[Prototype]]: Object
    error: null
    ▶ [[Prototype]]: Object
```

Figure — Connexion réussie : `{ data: { message }, error: null }` — Vindhellfest

```
▼ {data: null, error: ApiRequestError: Email ou mot de passe incorrect
  at apiRequest (webpack-internal:///...)} ⓘ
  data: null
  ▼ error: ApiRequestError: Email ou mot de passe incorrect at apiRequest
    name: "ApiRequestError"
    status: 401
    message: "Email ou mot de passe incorrect"
```

Figure — Mot de passe incorrect : `{ data: null, error: ApiRequestError }` — Vindhellfest

Les deux captures illustrent l'union discriminante `{ data: T, error: null } | { data: null, error: ApiRequestError }` définie dans `apiRequest.ts`

Zoom sur le jeu d'essai IX.6 — Création d'un artiste avec image

Exécutions réelles. Les captures ci-dessous montrent l'exécution effective (terminal Vitest) de plusieurs cas du jeu d'essai présenté en IX.6, dont le cas limite (409, limite de 2 artistes mis en avant) et le contrôle de présence de l'image (400).

```
stderr | tests/integration/admin/artists.test.ts > POST /admin/artists > retourne 409 si la limite de 2 artistes en avant est atteinte
AppError: Deux artistes sont déjà mis en avant sur la page d'accueil.
    at createArtist (/app/src/controllers/admin/artists/create_artist.controller.ts:108:13)
    at processTicksAndRejections (node:internal/process/task_queues:95:5) {
  status: 409
}
```

Figure — Exécution réelle du cas limite IX.6 : `POST /admin/artists` renvoie 409 à la limite de 2 artistes mis en avant — Vindhellfest

```
stderr | tests/integration/admin/artists.test.ts > POST /admin/artists > retourne 400 si aucune image n'est fournie
AppError: Image requise
    at createArtist (/app/src/controllers/admin/artists/create_artist.controller.ts:22:11)
    at /app/src/middlewares/asyncHandler.ts:12:10
    at Layer.handleRequest (/app/node_modules/router/lib/layer.js:152:17)
    at next (/app/node_modules/router/lib/route.js:157:13)
    at /app/src/middlewares/validateBody.ts:21:12
    at Layer.handleRequest (/app/node_modules/router/lib/layer.js:152:17)
    at next (/app/node_modules/router/lib/route.js:157:13)
    at done (/app/node_modules/multer/lib/make-middleware.js:70:7)
    at indicateDone (/app/node_modules/multer/lib/make-middleware.js:74:68)
    at Multipart.<anonymous> (/app/node_modules/multer/lib/make-middleware.js:258:7) {
  status: 400
}
```

Figure — Exécution réelle : `POST /admin/artists` renvoie 400 si aucune image n'est fournie — Vindhellfest

```
✓ tests/components/AddArtistModal.test.tsx (7 tests) 2745ms
  ✓ le bouton de progression est desactive a chaque etape si les champs requis sont vides 1265ms
  ✓ affiche une erreur si l'URL YouTube ne correspond pas au domaine attendu 548ms
  ✓ affiche une erreur si l'URL Spotify ne correspond pas au domaine attendu 413ms
  ✓ affiche le message d'erreur API sur l'etape 3 si useMutation retourne une erreur 357ms
```

Figure — Tests frontend du composant `AddArtistModal` (7 tests), même fonctionnalité de création d'artiste — Vindhellfest

F. Configuration DevOps

Configuration DevOps. L'ensemble du projet (frontend, backend, base de données) est orchestré via Docker Compose en développement, avec des images multi-stage pour la production, et testé automatiquement à chaque push via une pipeline GitHub Actions.

Orchestration des services — docker-compose.yml

```
version: "3.9"

services:
  db: ...
  backend: ...
  frontend: ...

networks:
  app-net: # Déclare un réseau Docker interne nommé app-net

volumes:
  pgdata: # Volume persistant pour les données PostgreSQL
```

Figure — Vue d'ensemble des services — docker-compose.yml — Vindhellfest

Trois services (*db*, *backend*, *frontend*) plus un réseau interne *app-net* et un volume persistant *pgdata* pour les données PostgreSQL.

```
db:
  image: postgres:16-alpine
  container_name: vindhellfest-db
  restart: always
  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data # volumes persistant
    - ./bd/init:/docker-entrypoint-initdb.d # Pour les scripts d'initialisation
  networks:
    - app-net # Permet au backend de l'appeler via le hostname db
  healthcheck: # Toutes les 10 secondes Eviter que le backend démarre avant que la BDD soit prête
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Figure — Service db — docker-compose.yml — Vindhellfest

Le service db utilise l'image postgres:16-alpine, monte un volume persistant et le script d'initialisation SQL, et expose un healthcheck qui retarde le démarrage du backend jusqu'à ce que la base soit prête.

```

>Run Service
backend:
  build:
    context: ./apps/backend
    dockerfile: Dockerfile
    target: dev
  container_name: vindhellfest-backend
  restart: always
  env_file:
    - .env
    - .env.backend
  depends_on:
    db:
      condition: service_healthy
  ports:
    - "4000:4000"
  volumes:
    - ./apps/backend:/app
    - /app/node_modules
    - ./apps/backend/uploads:/app/uploads # Volume persistant pour les images uploadées
    - ./bd/init:/app/bd/init # Migrations SQL accessibles pour les tests
  networks:
    - app-net

```

Figure — Service backend — docker-compose.yml — Vindhellfest

Le service backend attend que db soit healthy avant de démarrer, monte le code source et expose le port 4000.

```

>Run Service
✓ frontend:
  build:
    context: ./apps/frontend
    dockerfile: Dockerfile
    target: dev
  container_name: vindhellfest-frontend
  restart: always
  env_file:
    - .env
    - .env.frontend
  > environment: # env variables pour forcer le polling (utile dans Docker)...
  command: ["npm", "run", "dev", "--", "--webpack", "-H", "0.0.0.0", "-p", "3000"] # run dev avec webpack
  ✓ depends_on:
    - backend
  ✓ ports:
    - "3000:3000"
  ✓ volumes:
    - ./apps/frontend:/app
    - /app/node_modules
    - /app/.next
  ✓ networks:
    - app-net

✓ networks:
  app-net: # Déclare un réseau Docker interne nommé app-net

✓ volumes:
  pgdata: # Volume persistant pour les données PostgreSQL

```

Figure — Service frontend — docker-compose.yml — Vindhellfest

Le service frontend force le polling via des variables d'environnement (utile dans Docker pour détecter les changements de fichiers), monte le code et expose le port 3000.

Image Docker du backend — build multi-stage

```
> # --- 1 IMAGE de BUILD --- ...
FROM node:20-alpine AS builder

# Répertoire de travail à l'intérieur du conteneur
WORKDIR /app

# Copier les fichiers de dépendances et install (inclut devDependencies par défaut)
COPY package*.json ./
RUN npm install

# Copier le code source et la configuration TS.
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

# --- 2 IMAGE DE PRODUCTION --- ...
FROM node:20-alpine AS runner

# Répertoire de travail à l'intérieur du conteneur
WORKDIR /app
ENV NODE_ENV=production

# Copier les fichiers de dépendances de production et les installe
COPY package*.json ./
RUN npm install --only=production

# Copier les fichiers compilés depuis le build
COPY --from=builder /app/dist ./dist

# Ecoute sur le port 4000 et lance l'application quand le conteneur démarre
EXPOSE 4000
CMD ["node", "dist/index.js"]

# --- 3 IMAGE DE DEV --- ...
FROM node:20-alpine AS dev

# Répertoire de travail à l'intérieur du conteneur
WORKDIR /app
ENV NODE_ENV=development

# Copier les fichiers de dépendances et les installe
COPY package*.json ./
RUN npm install

# Copier les fichiers compilés depuis le build
COPY tsconfig.json ./
COPY src ./src

# Ecoute sur le port 4000 et lance l'application quand le conteneur démarre
EXPOSE 4000
CMD ["npm", "run", "dev"]
```

Figure — Dockerfile backend (builder / runner / dev) — apps/backend/Dockerfile — Vindhellfest

Trois étapes : builder compile le TypeScript, runner ne conserve que les dépendances de production et le code compilé pour une image légère, et dev pour le développement local.

Image Docker du frontend — build multi-stage

apps/frontend/Dockerfile

```
# --- 1 IMAGE DE BASE ---
# On choisit une image de base
FROM node:20-alpine AS base

# Répertoire de travail à l'intérieur du conteneur et suppression de la telemetry
WORKDIR /app
ENV NEXT_TELEMETRY_DISABLED=1

# --- 2 IMAGE DE DEPENDANCES
# installation des dépendances
FROM base AS deps
COPY package.json package-lock.json ./
RUN npm ci

# --- 3 IMAGE DE BUILD ---
# Compilation de l'application Next.js, copie de dépendances de l'étape deps et build le projet
FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

# --- 4 IMAGE DE PRODUCTION ---
# installation des dépendances sans les dep de dev et d'un environnement de production
FROM base AS runner
ENV NODE_ENV=production
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Copie seulement les fichier utiliser pour la prod dans l'image de prod
COPY --from=builder /app/.next ./next
COPY --from=builder /app/public ./public
COPY --from=builder /app/next.config.ts ./next.config.ts

# Ecoute sur le port 3000 et lance l'application quand le conteneur démarre
EXPOSE 3000
CMD ["npm", "run", "start"]

# --- 4 IMAGE DE DEV ---
# installation des dépendances sans les dep de dev et d'un environnement de production
FROM base AS dev
ENV NODE_ENV=development
COPY package.json package-lock.json ./
RUN npm ci
# Copie tout les fichier dans l'image de dev
COPY . .

# Ecoute sur le port 3000 et lance l'application quand le conteneur démarre
EXPOSE 3000
CMD ["npm", "run", "dev"]
```

Figure — Dockerfile frontend (base / deps / builder / runner / dev) — apps/frontend/Dockerfile — Vindhellfest

Le build Next.js sépare l'installation des dépendances, la compilation et l'exécution en production afin de limiter la taille de l'image finale, l'étape dev reproduit l'environnement complet pour le développement.

Pipeline d'intégration continue — GitHub Actions

Le détail du déclenchement et du contenu de chaque job est donné en V.6.

```
name: CI
# Nom du workflow tel qu'il apparaît dans GitHub Actions

on:
# Définit quand le workflow se déclenche

  push:    # Se déclenche à chaque push
    branches:    # Sur toutes les branches
      - "*"

  pull_request:    # Se déclenche aussi lors d'une Pull Request

    branches:    # Peu importe la branche cible de la PR
      - "*"

jobs:
# Contient tous les jobs du workflow

> backend: ...

> frontend: ...
```

Figure — Déclencheurs et jobs du workflow CI — `.github/workflows/ci.yml` — Vindhellfest

```
# Construit l'image Docker du backend
- name: Build backend image
  run: docker compose build backend

# Lance la base de données en arrière-plan
- name: Start database
  run: docker compose up -d db

# Attend que la BDD soit prête
- name: Wait for database
  run: |
    for i in {1..20}; do
      docker compose exec -T db pg_isready -U postgres -d vindhellfest && exit 0
      sleep 3
    done
    echo "Postgres did not become ready in time" >&2
    exit 1

# Crée la base de données de test
- name: Create test database
  run: docker compose exec -T db psql -U postgres -c "CREATE DATABASE vindhellfest_test;"

# Installe les dépendances npm
- name: Install dependencies
  run: docker compose run --rm backend npm ci

# Vérifie la qualité du code avec lint
- name: Lint
  run: docker compose run --rm backend npm run lint

# Lance les tests automatisés
- name: Test
  run: docker compose run --rm backend npm test

# Arrête et supprime les conteneurs, réseaux et volumes créés
- name: Shutdown
  if: always()
  run: docker compose down -v
```

Figure — Job backend, étapes détaillées — `.github/workflows/ci.yml` — Vindhellfest


```

# Clone le repo dans la VM
- name: Checkout...

# Créer les fichiers .env nécessaires au projet.
# .env et .env.backend sont créés pour éviter des erreurs de fichier manquant
# .env.frontend pouras être complété au fur et à mesure du développement
- name: Prepare env file...

# Construit l'image Docker du frontend
- name: Build frontend image
  run: docker compose build frontend

# Installe les dépendances npm
- name: Install dependencies
  run: docker compose run --rm --no-deps frontend npm ci

# Vérifie la qualité du code avec Lint
- name: Lint
  run: docker compose run --rm --no-deps frontend npm run lint

# Lance les tests automatisé
- name: Test
  run: docker compose run --rm --no-deps frontend npm run test:run

# Arrête et supprime les conteneurs
- name: Shutdown
  if: always()
  run: docker compose down -v

```

Figure — Job frontend, étapes détaillées — `.github/workflows/ci.yml` — Vindhellfest

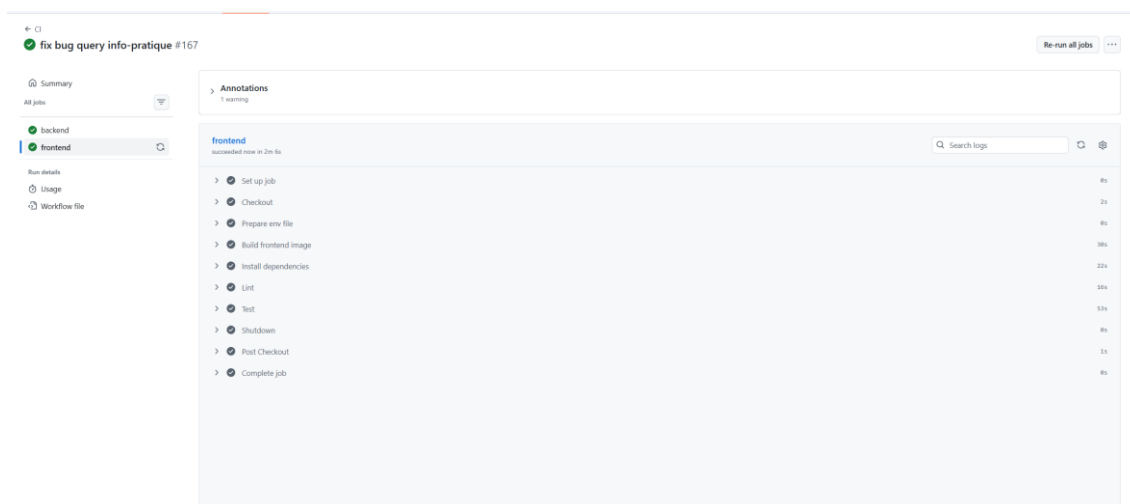


Figure — Exécution réussie des deux jobs sur GitHub Actions — Vindhellfest

Capture de l'onglet Actions du dépôt confirmant le succès des jobs backend et frontend sur un push réel (workflow entièrement vert).

Procédure de déploiement en production (proposée)

Pour automatiser le déploiement de l'application, une chaîne d'intégration et de déploiement continu (CI/CD) est envisagée, s'appuyant sur un runner auto-hébergé installé directement sur la machine virtuelle Debian destinée à la production. Ce workflow de déploiement serait distinct du pipeline `ci.yml` déjà en place (V.6, IX.5), dédié au lint et aux tests sur toutes les branches.

Le principe est le suivant : lorsqu'une pull request est acceptée et fusionnée sur la branche main, un workflow se déclenche automatiquement. Celui-ci est exécuté par un runner installé sur la VM elle-même, ce qui lui permet d'agir directement sur la

machine sans nécessiter de connexion distante (SSH). Le déclenchement est volontairement restreint aux événements de push sur main, et non aux pull requests elles-mêmes, afin d'éviter qu'un code externe non validé ne s'exécute directement sur l'environnement de production.

L'application étant conteneurisée avec Docker, la VM Debian doit disposer de Docker. Le workflow effectue un git pull pour récupérer la dernière version du code présente sur main, puis reconstruit l'image Docker de l'application et redémarre le conteneur associé (`docker compose up -d --build`). Cette approche provoque une brève coupure de service le temps de la reconstruction et du redémarrage, un déploiement sans interruption constituerait une évolution possible.

La partie exposition au trafic web est assurée par un reverse proxy Nginx, exécuté dans son propre conteneur, indépendant du cycle de vie de l'application. Nginx et le conteneur applicatif communiquent via un réseau Docker partagé, ce qui permet au proxy de rediriger les requêtes vers l'application en utilisant directement son nom de conteneur comme nom d'hôte, sans exposer son port directement sur la machine hôte.

G. Backlog produit — vue Kanban

Backlog produit. Le backlog produit est géré dans Notion sous forme de tableau kanban, avec cinq statuts possibles (À faire, En cours, En révision, Terminé, Bloqué). Le tableau suivant présente la synthèse des 32 user stories du projet, exportée depuis Notion le 12/07/2026 : couche fonctionnelle, couche technique concernée, priorité (P1 à P3), complexité estimée et état d'avancement.

User story	Couche fonctionnelle	Couche technique	Priorité	Compl.	État
Affichage de la page d'accueil	Présentation	Frontend	P1	2	Terminé
Déconnexion d'un utilisateur	Authentification	Backend, Frontend	P1	1	Terminé
Gestion des comptes utilisateurs (admin)	Administration	Backend, Frontend	P1	4	Terminé
Connexion d'un utilisateur	Authentification	Backend, Frontend	P1	3	Terminé
Upload d'une image	Administration	Backend, Frontend	P1	3	Terminé
Gestion des artistes (admin)	Administration	Backend, Frontend	P1	5	Terminé
Consultation du programme du festival	Programme & Artistes	Frontend	P1	3	Terminé
Tests d'intégration des endpoints API	Tests	Qualité & Docs	P1	5	Terminé
Pipeline CI/CD avec GitHub Actions	DevOps	Infrastructure	P2	3	Terminé
Consultation de la fiche d'un artiste	Programme & Artistes	Frontend	P1	2	Terminé
Accessibilité et responsive design	Présentation	Frontend	P2	3	Terminé
Tests unitaires des services backend	Tests	Qualité & Docs	P1	5	Terminé
Accès à l'espace d'administration	Administration	Backend, Frontend	P1	2	Terminé
Envoi d'un message de contact	Contact	Backend, Frontend	P2	3	Terminé
Containerisation avec Docker Compose	DevOps	Infrastructure	P1	3	Terminé
Consultation des news du festival	Présentation	Frontend	P2	2	Terminé
Lire une news du festival	Présentation	Frontend	P2	2	Terminé
Affichage de la liste des utilisateurs (admin)	Administration	Backend, Frontend	P1	2	Terminé
Gestion des news (admin)	Administration	Backend, Frontend	P2	4	Terminé
Consultation de la page infos pratiques	Présentation	Frontend	P2	2	Terminé
Conception et initialisation de la base de données	DevOps	Base de données	P1	3	Terminé
SEO par page (generateMetadata)	Présentation	Frontend	P2	3	Terminé
Directive robots/noindex sur les pages admin	Administration	Frontend	P2	2	À faire
Archives des éditions passées	Présentation	Backend, Base de données, Frontend	P3	5	À faire
Changement de mot de passe forcé	Authentification	Backend, Frontend	P1	3	Terminé
Balises Open Graph	Présentation	Frontend	P3	2	En cours
sitemap.xml et robots.txt	Présentation	Frontend	P3	1	À faire
Page 404 personnalisée	Présentation	Frontend	P3	1	Terminé
Mot de passe oublié	Authentification	Backend, Frontend	P1	3	Terminé
Rate limiting sur la connexion	Authentification	Backend	P1	2	Terminé
Pagination programmation et actualités	Programme & Artistes	Backend, Frontend	P3	3	À faire
Lien de définition de mot de passe (création de compte)	Authentification	Backend, Base de données, Frontend	P3	4	À faire

État d'avancement au moment de la rédaction : 26 user stories terminées, 1 en cours, 5 à faire, 0 bloquée(s). Répartition par priorité : 17 P1, 9 P2, 6 P3. Les items restants (SEO/robots.txt, pagination, archives des éditions passées, lien de définition

de mot de passe) sont des évolutions de priorité P2/P3 non bloquantes pour le périmètre fonctionnel principal du festival.

Vue Notion — table et tableau kanban

ID	Statut	Priorité	User Story	État	Couche technique
1	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
2	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
3	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
4	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
5	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
6	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
7	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
8	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
9	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
10	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
11	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
12	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
13	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
14	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
15	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
16	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
17	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
18	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
19	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
20	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
21	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
22	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
23	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
24	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
25	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
26	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
27	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
28	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
29	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend
30	À faire	P1	Écrire un administrateur contenu pour gérer les contenus	En cours	Backend

Figure — Product Backlog, vue table par défaut dans Notion — Vindhellfest

Vue table complète du backlog dans Notion, avec les colonnes complexité, couche fonctionnelle, priorité, user story, état et couche technique correspondant au tableau de synthèse ci-dessus.

Statut	ID	Priorité	User Story	Couche technique
À faire	1	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	2	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	3	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	4	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	5	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	6	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	7	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	8	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	9	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	10	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	11	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	12	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	13	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	14	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	15	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	16	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	17	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	18	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	19	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	20	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	21	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	22	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	23	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	24	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	25	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	26	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	27	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	28	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	29	P1	Écrire un administrateur contenu pour gérer les contenus	Backend
À faire	30	P1	Écrire un administrateur contenu pour gérer les contenus	Backend

Figure — Product Backlog, vue Kanban dans Notion — Vindhellfest

Vue Kanban du même backlog, organisée par statut (À faire, En cours, En révision, Terminé, Bloqué), utilisée au quotidien pour piloter l'avancement du développement.

H. Audits Lighthouse et WAVE

Captures réelles. Les captures ci-dessous documentent les résultats détaillés cités en VI.10 (Accessibilité et RGAA) : audit Lighthouse sur le build de production (npm run build && npm run start) — score 100/100 sur les quatre catégories — et test WAVE (WebAIM) sur la page d'accueil.

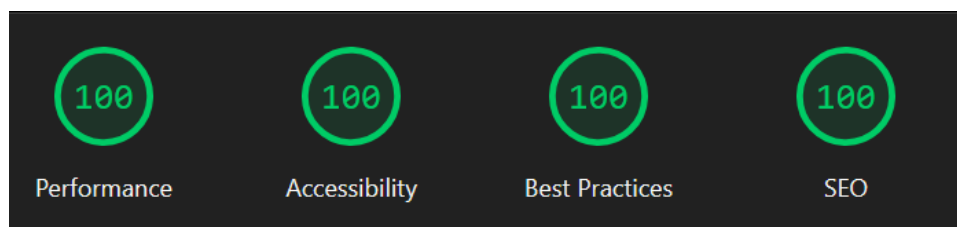


Figure — Lighthouse (build de production) — synthèse des 4 catégories : Performance 100, Accessibility 100, Best Practices 100, SEO 100 — Vindhellfest

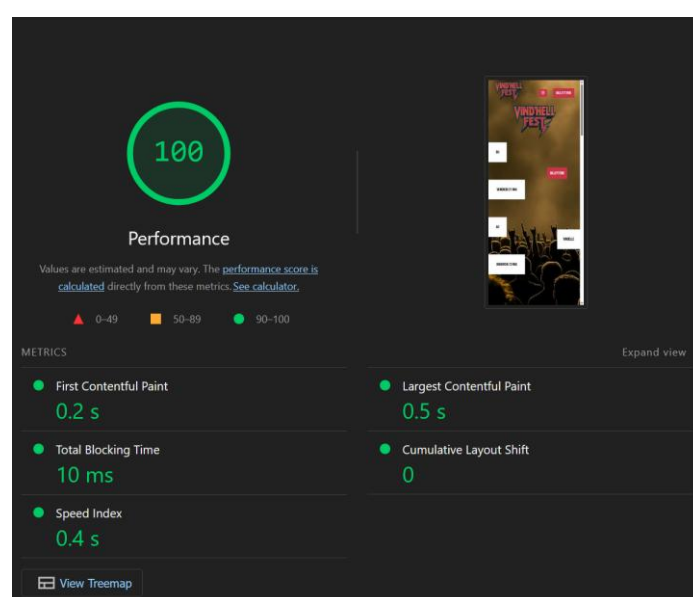


Figure — Lighthouse — détail Performance (100/100) : FCP 0,2 s, LCP 0,5 s, TBT 10 ms, CLS 0, Speed Index 0,4 s — Vindhellfest

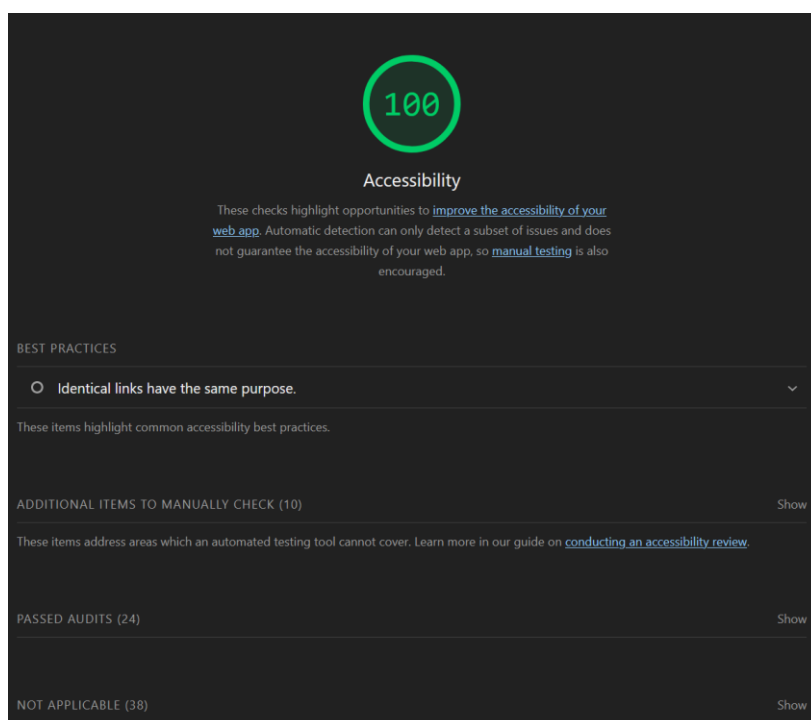


Figure — Lighthouse — détail Accessibility (100/100), dont le point informatif WCAG 2.4.9 sur les liens identiques — Vindhellfest

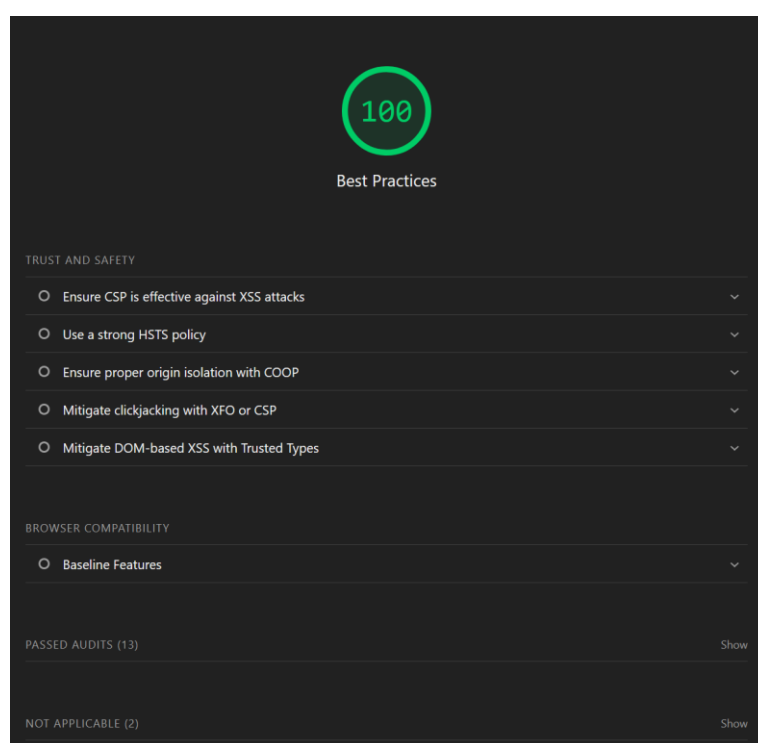


Figure — Lighthouse — détail Best Practices (100/100) — Vindhellfest

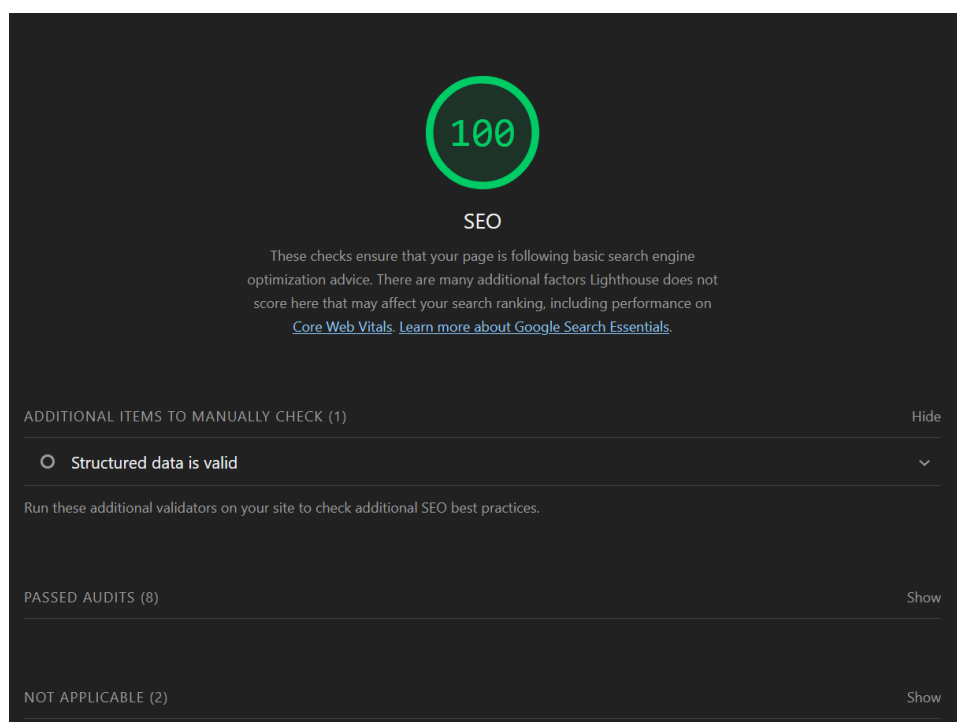


Figure — Lighthouse — détail SEO (100/100) — Vindhellfest

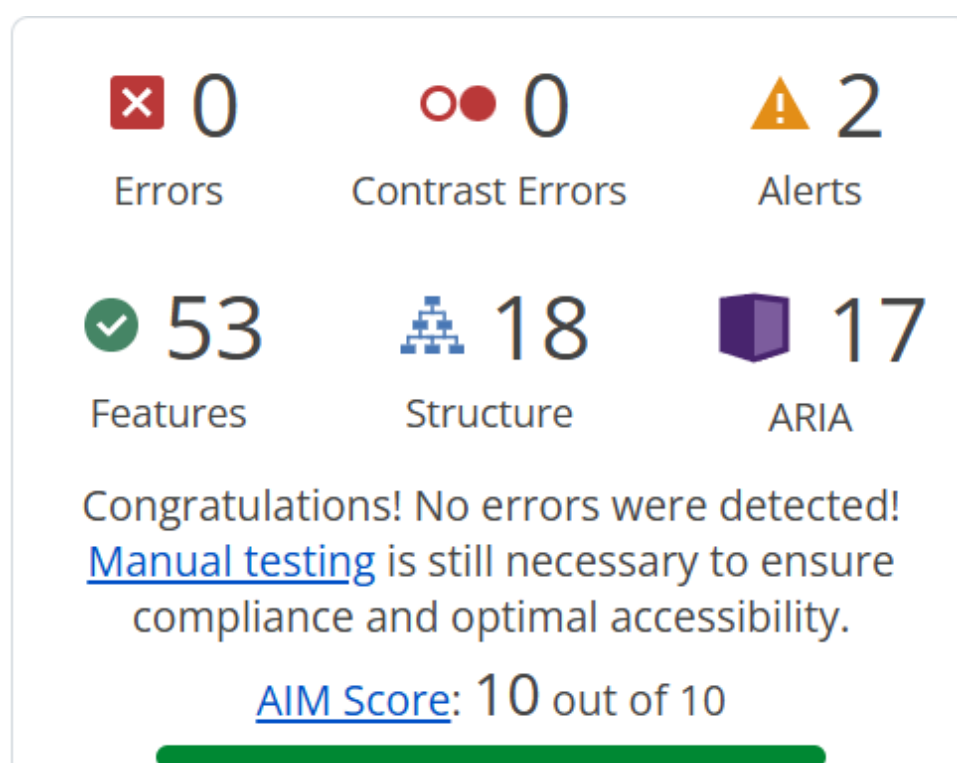


Figure — Test WAVE (WebAIM) — 0 erreur, 0 erreur de contraste, 2 alertes, AIM Score 10/10 — Vindhellfest

I. Diagramme de séquence

Diagramme de séquence. Illustration du scénario le plus représentatif du projet (cf. IV.1.2), la création d'un artiste avec upload d'image (POST /admin/artists), depuis la requête initiale jusqu'à la réponse, y compris le cas d'échec (rollback de la transaction et suppression de l'image déjà écrite sur le disque).

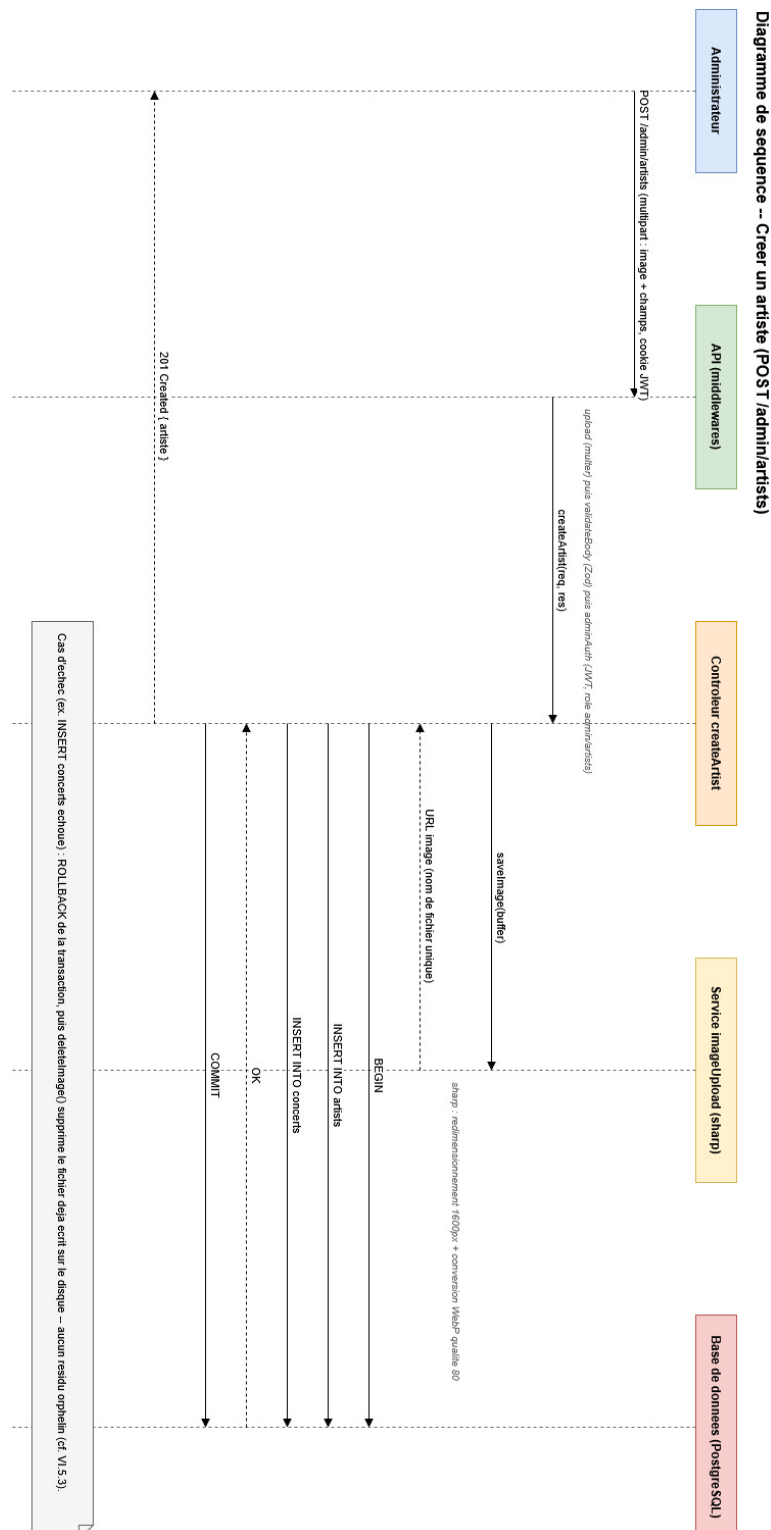


Figure — Diagramme de séquence — Création d'un artiste (POST /admin/artists) — Vindhellfest